

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

**Génération de trajectoire robotique sans collision dans un environnement
dynamique utilisant des modèles de Flow Matching**

FRÉDÉRICK LANTHIER

Département de génie mécanique

Mémoire présenté en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
Génie mécanique

Septembre 2026

POLYTECHNIQUE MONTRÉAL

affiliée à l'Université de Montréal

Ce mémoire intitulé :

**Génération de trajectoire robotique sans collision dans un environnement
dynamique utilisant des modèles de Flow Matching**

présenté par **Frédéric LANTHIER**

en vue de l'obtention du diplôme de *Maîtrise ès sciences appliquées*
a été dûment accepté par le jury d'examen constitué de :

Danielle DUBOIS, présidente

Sofiane ACHICHE, membre et directeur de recherche

Jérôme LE NY, membre et codirectrice de recherche

Jean TREMBLAY, membre

Joseph BROWN, membre externe

DÉDICACE

*À ma blonde que j'aime, mes parents,
et mes amis des labs...*

REMERCIEMENTS

Texte / Text.

RÉSUMÉ

Le résumé est un bref exposé du sujet traité, des objectifs visés, des hypothèses émises, des méthodes expérimentales utilisées et de l'analyse des résultats obtenus. On y présente également les principales conclusions de la recherche ainsi que ses applications éventuelles. En général, un résumé ne dépasse pas trois pages.

Le résumé doit donner une idée exacte du contenu du mémoire ou de la thèse. Ce ne peut pas être une simple énumération des parties du manuscrit. Le but est de présenter de façon précise et concise la nature, l'envergure de la recherche, les sujets traités, les questions de recherche ou les hypothèses soulevées, les méthodes utilisées, les principaux résultats ainsi que les conclusions retenues. Un résumé ne doit jamais comporter de références ou de figures.

ABSTRACT

Written in English, the abstract is a brief summary similar to the previous section (Résumé). However, this section is not a word for word translation of the abstract in French.

The abstract is a brief statement of the subject matter, objectives, research questions or hypotheses, experimental methods and analysis of results. It also presents the main research conclusions and their possible applications. In general, an abstract should not exceed three pages.

The abstract should provide an exact idea of the thesis or dissertation's contents and it cannot be a simple enumeration of the manuscript's parts. The goal is to precisely and concisely present the nature and scope of the research. An abstract should never include references or figures. If the thesis or the dissertation is in English, the résumé (French-language abstract) should come first followed by the abstract.

TABLE DES MATIÈRES

DÉDICACE	iv
REMERCIEMENTS	v
RÉSUMÉ	vi
ABSTRACT	vii
LISTE DES TABLEAUX	ix
LISTE DES FIGURES	x
LISTE DES SIGLES ET ABRÉVIATIONS	xi
LISTE DES ANNEXES	xii
CHAPITRE 1 INTRODUCTION	1
CHAPITRE 2 REVUE DE LITTÉRATURE	3
2.1 Génération de trajectoires robotisées	3
2.1.1 Planification classique par échantillonnage et optimisation	3
2.1.2 Apprentissage par démonstration, clonage de comportement et renforcement	4
2.1.3 Modèles génératifs, politiques de diffusion et Flow Matching	5
2.2 Perception visuelle et mémoire spatiale	7
2.2.1 Vision-langage, segmentation zéro-shot et suivi visuel	7
2.2.2 Reconstruction RGB-D et représentation par nuages de points	8
2.2.3 Cartes persistantes face aux occlusions et pertes de détection	9
2.3 Représentations géométriques pour la collision	9
2.3.1 Champs de distance de la scène et limites computationnelles	10
2.3.2 Primitives conservatives et modèles géométriques simplifiés	11
2.3.3 Champs de distance signée du robot	11
2.4 Sécurité réactive et fonctions de contrôle barrières	12
2.4.1 Champs de potentiel et méthodes réactives classiques	13
2.4.2 Fonctions barrières de contrôle et invariance avant	13
2.4.3 Filtres de sécurité prédictifs et méthodes d’atteignabilité	14
2.4.4 Limites des CBF sous perception bruitée et contrôle discret	15

2.5	Couplage entre modèles génératifs et filtres de sécurité	16
2.5.1	Contraintes intégrées à la génération	16
2.5.2	Filtres de sécurité appliqués à l'exécution	17
2.5.3	Découplage entre intention nominale et sécurité réactive	17
2.6	Synthèse critique et positionnement	18
2.6.1	Lacunes identifiées dans la littérature	18
2.6.2	Positionnement de l'architecture proposée	18
2.6.3	Hypothèse de recherche	19
CHAPITRE 3 MOTIVATION DE RECHERCHE ET OBJECTIFS		21
3.1	Positionnement de recherche et motivation	21
3.2	Objectifs de recherche	21
3.2.1	Objectif principal	21
3.2.2	Objectifs spécifiques	21
3.3	Portée et exclusions de la recherche	22
CHAPITRE 4 DESIGN DU SYSTÈME		23
4.1	Plateforme Expérimentale Et Architecture Globale	23
4.1.1	Scène robotique, capteur RGB-D, robot manipulateur	23
4.1.2	Architecture modulaire du système	23
4.2	Perception Sémantique et Reconstruction 3D	23
4.2.1	Extraction Sémantique	23
4.2.2	Suivi temporel de la cible	25
4.2.3	Projection RGB-D vers l'espace 3D	26
4.2.4	Séparation cible, obstacles et outil visible	26
4.2.5	Nettoyage spatial du nuage d'obstacles	26
4.3	Cartes Persistantes de la scène	26
4.3.1	Motivation : robustesse aux pertes de détection et occlusion	26
4.3.2	Génération des cartes persistantes	26
4.3.3	Mise à jour des cartes persistantes	26
4.4	Acquisition des données et représentation des démonstrations	26
4.4.1	Protocole de collecte des démonstrations	26
4.4.2	Prétraitement des données	30
4.5	Génération de trajectoire par Flow Matching	33
4.5.1	Formulation mathématique du Flow Matching conditionnel	33
4.5.2	Architecture du modèle	35
4.5.3	Entraînement du modèle	37

4.6	Conversion et exécution de la trajectoire nominale	38
4.6.1	Conversion outil–TCP	38
4.6.2	Cinématique inverse pour obtenir la trajectoire articulaire nominale .	40
4.6.3	Réajustement temporel de la trajectoire articulaire	41
4.7	Filtre de sécurité réactif SDF-CBF	43
4.7.1	Modèle de distance du robot appris hors ligne	43
4.7.2	Fonction barrière	45
4.7.3	Gradient analytique de la contrainte	47
4.7.4	Contrainte de sécurité et vitesse nominale	50
4.7.5	De la vitesse nominale à la commande sécurisée	52
4.7.6	Évitement des minima locaux du filtre	55
4.7.7	Portée des garanties	58
CHAPITRE 5	RÉSULTATS ET DISCUSSION	61
5.1	Protocol expérimental	62
5.1.1	Description des scénarios testés	62
5.1.2	Métriques utilisées	62
5.2	validation du modèle géométrique de sécurité	62
5.2.1	Précision du SDF du robot	63
5.2.2	Temps d’évaluation du SDF et du gradient	65
5.2.3	Visualisation des points critiques sélectionnés	65
5.3	Qualité de la génération nominale	67
5.3.1	Trajectoire générée sans obstacle critique	67
5.3.2	Cohérence de la trajectoire avec la cible	67
5.4	Sécurité et évitement d’obstacle	67
5.4.1	Comparaison des trajectoires avec et sans SDF-CBF	67
5.4.2	Évolution de $h(q)$ et de la contrainte de sécurité	67
5.4.3	Déviations entre trajectoire nominale et trajectoire sécurisée	72
5.5	Analyse des vitesses et de la fluidité	72
5.5.1	Décomposition des vitesses articulaires	72
5.5.2	Action du filtre de sécurité sur la vitesse commandée	72
5.5.3	Effet du rééchantillonnage du planificateur sur la vitesse exécutée . .	74
5.5.4	Effet du filtrage sur la commande sécurisée	75
5.5.5	Analyse de l’interface de commande	75
5.6	Performance et temps de calcul	77
5.7	Performances globales et limites	77

5.7.1	Taux de succès des scénarios	77
5.7.2	Cas d'échec	77
5.8	Discussion	77
CHAPITRE 6 CONCLUSION		78
6.1	Synthèse des travaux / Summary of Works	78
6.2	Limitations de la solution proposée / Limitations	78
6.3	Améliorations futures / Future Research	78
RÉFÉRENCES		79
ANNEXES		87

LISTE DES TABLEAUX

Tableau 2.1	Tableau comparatif des frameworks de planification et de sécurité de l'état de l'art.	20
Tableau 4.1	Hyperparamètres d'entraînement du modèle de Flow Matching. . . .	40
Tableau 4.2	Temps d'évaluation conjointe des barrières h_l et de la Jacobienne de contrainte complète ($L \times 7$) pour 100 points obstacles, en fonction du nombre L de liens protégés inclus : gradient analytique en forme close contre différentiation automatique (moyenne $\pm$ écart-type en ms, 100 répétitions), avec l'erreur absolue maximale entre les deux Jacobiennes.	50
Tableau 4.3	Paramètres du filtre de sécurité SDF-CBF.	60
Tableau 5.1	Statistiques descriptives par étape (stage)	77
Tableau B.1	Hyperparamètres d'entraînement du modèle de distance du robot. . .	93

LISTE DES FIGURES

Figure 2.1	Trajectoire hors distribution en clonage de comportement	5
Figure 2.2	Champ de vitesse du <i>Flow Matching</i>	7
Figure 4.1	Architecture globale du système asynchrone.	24
Figure 4.2	Exemple de masque obtenu par le modèle de segmentation SAM3 pour le prompt <i>Pink Block</i>	25
Figure 4.3	Protocole d'échantillonnage des cibles, vu de dessus dans le repère de base du robot $\mathcal{F}_b$. Les assiettes (26 cm de diamètre) se chevauchant, seules trois des neuf positions de la grille 3×3 sont illustrées (supérieur droit, centre, inférieur gauche), le centre de chacune étant marqué par une croix noire. Autour de chaque centre, les 5 points indiquent les positions de cible en quinconce.	27
Figure 4.4	Trois phases de la politique experte simulée pour la tâche d'alimentation.	29
Figure 4.5	Trois phases de la politique experte simulée pour la tâche de saisie et dépôt.	29
Figure 4.6	Représentation schématique des différents repères du système	31
Figure 4.7	Architecture du réseau de vitesse ConditionalUnet1D : encodeur-décodeur 1D à trois niveaux de résolution, connexions saut et injection du conditionnement global dans chaque bloc résiduel. Chaque triplet indique la forme du tenseur transitant entre les blocs : taille de lot B , nombre de canaux, longueur temporelle ($H = 16$).	38
Figure 4.8	Architecture du bloc résiduel conditionnel (ConditionalResidualBlock1D) du réseau de vitesse, avec sa modulation affine par le conditionnement global. B désigne la taille du lot, C_{in} et C_{out} les nombres de canaux à l'entrée et à la sortie du bloc, H la longueur temporelle du niveau de résolution courant et $D_{\text{cond}} = 384$ la dimension du conditionnement global c_g	39
Figure 5.1	Précision de la distance du Signed Distance Field (SDF) de Bernstein. (a) Profil de distance le long des normales à la surface du robot ; (b) distribution de l'erreur signée par rapport à la distance exacte calculée sur les maillages.	63
Figure 5.2	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	64

Figure 5.3	Trajectoire multimodale générée par <i>Flow Matching</i>	64
Figure 5.4	Temps de calcul de la distance et du gradient : modèle de Bernstein (Graphics Processing Unit (GPU)) contre vérité terrain sur maillages (Central Processing Unit (CPU)), en fonction du nombre de points de requête.	65
Figure 5.5	Sélection des points les plus proches pour contraindre le CBF	66
Figure 5.6	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	67
Figure 5.7	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	68
Figure 5.8	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	68
Figure 5.9	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	69
Figure 5.10	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	69
Figure 5.11	Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).	70
Figure 5.12	Évolution de la barrière h_{soft} et de sa valeur compensée h_{corr} pour une tâche en présence d'obstacles ; les bandes vertes indiquent les cycles où la projection corrige la vitesse, les bandes jaunes ceux où seule la réparation intervient.	71
Figure 5.13	Contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après réparation). Les valeurs négatives de la courbe nominale correspondent aux violations qu'aurait commises la trajectoire générée sans filtre.	71

Figure 5.14	Décomposition des vitesses articulaires. (a) Construction de la vitesse nominale : feedforward retimé à norme constante $\dot{\mathbf{q}}_{\text{ff}}$, correction de suivi $\dot{\mathbf{q}}_{\text{fb}}$ et vitesse nominale filtrée $\dot{\mathbf{q}}_{\text{base}}$. (b) Action du filtre de sécurité : corrections de projection $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ et de réparation $\Delta\dot{\mathbf{q}}_{\text{rep}}$ (reconstituée par rejeu du filtre de sortie), et vitesse commandée $\dot{\mathbf{q}}_{\text{final}}$. Courbes lissées sur 0,25 s.	73
Figure 5.15	Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.	74
Figure 5.16	Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.	75
Figure 5.17	Norme de la vitesse sécurisée avant et après le filtre passe-bas de sortie ($\tau_{\text{safe}} = 0,2$ s).	76
Figure A.1	Architecture du système d'acquisition des données d'entraînement. . .	88
Figure A.2	Structure des dossiers du jeu de données brut.	89
Figure A.3	Structure des dossiers du jeu de données prétraité.	90

LISTE DES SIGLES ET ABRÉVIATIONS

API	Application Programming Interface
ARA	Assistive Robotic Arm
BC	Behavior Cloning
CBF	Constraint Barrier Function
CFM	Constraint Flow Matching
CPU	Central Processing Unit
DDL	Degré de liberté
FM	Flow Matching
GPU	Graphics Processing Unit
IA	Intelligence Artificielle
JSON	JavaScript Object Notation
LLM	Large Language Model
QP	Quadratic Problem
RAF	Robotic Assisted Feeding
RDF	Robot Distance Field
RGB-D	Red-Green-Blue + Depth
RL	Reinforcement Learning
RLHF	Reinforcement Learning from Human Feedback
ROS	Robot Operating System
SAM	Segment Anything Model
SDF	Signed Distance Field
SO	Specific objective
VLM	Vision Language Model
YAML	YAML Ain't Markup Language

LISTE DES ANNEXES

Annexe A	Protocole détaillé d’acquisition des démonstrations	87
Annexe B	Entraînement du champ de distance signée du robot	92

CHAPITRE 1 INTRODUCTION

Présenter la problématique de recherche, comme quoi l'utilisation des modèles d'IA générative pour des tâches robotiques réelles ne sont pas couramment utiliser puisqu'ils nécessite le positionnement d'une caméra externe, parce que c'est dans des environnements dynamiques et que si on veut qu'ils soient réactifs aux collisions, il faut entraîner le modèle sur plusieurs centaines de données au lieu d'une quarantaine pour une tâche afin de lui faire faire apprendre l'évitement de collision. cela complexifie alors parler aussi des robots collaboratifs qui permettent de réaliser des tâches, mais que ces tâches sont pour la plupart hardcoder et ne réagissent pas aux stimuli environnants, ce qui peut générer des trajectoires dangereuses.

Parler de la motivation de la recherche. Pour moi, serait que la génération de trajectoire par intelligence artificielle, pour ne pas accumuler des centaines de données pour faire de l'évitement de collision doit avoir un système d'évitement de collision sur la trajectoire réalisée. Ce système peut se trouver lors de l'inférence du modèle ou bien après l'inférence du modèle. Lors de l'inférence du modèle, c'est trop long pour des environnements dynamiques et après il faut considérer l'environnement, ce qui nécessite l'entraînement d'un réseau que pour l'environnement. De plus, les systèmes actuels nécessitent une caméra externe calibrée adéquatement, ce qui n'est pas utilisable pour un usage quotidien. Ce qu'on propose c'est un système d'évitement de collision pour des environnements dynamiques pour des environnements non vues pendant l'entraînement avec une seule caméra au niveau de l'effecteur du robot.

Faire un paragraphe par rapport à la structure du mémoire. Celle de Félix c'était : Chapter 2 reviews the literature on robot-assisted feeding, highlighting current challenges in perception, task planning, and manipulation. Chapter 3 defines the specific scope and research objectives of this project. Chapter 4 details the design of the system, describing the system architecture, the integration of Vision-Language Models (VLMs) for autonomous food acquisition planning, and the implementation of the food perception and manipulation modules. Chapter 5 presents the experimental results, including an analysis of VLM-based food acquisition planning performance and a validation of the system on a physical robot. Finally, Chapter 6 summarizes the main contributions, discusses the system's limitations, and proposes directions for future research.

Donc nous cela pourrait être "Le chapitre 2 présente la revue de littérature en lien avec la génération de trajectoires robotiques en utilisant l'IA générative mettant en évidence l'exécution des trajectoires la vision par ordinateur et le suivi de cible ainsi que l'évitement de

collision. Le chapitre 3 présente la question de recherche ainsi que les sous objectifs associées pour répondre à cette hypothèse. Le chapitre 4 présente le design du système, mettant de l'avant son architecture, l'intégration des systèmes de vision par ordinateur pour détecter et suivre la cible, l'acquisition des données pour entraîner le modèle de Flow Matching, la génération des trajectoires générées par IA ainsi, la capture des obstacles, le filtre de sécurité utilisé et l'intégration du temps réel. Le chapitre 5 présente les résultats expérimentaux, la validation du modèle géométrique de sécurité, la qualité de la trajectoire nominale, l'analyse des vitesses, la sécurité et l'évitement d'obstacle, les performances computationnelles ainsi que les limites du système. "

CHAPITRE 2 REVUE DE LITTÉRATURE

Cette revue de littérature présente une analyse de l'état de l'art pour la génération de trajectoires robotisées sûres en environnement quasi-statique et non structuré. Elle est organisée selon l'architecture en chaîne « perception, planification, action » proposée dans ce travail. Nous abordons d'abord les méthodes de génération de mouvement, en situant l'émergence du *Flow Matching* conditionnel retenu comme planificateur nominal. Ensuite, nous examinons les techniques de perception visuelle et de mémoire spatiale qui fournissent le conditionnement géométrique nécessaire. Les représentations géométriques permettant d'évaluer les collisions en temps réel sont ensuite analysées, suivies par les lois de commande sécuritaires réactives fondées sur la théorie du contrôle. Enfin, nous étudions les stratégies de couplage entre modèles génératifs et filtres de sécurité, ce qui prépare le positionnement de l'architecture découplée proposée dans ce mémoire.

2.1 Génération de trajectoires robotisées

Objectif de la section : Analyser le développement théorique et les limites connues des méthodes de génération de mouvements afin de situer le choix du *Flow Matching* conditionnel comme planificateur d'intention nominale.

2.1.1 Planification classique par échantillonnage et optimisation

La génération de mouvements sans collision a d'abord été abordée par des algorithmes déterministes raisonnant directement dans l'espace des configurations sans apprentissage préalable. Historiquement, deux grandes familles se dégagent selon qu'elles explorent cet espace par tirages aléatoires ou qu'elles raffinent une trajectoire par optimisation continue. D'une part, les méthodes par échantillonnage, à l'instar de l'algorithme *Rapidly-exploring Random Tree star* (RRT*) de Karaman et Frazzoli [1], construisent incrémentalement un arbre de configurations valides. Bien que ces planificateurs, largement implémentés au sein de la bibliothèque *Open Motion Planning Library* (OMPL) [2], garantissent une complétude probabiliste, voire une optimalité asymptotique pour les variantes étoilées comme RRT*, ils produisent généralement des trajectoires géométriquement rugueuses et saccadées. Ce défaut impose une étape de lissage a posteriori qui dépend fortement de la complexité géométrique de la scène [3].

D'autre part, les approches par optimisation locale adoptent une démarche inverse en raffinant une trajectoire initiale par minimisation d'une fonctionnelle de coût. Par exemple,

CHOMP (*Covariant Hamiltonian Optimization for Motion Planning*) [4] exploite le gradient d'un champ de distance signé pré-calculé pour repousser la trajectoire d'initialisation (typiquement une interpolation rectiligne dans l'espace des configurations) hors des obstacles tout en la lissant. De même, TrajOpt [5] formule le problème sous forme d'une séquence d'optimisations convexes sous contraintes relaxées de non-collision, démontrant d'excellents taux de réussite sur des bras articulés complexes. Néanmoins, ces méthodes restent intrinsèquement sensibles à la trajectoire d'initialisation et tendent à converger vers des minima locaux sous-optimaux, ce qui peut conduire à des violations de contraintes si le champ de distance est bruité. Pour pallier cette dépendance à l'initialisation, les approches classiques d'amorçage (*warm-starting*) proposent d'utiliser des heuristiques géométriques ou des chemins sans collision issus d'algorithmes par échantillonnage, comme RRT-Connect [6] ou des cartes routières probabilistes (PRM) [7], pour ensemencher l'optimiseur. Ce couplage hybride échantillonnage-optimisation combine la complétude globale du premier et la régularité locale du second. L'évolution de ces stratégies classiques d'amorçage vers des représentations apprises assure une transition naturelle vers les méthodes fondées sur l'apprentissage, qui cherchent à s'affranchir de la recherche géométrique explicite en exploitant des démonstrations expertes.

2.1.2 Apprentissage par démonstration, clonage de comportement et renforcement

L'apprentissage par démonstration (*Learning from Demonstration*, LfD), dont les fondations théoriques ont été posées par Atkeson et Schaal [8,9] en 1997 et 1999, avant d'être synthétisées par Argall et al. [10] en 2009, vise à reproduire un comportement expert sans programmer explicitement la loi de commande. La formulation la plus directe, le clonage de comportement (*Behavior Cloning*, BC), dont le concept remonte aux travaux pionniers d'ALVINN de Pomerleau [11] et a été formalisé par Bain et Sammut [12], traite le problème comme une régression supervisée des actions expertes à partir des observations. Bien qu'appliqué avec succès à des tâches d'acquisition alimentaire par imitation visuelle ou de manipulation fine moderne [13–15], le BC sous perte quadratique (*Mean Squared Error*, MSE) souffre de deux limites structurelles majeures : l'erreur cumulative (*compounding errors*), où de petites déviations entraînent le robot hors de la distribution d'entraînement, et l'effondrement de la multimodalité, la moyenne de démonstrations divergentes produisant des actions aberrantes ou invalides.

Pour surmonter ces écueils, Florence et al. [17] ont introduit le clonage de comportement implicite (*Implicit BC*) représentant la politique par un modèle d'énergie. Cette formulation

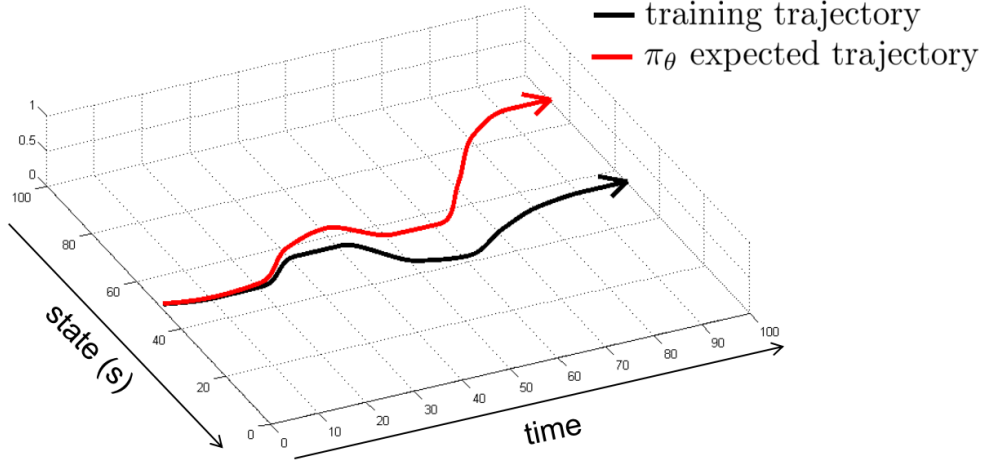


Figure 2.1 Représentation d’une trajectoire obtenue par *Behavior Cloning* hors distribution, tirée de [16]

permet de capturer des distributions d’actions multivaluées et discontinues, surpassant la régression directe sur des tâches nécessitant une grande précision géométrique. Par ailleurs, des travaux connexes étendent cette logique à l’imitation strictement hors ligne [18] ou exploitent des représentations auto-supervisées pour accroître la généralisation du système [19]. En parallèle, l’apprentissage par renforcement (*Reinforcement Learning*, RL) résout le problème de la multimodalité par exploration active de l’environnement [20,21]. Toutefois, le RL se heurte à une inefficacité d’échantillonnage prohibitive et pose des risques majeurs de collision, ce qui complique son déploiement direct sur des plateformes physiques réelles. Enfin, une approche alternative consiste à encoder le mouvement sous forme de systèmes dynamiques stables par construction (tels que l’estimateur SEDS [22]) afin de garantir la convergence vers la cible, bien que cela se fasse au détriment de l’expressivité par rapport aux modèles génératifs récents.

2.1.3 Modèles génératifs, politiques de diffusion et Flow Matching

Les politiques génératives modélisent explicitement la distribution des trajectoires expertes, ce qui leur permet de représenter naturellement la multimodalité inhérente aux comportements humains [23, 24]. Récemment, la *Diffusion Policy* de Chi et al. [25], inspirée des modèles probabilistes de débruitage (*Denoising Diffusion Probabilistic Models*) [26] et de la planification générative [27, 28], a démontré des performances remarquables en manipulant des objets par un processus de débruitage stochastique itératif. De même, sa variante tridimensionnelle (*3D Diffusion Policy*) conditionne cette génération sur un nuage de points [29], renforçant la robustesse géométrique. Toutefois, le principal inconvénient des politiques de

diffusion réside dans leur latence d'inférence élevée : la génération d'une trajectoire requiert de nombreuses évaluations séquentielles du réseau de neurones, limitant leur application en boucle fermée à haute fréquence. Bien que des techniques de distillation de modèle (telles que ManiCM [30] ou d'autres approches accélérées [31,32]) parviennent à ramener l'inférence à une seule étape, elles nécessitent des phases d'entraînement complexes et peuvent dégrader la stabilité des trajectoires générées.

Pour offrir une alternative déterministe plus directe et efficace, Lipman et al. [33] ont introduit le formalisme du *Flow Matching*. L'objectif est d'apprendre un champ de vitesses $u_t^\theta(X)$, indexé par un temps d'écoulement $t \in [0, 1]$, qui déplace continûment un échantillon initial issu d'une distribution simple p_0 (typiquement une gaussienne standard $\mathcal{N}(\mathbf{0}, \mathbf{I})$) vers la distribution cible des trajectoires expertes p_1 . Une trajectoire candidate X_t est générée en intégrant l'équation différentielle ordinaire (ODE) associée :

$$\frac{dX_t}{dt} = u_t^\theta(X_t). \quad (2.1)$$

Le champ de vitesses marginal engendrant la distribution de probabilité p_t étant généralement incalculable analytiquement, le *Conditional Flow Matching* (CFM) [34] résout ce problème en définissant un chemin de probabilité conditionnel $p_t(X \mid X_0, X_1) = \mathcal{N}(X \mid \mu_t(X_0, X_1), \sigma_t^2 \mathbf{I})$ et son champ de vitesses associé $u_t(X \mid X_0, X_1)$ qui sont analytiquement traitables. Pour des distributions gaussiennes conditionnelles, ce champ cible s'exprime par :

$$u_t(X \mid X_0, X_1) = \frac{\dot{\sigma}_t}{\sigma_t} (X - \mu_t(X_0, X_1)) + \dot{\mu}_t(X_0, X_1), \quad (2.2)$$

où $\dot{\mu}_t$ et $\dot{\sigma}_t$ désignent les dérivées temporelles des paramètres du chemin. Le réseau de neurones u_t^θ est alors entraîné en minimisant la perte de régression du champ de vitesses conditionnel :

$$\mathcal{L}(\theta) = \mathbb{E}_{t \sim \mathcal{U}[0,1], X_1 \sim p_1, X_0 \sim p_0, X_t \sim p_t(\cdot \mid X_0, X_1)} \left[\left\| u_t^\theta(X_t) - u_t(X_t \mid X_0, X_1) \right\|^2 \right]. \quad (2.3)$$

Dans le cadre du *Rectified Flow* proposé par Liu et al. [35], le chemin est défini par interpolation linéaire simple entre le bruit et la donnée réelle, avec une variance constante :

$$\mu_t(X_0, X_1) = tX_1 + (1-t)X_0, \quad \sigma_t = \sigma_{\min}, \quad (2.4)$$

ce qui réduit le champ cible u_t au vecteur constant reliant le point initial au point final :

$$u_t(X_t \mid X_0, X_1) = X_1 - X_0. \quad (2.5)$$

Cette formulation linéaire favorise des trajectoires d'intégration quasi rectilignes, permettant d'intégrer l'ODE en un nombre réduit d'étapes de discrétisation tout en préservant la qualité des trajectoires. Pour adapter ce formalisme à la robotique, il suffit de conditionner le champ $u_t^\theta(X \mid c)$ sur une observation sémantique ou géométrique c (comme un nuage de points), ce qui permet d'obtenir un planificateur de trajectoire réactif, comme l'illustre le framework PointFlowMatch [36].

Plus récemment, de nombreuses extensions ont cherché à accroître l'efficacité et la généralisation géométrique de ces modèles. Des approches telles que ManiFlow [37], MeanFlow [38, 39] et les politiques à flux continu [40] visent la génération de mouvements en une ou deux étapes d'inférence. D'autres travaux, à l'instar de FlowMP [41, 42], étendent le *Flow Matching* aux dynamiques du second ordre afin de générer des trajectoires respectant les contraintes d'accélération et de couple. Enfin, la politique riemannienne (*Riemannian Flow Matching Policy*, RFMP) de Braun et al. [43] transpose ce formalisme directement sur des variétés non euclidiennes telles que $SE(3)$ par interpolation géodésique, réduisant les saccades de rotation et le temps de calcul. Cette combinaison unique d'expressivité multimodale, de rapidité d'intégration déterministe et de robustesse géométrique justifie l'adoption du *Flow Matching* comme planificateur nominal d'intention au sein de notre architecture.

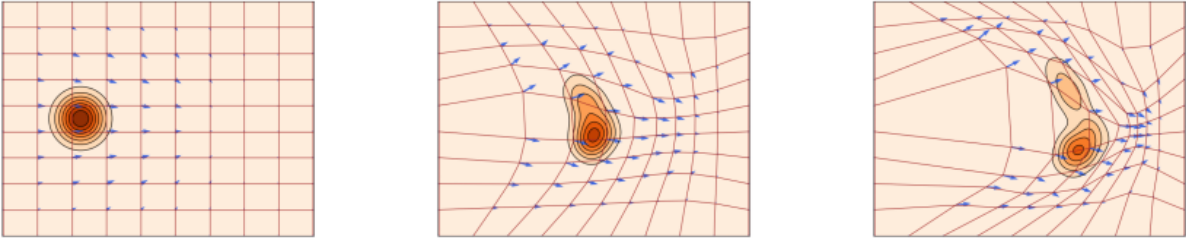


Figure 2.2 Représentation du champ de vitesses u_t transportant la distribution initiale p_0 vers la distribution cible p_1 pour $t \in [0, 1]$, tirée de [44]

2.2 Perception visuelle et mémoire spatiale

Objectif de la section : Documenter les mécanismes permettant de convertir un flux de données visuelles bidimensionnelles et bruitées en un signal de conditionnement géométrique statique et persistant.

2.2.1 Vision-langage, segmentation zéro-shot et suivi visuel

La première étape de l'architecture consiste à extraire de la scène l'objet pertinent pour la tâche et à l'ancrer sémantiquement. Dans cette perspective, deux grands paradigmes s'oc-

cupent d’associer perception et action : les modèles monolithiques, qui fusionnent perception et action, et les architectures modulaires, qui séparent la compréhension sémantique de la génération de mouvement. D’une part, les architectures *Vision-Language-Action* (VLA) monolithiques intègrent la perception sémantique et le contrôle dans un unique réseau de neurones entraîné de bout en bout. Des modèles tels que RT-2 de Brohan et al. [45] ou *OpenVLA* de Kim et al. [46] en sont particulièrement représentatifs. Ce dernier, fort de sept milliards de paramètres, surpasse significativement les modèles propriétaires antérieurs sur une grande variété de tâches de manipulation [47, 48]. Néanmoins, leur principale limite réside dans leur latence d’inférence élevée, peu compatible avec un contrôle réactif en temps réel. Bien que des versions compressées comme TinyVLA [49] tentent d’accélérer ce processus, le temps de calcul reste un verrou majeur.

D’autre part, les approches modulaires découplent l’exécution en confiant l’interprétation sémantique de l’invite textuelle (par exemple, identifier l’aliment à acquérir) à un modèle de vision-langage ou à un grand modèle de langage (LLM) [50–53], laissant la localisation géométrique fine à des modules dédiés. Ce découplage préserve l’interprétabilité du système et permet de remplacer indépendamment chaque composant. Pour la segmentation fine des objets, le modèle fondateur *Segment Anything* (SAM) de Kirillov et al. [54], sorti en 2023, a démontré d’excellentes capacités de généralisation zéro-shot sans réentraînement. Plus récemment, en 2025, sa troisième version (*Segment Anything 3* ou SAM3) [55] a introduit la segmentation par concept, permettant de localiser toutes les instances correspondant à une requête sémantique en une seule inférence. Enfin, le modèle SAM2, sorti en 2024, de Ravi et al. [56] permet d’étendre cette logique dans le domaine temporel en assurant la propagation cohérente et le suivi des masques sémantiques le long du flux d’images, préparant ainsi la reconstruction 3D de la scène.

2.2.2 Reconstruction RGB-D et représentation par nuages de points

Une fois le masque sémantique obtenu en deux dimensions, la rétroprojection géométrique de ses pixels dans l’espace cartésien, facilitée par les caméras RGB-D, permet de générer des nuages de points tridimensionnels de la cible et des obstacles. Se pose alors la question de la représentation géométrique optimale pour conditionner les politiques génératives. À cet égard, le conditionnement direct sur des nuages de points s’est imposé comme une alternative solide aux images brutes, car il capture explicitement les relations spatiales tridimensionnelles de la scène [57, 58]. Cette efficacité est illustrée par la politique *3D Diffusion Policy* [29] et le modèle fondationnel FP3 [59], qui démontrent une excellente généralisation géométrique avec un nombre réduit de démonstrations expertes.

Toutefois, l’acquisition d’un nuage de points à partir d’une unique caméra embarquée reste vulnérable aux occlusions physiques induites par les mouvements de l’effecteur ou du bras articulé lui-même. Pour y faire face, des méthodes comme CordViP de Fu et al. [60] proposent de reconstruire les zones masquées en combinant proprioception et estimation de pose 6D, restaurant ainsi la géométrie de la zone d’interaction. Bien que certains travaux suggèrent que des approches bidimensionnelles enrichies en pseudo-3D puissent parfois suffire [61, 62], l’utilisation de nuages de points 3D bruts présente l’avantage de ne requérir aucun modèle CAO préalable des objets de la scène, ce qui s’avère indispensable pour manipuler des formes complexes ou non structurées.

2.2.3 Cartes persistantes face aux occlusions et pertes de détection

Néanmoins, la perception instantanée en direct est insuffisante lors de tâches robotiques interactives dans des environnements dynamiques. Dès que le robot effectue une approche, son propre corps engendre des occlusions physiques massives, masquant temporairement la cible ou les obstacles proches [63]. Une politique fonctionnant purement en boucle ouverte et sans mémoire spatiale serait alors incapable de maintenir sa consigne de mouvement. Pour résoudre cette limite, la cartographie d’environnements dynamiques propose d’intégrer des graphes de facteurs (à l’instar de DynoSAM [64]) afin de séparer la géométrie statique des trajectoires d’objets mobiles. De même, des approches probabilistes comme SE(3)-PoseFlow de Jin et al. [65] utilisent le *Flow Matching* pour estimer les distributions de poses sous forte occlusion.

Afin de stabiliser durablement le signal de conditionnement fourni au planificateur nominal, l’intégration d’une couche de mémoire spatiale persistante, telle qu’une grille de voxels locale ou une carte d’occupation historique, s’appuie classiquement sur des méthodes de cartographie par grille d’occupation [66, 67]. En accumulant et en filtrant les nuages de points capturés au fil du temps par fusion spatio-temporelle [68, 69], cette mémoire spatiale maintient une représentation géométrique tridimensionnelle fiable des obstacles et de la cible, même lors de pertes de détection visuelle. Ce mécanisme garantit la continuité géométrique du signal envoyé au planificateur, constituant une brique essentielle de l’architecture développée pour naviguer de manière sécuritaire dans les zones confinées.

2.3 Représentations géométriques pour la collision

Objectif de la section : Évaluer les méthodes de modélisation de l’encombrement spatial du robot et des obstacles pour formuler des métriques de distance différentiables.

2.3.1 Champs de distance de la scène et limites computationnelles

Imposer une contrainte d'évitement active suppose de disposer d'une métrique de distance différentiable entre le robot et les obstacles de son environnement. Une première famille de méthodes centre cette métrique sur la scène en discrétisant l'espace de travail. Les champs de distance signés euclidiens (*Euclidean Signed Distance Fields*, ESDF) encodent, en chaque point de l'espace, la distance à la surface de l'obstacle le plus proche. Formellement, le champ associe à un point $\mathbf{p} \in \mathbb{R}^3$ sa distance signée à la surface d'un obstacle $\mathcal{O} \subset \mathbb{R}^3$:

$$d(\mathbf{p}) = s_{\mathcal{O}}(\mathbf{p}) \min_{\mathbf{q} \in \partial\mathcal{O}} \|\mathbf{p} - \mathbf{q}\|_2, \quad s_{\mathcal{O}}(\mathbf{p}) = \begin{cases} -1, & \mathbf{p} \in \mathcal{O}, \\ +1, & \text{sinon,} \end{cases} \quad (2.6)$$

où la distance est ainsi comptée négativement à l'intérieur du volume de l'obstacle et positivement à l'extérieur, la frontière $\partial\mathcal{O}$ correspondant au niveau zéro $d(\mathbf{p}) = 0$. Un tel champ de distance théorique vérifie l'équation eikonale :

$$\|\nabla d(\mathbf{p})\|_2 = 1, \quad (2.7)$$

indiquant que le gradient du champ est unitaire et pointe dans la direction d'éloignement la plus rapide de l'obstacle, ce qui fournit directement la direction d'évitement.

Pour évaluer numériquement ce champ de distance de la scène en robotique, trois grands paradigmes coexistent :

1. **Le pré-calcul hors ligne (Offline) :** Pour des environnements statiques et connus, l'ESDF est échantillonné à l'avance sur une grille de voxels à partir du modèle CAO de la scène. Cette méthode, utilisée historiquement dans des planificateurs comme CHOMP [4], permet des requêtes de distance en temps constant $\mathcal{O}(1)$ lors de la planification, mais s'avère incapable de s'adapter au moindre changement d'environnement ou à l'apparition d'obstacles imprévus.
2. **La reconstruction en ligne par grilles volumétriques incrémentales (Online Grids) :** Les bibliothèques de cartographie en direct, à l'instar d'OctoMap [67], Voxblox [70] ou nvblox [71], maintiennent et mettent à jour incrémentalement l'ESDF à partir des flux de données de capteurs RGB-D. Pour accroître la rapidité ou la précision de ces grilles, des architectures optimisées comme FIESTA [72] (optimisant la vitesse de calcul pour la planification de drones) ou Voxfield [73] (qui exploite des TSDF non projectifs pour améliorer la fidélité de la distance) ont été développées. Bien que ces grilles s'adaptent aux scènes dynamiques et exploitent le parallélisme des GPU [74], leur coût de mise à jour tridimensionnelle demeure élevé et dépend

fortement de la résolution choisie, ce qui devient prohibitif à la fréquence d’une boucle de commande réactive.

3. **Les champs de distance neuronaux continus en ligne (Online Continual Neural SDF)** : Afin de s’affranchir de la discrétisation par grilles volumétriques, des approches de reconstruction continue en ligne, comme iSDF d’Ortiz et al. [75], entraînent en temps réel un réseau de neurones de coordonnées sur le flux sensoriel actif pour apprendre une représentation continue et différentiable de la scène. Ce concept s’inscrit dans la lignée des frameworks de SLAM implicite en temps réel tels qu’iMAP [76] et sa variante hiérarchique NICE-SLAM [77]. Cependant, ces modèles neuronaux souffrent d’une latence de convergence lors de changements rapides de la scène et de risques d’oubli catastrophique, ce qui complique leur intégration directe dans une boucle de contrôle sécuritaire à haute fréquence.

Ces limitations computationnelles partagées motivent le développement de représentations alternatives centrées sur le robot plutôt que sur la scène.

2.3.2 Primitives conservatives et modèles géométriques simplifiés

Pour contourner la lourdeur du calcul des champs de distance globaux de la scène, une seconde famille de méthodes choisit plutôt de simplifier la géométrie du robot lui-même. L’approche classique consiste à encapsuler les différents liens de la chaîne cinématique dans des volumes englobants simples, tels que des sphères ou des capsules, dont les tests de collision analytiques se parallélisent très efficacement sur GPU. La bibliothèque cuRobo de Sundaralingam et al. [78] exploite cette idée pour générer des mouvements sûrs en quelques dizaines de millisecondes, surpassant de plus d’un ordre de grandeur les solveurs classiques de l’état de l’art. Récemment, ces enveloppes ont été adaptées pour prendre en compte l’outil monté à l’effecteur en temps réel [79]. Néanmoins, le revers de cette approximation géométrique conservative est qu’elle tend à surestimer grandement l’encombrement réel du robot. Cette perte d’espace libre exploitable restreint la capacité du robot à évoluer dans des zones confinées, ce qui est problématique pour des tâches d’assistance physique comme l’alimentation.

2.3.3 Champs de distance signée du robot

Afin de concilier la précision géométrique et la rapidité de calcul, un changement de paradigme récent propose de déplacer le champ de distance de la scène vers le robot en modélisant directement la géométrie du bras articulé. Les champs de distance du robot (*Robot Distance Fields*, RDF), introduits par Li et al. [80], approchent la SDF de chaque lien du robot par des polynômes de Bernstein. Pour une coordonnée spatiale normalisée $\mathbf{u} = (u_1, u_2, u_3) \in [0, 1]^3$,

la distance signée associée à un lien l s'écrit comme un produit tensoriel 3D de la base polynomiale de Bernstein de degré n :

$$\bar{d}_l(\mathbf{u}) = \sum_{i=0}^n \sum_{j=0}^n \sum_{k=0}^n c_{ijk}^l B_{i,n}(u_1) B_{j,n}(u_2) B_{k,n}(u_3), \quad (2.8)$$

où les coefficients c_{ijk}^l , appris hors ligne à partir du modèle CAO, encodent précisément la forme géométrique tridimensionnelle du lien. Les polynômes de base de Bernstein de degré n sont définis analytiquement par :

$$B_{i,n}(u) = \binom{n}{i} u^i (1-u)^{n-i}. \quad (2.9)$$

Puisque la distance approchée $\bar{d}_l(\mathbf{u})$ est une fonction polynomiale, son gradient par rapport aux coordonnées spatiales s'obtient analytiquement sans nécessiter de coûteuses approximations par différences finies. En particulier, la dérivée d'un polynôme de base par rapport à la coordonnée normalisée u s'écrit simplement sous la forme :

$$\frac{d}{du} B_{i,n}(u) = n (B_{i-1,n-1}(u) - B_{i,n-1}(u)), \quad (2.10)$$

avec la convention que $B_{-1,n-1}(u) = 0$ et $B_{n,n-1}(u) = 0$. Cette propriété permet un calcul exact et instantané du gradient spatial sur GPU, pour une empreinte mémoire négligeable.

Contrairement aux représentations neuronales alternatives (telles que les *Neural SDF* [81]) qui souffrent de temps d'inférence plus élevés et d'un manque de régularité mathématique, l'approximation polynomiale de Bernstein fournit une structure différentiable et continue. Ses gradients pouvant être interrogés directement par rapport aux points de l'environnement, Govoni et al. [82] les ont exploités pour l'évitement réactif dans un cadre de commande de tâches. Toutefois, l'usage d'opérateurs de composition non lisses (comme le minimum discret sur les distances des différents liens ou le *soft-min* pour regrouper les contributions de l'environnement) implique que la fonction de distance globale du robot n'est pas strictement de classe $\mathcal{C}^2$, une limitation théorique dont il faut tenir compte lorsque cette distance est exploitée comme fonction barrière dans une loi de commande.

2.4 Sécurité réactive et fonctions de contrôle barrières

Objectif de la section : Analyser les lois de commande réactives issues de la théorie du contrôle afin d'imposer une contrainte de sécurité en vitesse sous certaines hypothèses.

2.4.1 Champs de potentiel et méthodes réactives classiques

La sécurité réactive consiste à modifier localement et en temps réel le mouvement du robot directement lors de son exécution sur la plateforme physique. Historiquement, la première formulation repose sur des forces virtuelles à travers la méthode des champs de potentiel artificiels (APF) introduite par Khatib [83]. Le principe est d'associer à la cible une force attractive et aux obstacles des forces répulsives. Bien que cette approche soit très légère en calcul et s'implémente facilement en ligne, elle souffre de limites théoriques majeures : l'absence de garanties de stabilité pour des dynamiques complexes, des oscillations à proximité des obstacles et un blocage fréquent dans des minima locaux induits par des géométries concaves. Néanmoins, le concept reste exploité dans des architectures génératives récentes pour guider des politiques d'imitation vers des trajectoires sûres [84].

2.4.2 Fonctions barrières de contrôle et invariance avant

Pour pallier l'absence de garanties formelles de sécurité des champs de potentiel, les fonctions barrières de contrôle (*Control Barrier Functions*, CBF), formalisées par Ames et al. [85] sur la base des certificats de barrière de Wieland et Allgöwer [86], offrent un cadre d'invariance rigoureux. Considérons un système dynamique non linéaire affine en contrôle de la forme :

$$\dot{\mathbf{x}} = f(\mathbf{x}) + g(\mathbf{x})\mathbf{u}, \quad (2.11)$$

où $\mathbf{x} \in \mathbb{R}^d$ désigne l'état et $\mathbf{u} \in \mathbb{R}^m$ l'action de commande. On définit un ensemble de sécurité $\mathcal{C}$ comme le sous-ensemble de l'espace d'état où une fonction scalaire continûment différentiable $h : \mathbb{R}^d \rightarrow \mathbb{R}$ reste positive :

$$\mathcal{C} = \{\mathbf{x} \in \mathbb{R}^d : h(\mathbf{x}) \geq 0\}, \quad (2.12)$$

la frontière $h(\mathbf{x}) = 0$ marquant le contact ou la limite de sécurité avec l'obstacle. L'ensemble $\mathcal{C}$ est dit invariant vers l'avant si toute trajectoire débutant dans $\mathcal{C}$ y demeure indéfiniment. En exploitant les dérivées de Lie $L_f h(\mathbf{x}) = \nabla h(\mathbf{x})^\top f(\mathbf{x})$ et $L_g h(\mathbf{x}) = \nabla h(\mathbf{x})^\top g(\mathbf{x})$, la fonction h est une fonction barrière de contrôle s'il existe une fonction α de classe $\mathcal{K}$, c'est-à-dire strictement croissante et nulle à l'origine, telle que, dans le cas général :

$$\sup_{\mathbf{u} \in \mathbb{R}^m} [L_f h(\mathbf{x}) + L_g h(\mathbf{x})\mathbf{u}] \geq -\alpha(h(\mathbf{x})). \quad (2.13)$$

Cette condition autorise la fonction barrière à décroître à l'approche de la frontière, mais à un taux borné par α , qui s'annule exactement sur celle-ci : la trajectoire ne peut donc

jamais quitter l'ensemble sûr $\mathcal{C}$. En pratique, la fonction α est le plus souvent choisie linéaire, $\alpha(h) = \kappa h$ avec un gain $\kappa > 0$ réglant l'agressivité du freinage près de la frontière, ce qui conduit à la condition d'invariance affine en la commande :

$$L_f h(\mathbf{x}) + L_g h(\mathbf{x})\mathbf{u} \geq -\kappa h(\mathbf{x}). \quad (2.14)$$

Lorsque le contrôle du robot s'effectue directement en vitesse articulaire (degré relatif un, où l'état correspond à la configuration articulaire $\mathbf{x} = \mathbf{q}$ et l'action à la vitesse $\mathbf{u} = \dot{\mathbf{q}}$), la dynamique se réduit à $\dot{\mathbf{q}} = \mathbf{u}$. La condition barrière (2.14) se simplifie alors sous la forme linéaire suivante :

$$\nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \geq -\kappa h(\mathbf{q}). \quad (2.15)$$

Dans le cadre de l'évitement de collision articulé, la fonction barrière $h_i(\mathbf{q})$ associée à un point critique i du robot s'exprime directement à partir de la distance physique aux obstacles $d_{\text{robot},i}(\mathbf{q})$, évaluée via le modèle de Bernstein (SDF locale), minorée d'une marge de sécurité minimale d_{safe} :

$$h_i(\mathbf{q}) = d_{\text{robot},i}(\mathbf{q}) - d_{\text{safe}}. \quad (2.16)$$

La condition (2.15) étant affine par rapport aux vitesses articulaires $\dot{\mathbf{q}}$, elle peut s'intégrer sous forme de contraintes linéaires au sein d'un programme quadratique (QP-CBF) résolu à chaque cycle de commande [87] :

$$\dot{\mathbf{q}}^* = \arg \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{nom}}\|_2^2 \quad \text{s.c.} \quad \nabla h_i(\mathbf{q})^\top \dot{\mathbf{q}} \geq -\kappa h_i(\mathbf{q}), \quad \forall i \in \{1, \dots, N_c\}, \quad (2.17)$$

où $\dot{\mathbf{q}}_{\text{nom}}$ représente la consigne nominale et N_c désigne le nombre de contraintes actives (les points critiques les plus proches de l'environnement). Le programme quadratique (2.17) agit ainsi comme un filtre de sécurité réactif en projetant la vitesse nominale sur le demi-espace admissible le plus proche. Des extensions étendent cette formulation aux systèmes mécaniques de degré relatif deux (commande en couples ou accélérations) [88].

2.4.3 Filtres de sécurité prédictifs et méthodes d'atteignabilité

La projection QP-CBF n'est toutefois qu'une instance d'un concept plus large, celui du filtre de sécurité, dont Hsu et al. [89] proposent une vue unifiée : un module de supervision minimalement invasif qui surveille la commande nominale, quelle qu'en soit l'origine, et ne la modifie que lorsque la sécurité l'exige. Au-delà des CBF, deux autres familles dominent ce paradigme. D'une part, les filtres de sécurité prédictifs de Wabersich et Zeilinger [90] transposent la philosophie de la commande prédictive : à chaque cycle, un problème de

commande optimale est résolu sur un horizon fini afin de vérifier qu’il existe encore une trajectoire de repli ramenant le système vers un ensemble sûr terminal. Cette anticipation corrige la principale faiblesse structurelle des CBF, leur myopie : là où la contrainte barrière ne raisonne que sur la dérivée instantanée de h , le filtre prédictif détecte à l’avance les situations où les saturations d’actionneurs rendront la contrainte infaisable. D’autre part, les méthodes fondées sur l’atteignabilité de Hamilton-Jacobi calculent l’ensemble des états à partir desquels la collision devient inévitable, offrant les garanties les plus robustes face aux perturbations bornées [91].

Ces gains d’anticipation et de robustesse ont cependant un coût computationnel élevé : le filtre prédictif requiert la résolution en ligne d’un programme non convexe sur tout l’horizon ainsi qu’un modèle dynamique fiable, tandis que l’atteignabilité de Hamilton-Jacobi souffre de la malédiction de la dimension et reste difficilement applicable au-delà de quelques dimensions d’état sans approximation [91]. Pour un bras à sept degrés de liberté contraint par des milliers de points de nuage renouvelés à chaque cycle de perception, la projection QP-CBF demeure la seule de ces trois familles à s’exécuter à la fréquence de commande visée, au prix d’un raisonnement purement instantané.

2.4.4 Limites des CBF sous perception bruitée et contrôle discret

Néanmoins, les garanties d’invariance théorique établies en temps continu et sous l’hypothèse d’un modèle géométrique parfait se dégradent fortement lors de leur déploiement sur une plateforme physique réelle. En effet, trois facteurs limitent l’invariance absolue :

1. **La discrétisation temporelle** : La boucle de commande s’exécute à fréquence finie, ce qui peut provoquer des franchissements de la barrière entre deux pas de calcul. Cette limite a conduit au développement des formulations en temps discret (DT-CBF) de la condition barrière [92].
2. **Les saturations matérielles** : Les limites de vitesse et de couple des actionneurs restreignent le domaine d’admissibilité du QP. Si la contrainte CBF exige une accélération ou une vitesse supérieure aux capacités physiques du robot, le programme quadratique devient infaisable [93], compromettant la sécurité.
3. **Le bruit de perception et l’erreur d’interface** : Le calcul du gradient sur des nuages de points réels et bruités introduit des fluctuations dans les contraintes. De plus, le suivi de la consigne de vitesse $\dot{\mathbf{q}}^*$ par un contrôleur en position bas niveau engendre des retards et des dérives cinématiques.

Par conséquent, dans des applications réelles non structurées, la loi de commande CBF doit être traitée comme un filtre de sécurité pragmatique, plutôt que comme une garantie formelle

infaillible, ce qui justifie l'utilisation de marges de sécurité conservatives.

2.5 Couplage entre modèles génératifs et filtres de sécurité

Objectif de la section : Examiner les différentes stratégies d'unification de l'IA générative et des lois de commande sécuritaires afin de positionner la méthode de découplage retenue.

Pour allier l'expressivité et la multimodalité des modèles génératifs avec la rigueur des garanties de non-collision, deux grandes stratégies d'unification se distinguent dans la littérature : l'intégration des contraintes directement au cœur du processus de génération, ou l'application d'un filtre de sécurité en aval lors de l'exécution de la trajectoire.

2.5.1 Contraintes intégrées à la génération

La première stratégie injecte la contrainte de sécurité au sein même du processus d'intégration de l'équation différentielle (guidage intra-ODE). Par exemple, SafeFlow de Dai et al. [94] formule des fonctions barrières adaptées au *Flow Matching* afin de confiner le flux de trajectoires dans l'espace libre sans nécessiter de réentraînement du réseau. Dans la même ligne, SafeFlowMatcher [95] propose un intégrateur de type prédiction-correction certifié pour converger en temps fini vers la zone sûre. De son côté, CASF (Constrained Action Streaming Flows) de Long et al. [96] déforme localement la métrique riemannienne de l'espace de commande pour rejeter les trajectoires s'approchant des frontières d'obstacles. D'autres variantes récentes adaptent ces concepts sous forme de contraintes de Lyapunov ou de barrières stochastiques intégrées aux processus de diffusion [97, 98], ou s'appuient sur des optimisations unifiées pour garantir l'admissibilité dynamique des mouvements [99–101].

Cependant, bien qu'efficaces en simulation, ces méthodes intra-génération se heurtent à une limite structurelle majeure : la dérive distributionnelle (*distributional drift*). En forçant le processus d'intégration à traverser des états latents qui s'écartent de la distribution des données expertes vue à l'entraînement, ces contraintes perturbent la dynamique interne du modèle. Par conséquent, elles peuvent dégrader la qualité globale du mouvement généré et diminuer le taux de succès sur la tâche [102]. De surcroît, le coût computationnel d'une résolution de contraintes répétée à chaque étape d'intégration limite fortement leur utilisation pour du contrôle réactif en boucle fermée.

2.5.2 Filtres de sécurité appliqués à l'exécution

Pour préserver l'intégrité du modèle génératif, la seconde stratégie maintient la phase de génération nominale totalement inchangée et n'applique les contraintes de sécurité qu'en aval, lors de la commande physique du robot. L'archétype de ce paradigme est le cadre PACS (*Path-Consistent Safety filtering*) proposé par Römer et al. [102] pour les politiques de diffusion. Plutôt que de déformer géométriquement les actions générées, PACS convertit chaque bloc d'actions en une trajectoire d'intention, puis applique un freinage cohérent avec le chemin (*path-consistent braking*) : le robot est ralenti ou arrêté le long de cette trajectoire, dont la sécurité est certifiée en temps réel par une analyse d'atteignabilité ensembliste. Ce filtrage externe maintient l'exécution dans la distribution d'entraînement de la politique et fournit des garanties formelles de sécurité en environnement dynamique, en supposant toutefois des mesures de position des objets à erreur bornée. Les auteurs montrent qu'à l'inverse, un filtre réactif naïf de type CBF appliqué à l'effecteur pousse la politique hors de sa distribution d'entraînement, avec des taux de réussite inférieurs allant jusqu'à 68 % en simulation et 37 % sur plateforme réelle. Cette cohérence au chemin restreint cependant la correction au seul freinage : face à un obstacle qui bloque durablement la trajectoire nominale, le robot s'immobilise sans pouvoir le contourner, et les auteurs identifient eux-mêmes la gestion des obstacles statiques ou quasi-statiques comme une perspective future. Ce constat motive une correction géométrique locale qui, lorsqu'elle doit s'écarter de la trajectoire nominale, cherche à en rester le plus proche possible afin de limiter la dérive distributionnelle.

Dans cette perspective, d'autres approches proposent d'appliquer des corrections locales réactives (comme FlowCorrect [103]) pour adapter à la volée les trajectoires nominales à partir de signaux sporadiques, résolvant des cas d'échecs complexes sans réentraîner le réseau. Enfin, des filtres temporels spécifiques permettent également de compenser les délais d'inférence inhérents aux grands modèles génératifs [104, 105].

2.5.3 Découplage entre intention nominale et sécurité réactive

L'analyse croisée de ces deux familles d'approches justifie l'architecture découplée mise en œuvre dans ce travail. Plutôt que d'entrelacer la sécurité et la génération nominale, nous optons pour une isolation fonctionnelle complète entre :

1. **Un planificateur global en boucle ouverte** basé sur le *Flow Matching*, chargé de générer l'intention nominale à long terme sans subir de dérive distributionnelle.
2. **Un filtre de sécurité réactif local** basé sur une projection QP-CBF, s'exécutant à haute fréquence en ligne pour intercepter les collisions physiques sur le robot.

Ce découplage fonctionnel concilie efficacement la richesse et la multimodalité des trajectoires apprises par le modèle génératif avec la réactivité et la rigueur théorique indispensables pour améliorer la sécurité physique sur une plateforme réelle. Ce principe de séparation constitue le fil conducteur de la méthodologie proposée dans ce mémoire.

2.6 Synthèse critique et positionnement

Objectif de la section : Résumer les lacunes de l'état de l'art, formaliser l'hypothèse de recherche et présenter un tableau comparatif synthétique des architectures de la littérature.

2.6.1 Lacunes identifiées dans la littérature

L'analyse croisée de la littérature scientifique contemporaine met en évidence trois lacunes majeures :

1. **L'hypothèse d'une perception parfaite :** La plupart des cadres de travail actuels validant l'alliance du *Flow Matching* et des CBF supposent des obstacles analytiques parfaits connus a priori, se montrant ainsi inadaptés face aux nuages de points 3D bruts et bruités.
2. **L'absence de persistance temporelle des modèles génératifs :** Lors d'une occlusion temporaire de la cible, les planificateurs génératifs perdent leur signal de conditionnement géométrique et produisent des trajectoires aberrantes ou figées, faute d'une couche cartographique persistante capable de stabiliser les points de conditionnement.
3. **Le coût numérique des représentations globales :** Les approches basées sur la mise à jour de champs de distance globaux ou de *Neural SDF* s'avèrent trop lourdes pour soutenir une boucle de rétroaction à la fréquence de commande réactive face à des obstacles imprévus.

2.6.2 Positionnement de l'architecture proposée

Pour répondre à ces limites, ce mémoire soutient la pertinence d'une *architecture découplée entre génération nominale et filtrage réactif de sécurité*. La trajectoire nominale est générée en boucle ouverte par un modèle *Flow Matching* conditionné par une observation géométrique persistante issue de la perception RGB-D et du suivi SAM2, tandis que la sécurité immédiate est déléguée à un filtre de projection CBF articulaire bas niveau s'appuyant sur une approximation polynomiale de Bernstein du SDF local du robot.

2.6.3 Hypothèse de recherche

L'hypothèse centrale de cette recherche s'énonce ainsi :

« L'utilisation d'un filtre SDF-CBF fondé sur un SDF local du robot, approximé par des polynômes de Bernstein et alimenté par une carte perceptuelle persistante, permet de corriger en temps réel les vitesses nominales générées par un modèle *Flow Matching*, afin d'améliorer l'exécution sécuritaire de trajectoires dans des environnements quasi-statiques, non structurés et inconnus, mesurée par le taux de réussite de la tâche, la distance minimale de séparation aux obstacles et la durée des violations de la marge de sécurité, sans réentraîner le modèle génératif ni construire un champ de distance global de l'environnement. »

Le Tableau 2.1 résume le positionnement de l'approche proposée vis-à-vis des cadres de référence de la littérature. Les critères de comparaison s'y lisent comme suit : le modèle nominal désigne la politique générative produisant l'intention de mouvement ; la sécurité qualifie la nature du mécanisme d'évitement (implicite lorsqu'elle est seulement apprise des démonstrations, souple lorsqu'elle guide la génération sans imposer de contrainte, explicite lorsqu'une contrainte est imposée à l'exécution, formelle lorsqu'elle est certifiée mathématiquement) ; la représentation précise la forme sous laquelle les obstacles sont fournis au mécanisme de sécurité ; la correction décrit comment la sortie nominale est modifiée ; la colonne occlusions indique si le conditionnement perceptif résiste à une perte temporaire de visibilité ; enfin, la réactivité évalue la capacité à réagir à un changement de scène à la fréquence de commande.

En somme, l'examen critique de la littérature scientifique met en évidence un compromis entre l'expressivité des politiques génératives et la rigueur des algorithmes de préservation de la sécurité. Si le *Flow Matching* lissant et déterministe permet de capturer efficacement la multimodalité des mouvements experts tout en réduisant la latence propre aux processus de diffusion stochastique, son exécution en boucle ouverte au sein d'environnements réels et changeants demeure sujette aux incertitudes perceptives. Les méthodes contemporaines tentant d'incorporer des contraintes au cours de la phase de génération tendent à alourdir le temps de calcul ou à provoquer une dérive distributionnelle nuisible pour le succès de la tâche. Afin d'apporter une solution pragmatique à ces verrous technologiques, le Chapitre 4 détaillera la méthodologie de l'architecture découplée proposée dans ce travail. Nous y exposerons comment une architecture de perception persistante s'articule de manière synchrone avec un filtre réactif basé sur une approximation par polynômes de Bernstein pour améliorer, en temps réel et sous des contraintes vérifiables localement, l'exécution sécurisée définie au cœur de notre hypothèse de recherche.

Tableau 2.1 Tableau comparatif des frameworks de planification et de sécurité de l'état de l'art.

Cadre / Méthode	Modèle nominal	Sécurité	Représentation	Correction	Occlusions	Réactivité
Diffusion Policy	Diffusion	Implicite	Espace latent	Replanification en horizon fuyant, sans filtre de sécurité	Non	Limitée par le débruitage
OmniGuide	Flow Matching	Souple	Grille / énergie	Guidage intra-ODE	Non	Moyenne
CASF	Streaming Flow	Souple	Neural SDF	Déformation intra-ODE	Partielle	Élevée mais coûteuse
SafeFlow-Matcher	Flow Matching	Explicite CBF	Obstacles analytiques	Correction intra-génération	Non	Élevée en simulation
PACS	Diffusion	Formelle : atteignabilité ensembliste	Objets suivis, erreur bornée	Freinage sur la trajectoire nominale, sans contournement	Non	Très élevée (1 kHz)
Notre approche	Flow Matching	Explicite : projection SDF-CBF	Nuage filtré persistant + SDF robot	Correction vitesse articulaire à l'exécution	Oui, via persistance	Élevée, mesurée expérimentalement

CHAPITRE 3 MOTIVATION DE RECHERCHE ET OBJECTIFS

Ce chapitre a comme objectif de présenter les objectifs de ce mémoire et de définir la portée ainsi que les contraintes autour du développement du système.

3.1 Positionnement de recherche et motivation

L'intégration d'un filtre de sécurité SDF-CBF fondé sur un SDF local du robot permet de corriger en temps réel les vitesses nominales générées par un modèle Flow Matching, afin d'exécuter des trajectoires dans des environnements non vus tout en réduisant les risques de collision et en préservant l'intention de la trajectoire nominale.

3.2 Objectifs de recherche

3.2.1 Objectif principal

Développer un système permettant de réaliser des trajectoires générées par intelligence artificielle générative sans collision dans un environnement quasi-statique non vue durant l'entraînement sans réentraîner le modèle en temps réel.

3.2.2 Objectifs spécifiques

- **SO1** : Implémenter un système de détection et de segmentation afin de séparer la cible de l'environnement.
- **SO2** : Récolter des données
- **SO3** :
- **SO4** : Intégrer un bouclier basé sur des fonctions de contraintes barrières pour éviter les collisions du robot avec l'environnement
- **SO5** : Valider le système complet en simulation sur un robot Franka dans différents environnements quasi-statiques en mesurant les temps de calculs de chaque sous système et le taux de succès des différentes tâches basé sur la complétion des tâches ainsi que l'évitement des collisions.

3.3 Portée et exclusions de la recherche

l'objectif n'est pas de déterminer le nombre de caméra nécessaire pour réaliser cette tâche, n'a pas été testé sur un robot réel, testé seulement sur deux scénarios, pas de déterminer le nombre de trajectoires nécessaires pour entraîner le modèle,4.5

CHAPITRE 4 DESIGN DU SYSTÈME

Ce chapitre présente

4.1 Plateforme Expérimentale Et Architecture Globale

Cette section a comme objectif de présenter l’architecture globale du système développé dans le cadre de cette recherche. Elle couvre la fondation et les données d’entrée et de sortie de chacun des modules utilisés pour réaliser l’ensemble des objectifs listés au chapitre 3.

4.1.1 Scène robotique, capteur RGB-D, robot manipulateur

Dire que cela fonctionne pour les robots avec cinématique redondante

4.1.2 Architecture modulaire du système

La figure 4.1 illustre l’architecture du système développé. Elle met de l’avant les relations entre chaque sous-système et comment les données d’entrée, soit des données RGB-D, sont utilisées pour exécuter des trajectoires générées par IA sans collision en temps réel. Ces données sont d’abord utilisées pour détecter la cible et créer deux nuages de points distincts : celui de la cible et celui de l’environnement au temps t . Le nuage de points de la cible est fusionné au nuage de points de l’outil pour conditionner le modèle de *Flow Matching* et générer des trajectoires en boucle fermée. Le nuage de points de l’environnement, quant à lui, est utilisé pour générer un nuage de points persistant de l’environnement afin de résister aux occlusions. Ce nuage de points persistant est utilisé pour contraindre les trajectoires et éviter les collisions en temps réel.

4.2 Perception Sémantique et Reconstruction 3D

4.2.1 Extraction Sémantique

Afin de détecter les objets à détecter pour contraindre le modèle de génération de trajectoire utilisée, le modèle de fondation *Segment Anything Model 3* (SAM 3) de meta [55] est utilisé. Ce modèle permet de calculer la boîte englobante, la segmentation au niveau du pixel, nommé masque, ainsi qu’un score de confiance de détection à partir d’une photo RGB et d’un prompt. Le fait que le masque soit obtenu directement à la sortie du modèle permet par la suite

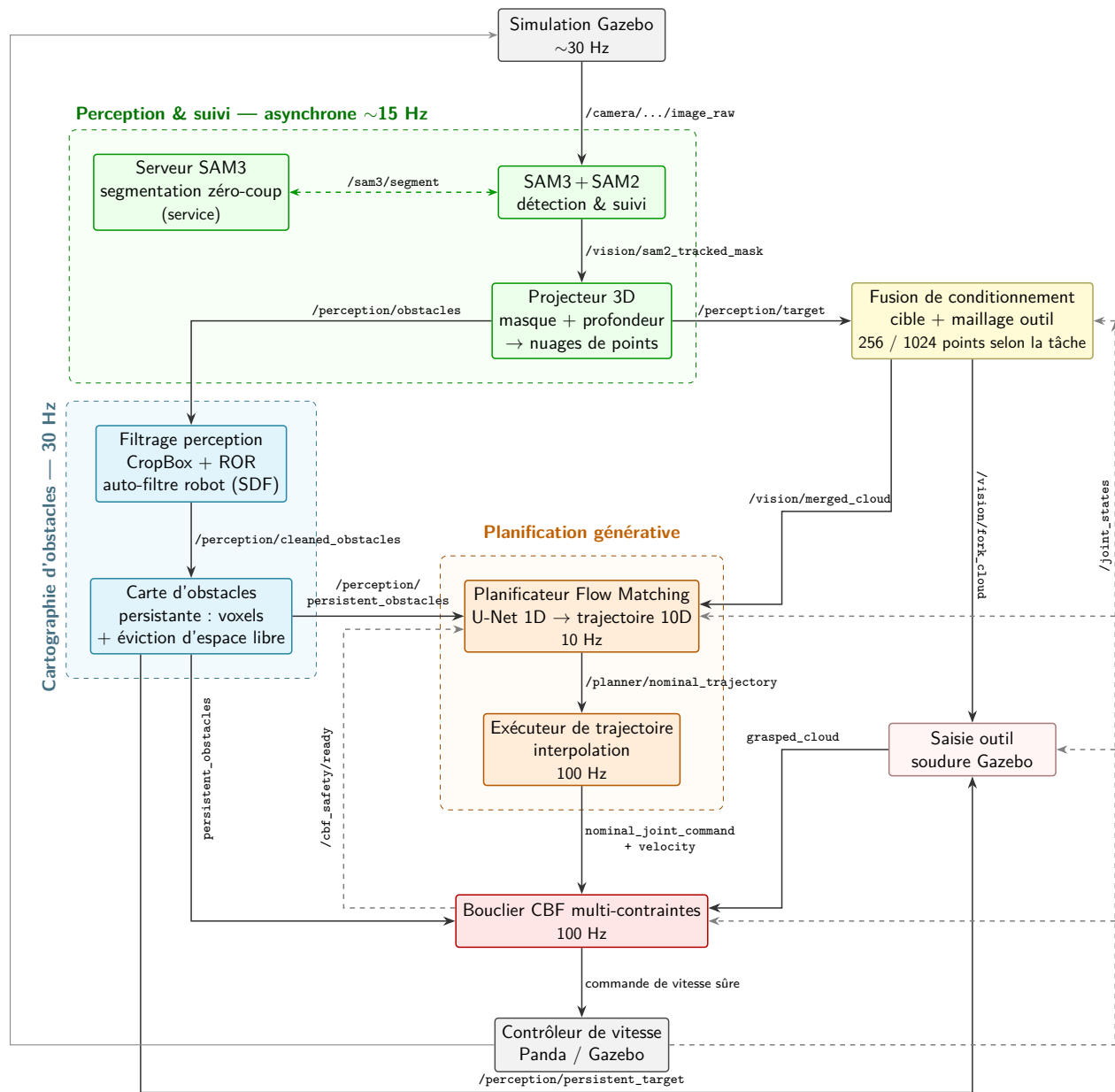


Figure 4.1 Architecture globale du système asynchrone.

à un modèle contenant moins de paramètres de faire le suivi des objets détectés par SAM 3, permettant ainsi de grandement diminuer le temps d'inférence pour le suivi temporel. Ce modèle de fondation a été entraîné sur plus de quatre millions de concepts textuels et 38 millions de phrases synthétiques, ce qui lui confert la capacité de détecter une quantité quasi illimitée d'objets. La figure 4.2 présente un exemple d'un masque obtenu suite à la demande *Pink Block*.



Figure 4.2 Exemple de masque obtenu par le modèle de segmentation SAM3 pour le prompt *Pink Block*

4.2.2 Suivi temporel de la cible

À partir du masque détecté par SAM 3 à la section 4.2.1, le suivi...

4.2.3 Projection RGB-D vers l'espace 3D

4.2.4 Séparation cible, obstacles et outil visible

4.2.5 Nettoyage spatial du nuage d'obstacles

4.3 Cartes Persistantes de la scène

4.3.1 Motivation : robustesse aux pertes de détection et occlusion

On veut pouvoir éviter les occlusions et les pertes de détection de SAM 3, en gardant en mémoire les surfaces des objets

4.3.2 Génération des cartes persistantes

4.3.3 Mise à jour des cartes persistantes

Conservation des points hors frame

Voxelisation de la map pour alléger les calculs

Suppression des doublons dans chaque Voxelisation

fusion avec la carte existante

jusqu'à la limite $\max_voxels$ (suppression des points par vieillissement temporel par la suite pour voir les points les plus proches et les nouveaux points)

Suppression des voxels supprimées si $z_{measure} - z_{voxel} > seuil$ (car la caméra voit là où le voxel était supposé caché)

4.4 Acquisition des données et représentation des démonstrations

Cette section présente le protocole utilisé pour collecter les démonstrations qui servent à entraîner le modèle génératif, puis les étapes de prétraitement qui transforment ces enregistrements bruts en données d'entraînement.

4.4.1 Protocole de collecte des démonstrations

Les démonstrations proviennent de deux sources complémentaires : le robot et les capteurs réels d'une part, la simulation d'autre part, cette dernière reposant sur la librairie *ManiSkill 3*, retenue pour sa facilité d'utilisation et son moteur physique à l'état de l'art fondé sur

SAPIEN [106]. Les deux sources reproduisent des environnements quasi identiques, de sorte que les données y sont récoltées de la même manière.

Pour garantir un protocole reproductible, l'échantillonnage des configurations est défini dans deux repères imbriqués plutôt que de façon qualitative : le repère robot $\mathcal{F}_b$ (base du manipulateur, fixe) et un repère assiette $\mathcal{F}_p$ centré sur l'assiette. L'assiette est placée successivement aux 9 positions d'une grille 3×3 exprimée directement dans $\mathcal{F}_b$: centrée à $(0,40, 0)$ m, de pas 0,08 m selon x_b et 0,20 m selon y_b . Pour chacune, la cible occupe 5 positions disposées en quinconce dans $\mathcal{F}_p$ (demi-écart de 0,06 m), chacune assortie d'une orientation aléatoire. Cette grille produit $9 \times 5 = 45$ configurations, chacune réalisée une fois. La Figure 4.3 illustre cet échantillonnage. À chaque configuration, l'outil descend verticalement vers la cible (axe $-z$ de $\mathcal{F}_b$) en maintenant les dents verticales et alignées sur l'axe principal de la cible.

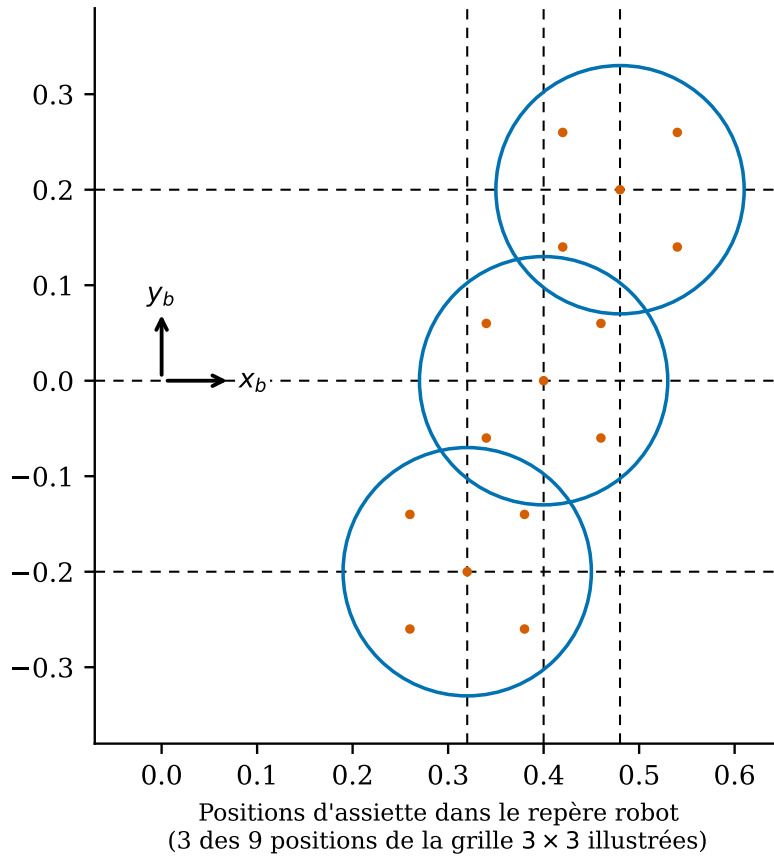


Figure 4.3 Protocole d'échantillonnage des cibles, vu de dessus dans le repère de base du robot $\mathcal{F}_b$. Les assiettes (26 cm de diamètre) se chevauchant, seules trois des neuf positions de la grille 3×3 sont illustrées (supérieur droit, centre, inférieur gauche), le centre de chacune étant marqué par une croix noire. Autour de chaque centre, les 5 points indiquent les positions de cible en quinconce.

Chaque démonstration réelle est fournie par guidage kinesthésique : le robot, amené dans la configuration articulaire initiale $\mathbf{q}_0 = [0, -0,126, 0, -2,193, 0, 2,065, 0,786]$ rad puis placé en mode de guidage manuel, est conduit par l’opérateur depuis cette posture jusqu’à la cible selon le schéma d’approche décrit ci-dessus. Pendant l’exécution, la chaîne d’acquisition échantillonne à 10 Hz la pose du TCP dans le repère de base (position cartésienne et orientation en quaternion), sa vitesse linéaire et angulaire, ainsi que le flux RGB-D de la caméra montée à l’effecteur, chaque échantillon étant horodaté depuis le début de la trajectoire. La mise en œuvre de cette chaîne (deux nœuds ROS dédiés), le format d’enregistrement et l’organisation des fichiers sont détaillés à l’Annexe A.

En simulation, l’environnement réplique la scène réelle : même configuration articulaire initiale $\mathbf{q}_0$, même montage de la caméra à l’effecteur et même cadence d’enregistrement de 10 Hz, le robot y étant équipé de la fourchette par composition de son modèle URDF. Les démonstrations n’y sont toutefois pas fournies par guidage kinesthésique, mais par une politique experte scriptée qui reproduit le geste de l’opérateur en trois phases : approche alignée sur l’axe principal de la cible en conservant les dents de la fourchette à la vertical, redressement du poignet qui ramène les dents à l’horizontale, puis transport vers une pose de livraison fixe ; une fois piquée, la cible est rigidement attachée à la fourchette. L’échantillonnage des configurations y est continu plutôt que discret. La position de la cible est tirée uniformément dans un rectangle du plan de travail délimité par le champs de vision de la caméra, son orientation uniformément sur $[-\pi, \pi]$, et sa géométrie parmi sept aliments maillés. Les sept aliments ont été sélectionnés pour leur différentes formes offrant ainsi une distribution plus élevée de la forme de la cible. Ces derniers sont : une rondelle de carotte, une poitrine de poulet, une saucisse, un sushi, un fleuron de brocoli, une pomme de terre, un cube et un nigiri. La randomisation de position de l’aliment étend en outre le protocole réel selon la hauteur. Pour ce faire, la cible repose sur un piédestal dont la hauteur est tirée uniformément entre 4 et 10 cm. Quinze démonstrations simulées complètent ainsi les 45 démonstrations réelles de la tâche d’alimentation. La Figure 4.4 illustre ces trois phases.

Le jeu de données de saisie et dépôt (*pick-and-place*), entièrement simulé, suit le même principe avec le Panda standard. Le cube y est un corps dynamique réellement saisi par la pince, sans attachement artificiel, et la politique experte enchaîne cinq phases listées ci-dessous :

1. Descente : Approche du TCP du robot vers la cible pour centrer son centroïde au niveau du TCP.
2. Saisie : fermeture de la pince pour agripper la cible.
3. Transport : déplacement du TCP du robot selon un arc dont les pentes de départ et

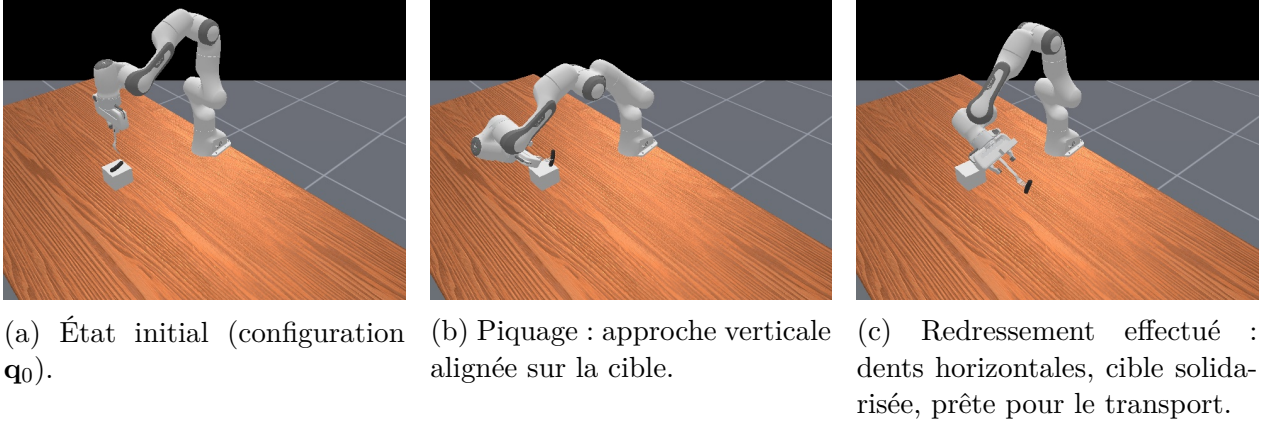


Figure 4.4 Trois phases de la politique experte simulée pour la tâche d'alimentation.

d'arrivée sont prescrites à 60° et -60° par rapport à la table, faisant par le fait même varier la hauteur.

4. Dépôt : Ouverture de la pince pour déposer la cible dans la zone blanche.
5. Retrait : Soulèvement de la pince à une hauteur de 5 cm.

Les positions du cube et de la boîte de dépôt sont tirées uniformément dans leurs zones respectives à chaque épisode, la randomisation de la boîte évitant que le modèle n'apprenne une position de dépôt fixe ; 40 trajectoires sont ainsi collectées. Le détail des politiques expertes et des plages de randomisation est donné à l'Annexe A. La Figure 4.5 illustre trois de ces phases (descente, transport, dépôt).

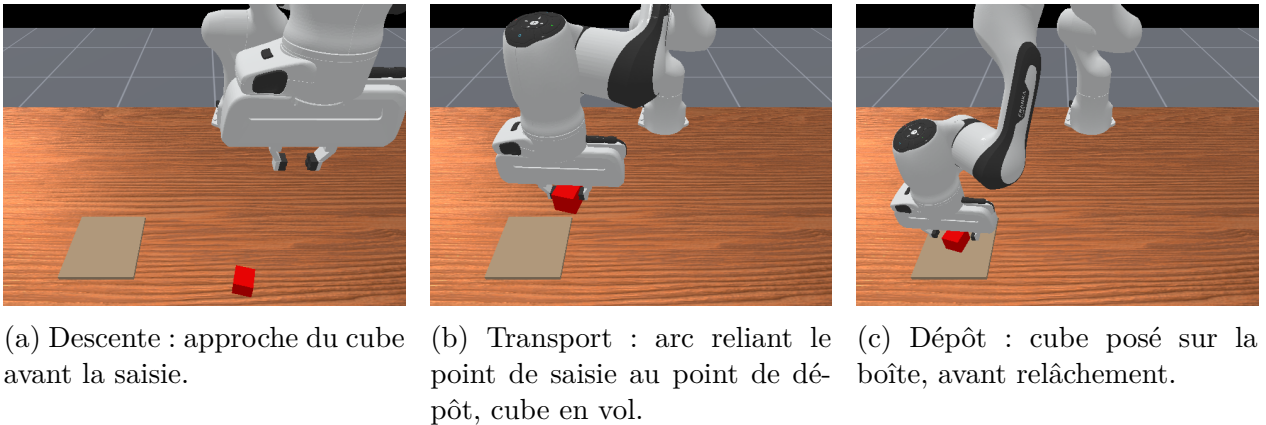


Figure 4.5 Trois phases de la politique experte simulée pour la tâche de saisie et dépôt.

4.4.2 Prétraitement des données

Les enregistrements bruts issus du protocole de la Section 4.4.1 contiennent les poses du TCP et les flux RGB-D, mais pas le nuage de points conditionnel attendu par le modèle génératif de la Section 4.5. Un prétraitement hors ligne les transforme donc en données d’entraînement en trois étapes : la détection de la cible dans la première image de chaque trajectoire, la génération du nuage de points conditionnel à chaque pas, puis la mise à jour du fichier de trajectoire. Réaliser ces étapes hors ligne évite de recalculer la perception à chaque époque d’entraînement et permet de valider visuellement les nuages générés avant celui-ci.

Détection de la cible par segmentation

La première étape consiste à détecter la cible dans la première image de la caméra à l’effecteur de chaque trajectoire. Cette détection est réalisée avec le modèle SAM 3 présenté à la Section 4.2.1 : l’image, préalablement convertie au format `.jpg` attendu par le modèle, lui est fournie accompagnée d’une requête textuelle décrivant la cible (*Pink block*, *Broccoli*, *Carrot slice*, *Chicken breast*, *Sushi*, *Nigiri*, *Sausage* ou *Potato* pour la tâche d’alimentation ; *Red Cube* et *Brown box* pour la tâche de saisie et dépôt), et le masque au score de confiance le plus élevé est retenu. La détection n’est effectuée que sur la première image en raison de la distance minimale de mesure du capteur de profondeur, de 45 cm selon le fabricant : l’approche amène rapidement la caméra sous cette distance, de sorte que seule les premières images garantissent une carte de profondeur valide de la cible.

Génération du nuage de points conditionnel

Le modèle de Flow Matching décrit à la Section 4.5 étant conditionné sur les nuages de points de l’outil et de la cible, un nuage fusionné est généré pour chaque pas de chaque trajectoire. Cette génération s’appuie sur quatre éléments : le masque obtenu à l’étape précédente, la carte de profondeur initiale, la pose du TCP à chaque pas et le modèle CAO de l’outil.

Le nuage de la cible est d’abord reconstruit dans le repère de la caméra. Le masque est superposé à la carte de profondeur initiale afin de ne conserver que les pixels de la cible dotés d’une profondeur valide, puis chaque pixel (u, v) de profondeur Z est rétroprojeté par

le modèle sténopé :

$$X_c = \frac{Z}{f_x}(u - c_x) \quad (4.1)$$

$$Y_c = \frac{Z}{f_y}(v - c_y) \quad (4.2)$$

$$Z_c = Z \quad (4.3)$$

où f_x et f_y sont les longueurs focales de la caméra et c_x et c_y les coordonnées de son point principal, les paramètres intrinsèques de la caméra D415 utilisée ayant été obtenus auprès du fabricant. Le nuage résultant est ensuite exprimé dans le repère de la base du robot par la relation (4.4), évaluée à la pose du TCP du premier pas, où $\mathbf{T}_{cible}^b$ est la matrice de transformation entre la cible et la base du robot. Cette relation se déduit de la Figure 4.6, qui représente l'ensemble des repères du système : $\mathcal{F}_b$ le repère de la base du robot, $\mathcal{F}_c$ celui de la caméra à l'effecteur, $\mathcal{F}_e$ celui du TCP et $\mathcal{F}_o$ celui de l'outil utilisé.

$$\mathbf{T}_{cible}^b = \mathbf{T}_e^b \mathbf{T}_c^e \mathbf{T}_{cible}^c \quad (4.4)$$

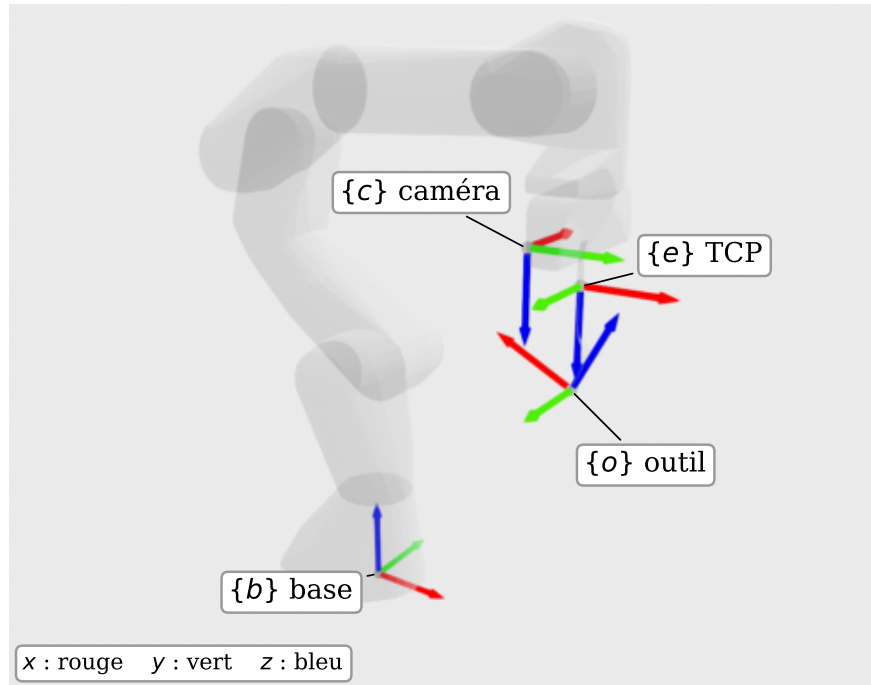


Figure 4.6 Représentation schématique des différents repères du système

Le nuage de l'outil est quant à lui obtenu à partir de son modèle CAO, dont la surface est échantillonnée avec la librairie `trimesh` (fonction `trimesh.sample.sample_surface`). Ce

nuage, défini dans le repère de l'outil $\mathcal{F}_o$, est positionné dans le repère de base à chaque pas par la relation (4.5), où $\mathbf{T}_o^e$ est la transformation constante entre le TCP et l'outil.

$$\mathbf{T}_o^b = \mathbf{T}_e^b \mathbf{T}_o^e \quad (4.5)$$

À noter que $\mathbf{T}_o^e = \mathbf{I}$ dans le cas où l'outil est le TCP, comme pour la tâche de saisie et dépôt.

La carte de profondeur n'étant exploitable qu'au premier pas, la position de la cible aux pas suivants est reconstruite géométriquement plutôt que mesurée. Avant la saisie, la cible repose sur son support et son nuage demeure à sa position initiale. À partir de l'instant de saisie, elle est considérée rigidement solidaire de l'outil et son nuage suit le mouvement de celui-ci selon la relation (4.6),

$$\tilde{\mathbf{p}}_k = \mathbf{T}_{o,k}^b \left(\mathbf{T}_{o,g}^b \right)^{-1} \tilde{\mathbf{p}}_0, \quad k \geq g \quad (4.6)$$

où $\tilde{\mathbf{p}}_0$ est un point de la cible en coordonnées homogènes à sa position initiale, $\tilde{\mathbf{p}}_k$ ce même point au pas k , et g l'indice du pas de saisie. Cette reconstruction suppose une saisie ferme et sans glissement, hypothèse cohérente avec le protocole de collecte de la Section 4.4.1.

L'indice de saisie g est déterminé différemment selon la tâche. Pour la tâche d'alimentation, la saisie s'effectue par piquage : les dents de la fourchette pénètrent la cible, de sorte que la distance euclidienne entre la pointe de l'outil, positionnée par la relation (4.5), et le centroïde de la cible décroît durant l'approche et atteint son minimum à l'insertion maximale. Cette distance est donc évaluée à chaque pas, lissée par un filtre gaussien d'écart-type de deux pas d'échantillonnage afin d'atténuer le bruit sur la pose et sur le centroïde, et l'indice de son minimum définit g .

Pour la tâche de saisie et dépôt, la saisie s'effectue par fermeture de la pince. Un état binaire de la pince, ouverte ou fermée, est plutôt dérivé à chaque pas des données enregistrées, à partir de la commande envoyée à la pince ou, à défaut, de l'ouverture moyenne des doigts comparée à un seuil de 3 cm. La première transition vers l'état fermé définit g . La relation (4.6) solidarise alors le cube au TCP jusqu'à la réouverture de la pince au dépôt, après quoi son nuage demeure à sa position de dépôt. La boîte de dépôt, qui n'est jamais saisie, conserve sa position initiale pendant toute la trajectoire.

Les nuages de l'outil et de la cible sont ensuite fusionnés, puis sous-échantillonnés à 1024 points par la stratégie d'échantillonnage du point le plus éloigné, qui préserve la couverture géométrique des deux objets. Pour la tâche de saisie et dépôt (*pick-and-place*), le cube à saisir et la boîte de dépôt sont détectés puis reconstruits par la même chaîne de segmentation et de rétroprojection ; le nuage fusionné compte également 1024 points, répartis entre la pince (40 %), le cube (20 %) et la boîte de dépôt (40 %), avec la même stratégie de sous-

échantillonnage.

Le nuage fusionné est finalement centré, en translation seulement, sur l'origine du repère de l'outil. La position de l'outil est soustraite de chaque point, sans qu'aucune rotation ne soit appliquée, de sorte que les axes du nuage restent alignés sur ceux du repère de base $\mathcal{F}_b$. Ce centrage rend la représentation invariante à la position absolue de la scène dans l'espace de travail, ce qui favorise la généralisation aux différentes configurations du protocole, tout en préservant l'information d'orientation. Un changement de repère complet vers $\mathcal{F}_o$, rotation comprise, serait au contraire dégénéré : après la saisie, l'outil et la cible sont immobiles dans le repère de l'outil et l'observation deviendrait constante, en plus de masquer la direction de la gravité, nécessaire pour distinguer les différentes phases.

Mise à jour des données de trajectoire

La dernière étape recopie le fichier JSON de chaque trajectoire dans le jeu de données prétraité et l'augmente de quatre éléments : le nom du fichier du nuage conditionnel associé à chaque pas ; la pose de l'outil dans le repère de base $\mathcal{F}_b$ à chaque pas (position et orientation en quaternion), calculée par la relation (4.5) à partir de la pose du TCP enregistrée ; l'état binaire de la pince $s_k \in \{0, 1\}$ à chaque pas, dérivé des données enregistrées comme décrit plus haut pour la tâche de saisie et dépôt, et constant à 1 pour la tâche d'alimentation, la pince y demeurant fermée sur la fourchette pendant toute la trajectoire ; et la transformation constante $\mathbf{T}_o^e$, consignée une seule fois à la racine du fichier. La pose de l'outil et l'état binaire de la pince forment ensemble l'observation proprioceptive à dix dimensions du modèle de la Section 4.5 et définissent l'espace dans lequel les trajectoires sont générées. L'organisation des fichiers du jeu de données prétraité est détaillée à l'Annexe A.

4.5 Génération de trajectoire par Flow Matching

Cette section présente le modèle génératif du système : la formulation mathématique du Flow Matching conditionnel, l'architecture qui la réalise, puis la procédure d'entraînement et ses hyperparamètres. Le même modèle, avec les mêmes hyperparamètres, est employé pour les deux tâches ; seul le jeu de données d'entraînement change.

4.5.1 Formulation mathématique du Flow Matching conditionnel

Le Flow Matching [33, 35] est un paradigme génératif qui transporte une distribution source simple p_0 , ici la gaussienne standard, vers la distribution cible des données p_1 . Ce transport est décrit par un champ de vitesses dépendant du temps u_t , dont le flot ψ_t est défini par

l'équation différentielle ordinaire (4.7) avec la condition initiale $\psi_0(x) = x$:

$$\frac{d}{dt}\psi_t(x) = u_t(\psi_t(x)) \quad (4.7)$$

Un échantillon $X_0 \sim p_0$ transporté par ce flot, $X_t = \psi_t(X_0)$, suit une courbe continue dans l'espace des données ; si le champ u_t est bien choisi, la distribution de X_1 coïncide avec p_1 , et générer un échantillon revient à intégrer numériquement cette équation de $t = 0$ à $t = 1$.

Plutôt que d'apprendre ce champ implicitement comme le font les modèles de diffusion, le Flow Matching le régresse directement. Le chemin probabiliste retenu est l'interpolation linéaire (4.8) entre une paire bruit-donnée (X_0, X_1) ,

$$X_t = (1 - t)X_0 + tX_1 \quad (4.8)$$

dont la vitesse conditionnelle vaut $\dot{X}_t = X_1 - X_0$. L'objectif d'entraînement minimise l'écart quadratique entre le champ appris u_t^θ , paramétré par un réseau de neurones de poids θ , et cette vitesse :

$$\mathcal{L}(\theta) = \mathbb{E}_{t, X_0, (X_1, c)} \left\| u_t^\theta(X_t \mid c) - (X_1 - X_0) \right\|^2, \text{ où } t \sim \mathcal{U}[0, 1], X_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I}), (X_1, c) \sim \mathcal{D} \quad (4.9)$$

où $\mathcal{D}$ est le jeu de démonstrations et c l'observation associée à chaque segment, décrite plus bas. Lipman et al. démontrent que ce simple objectif conditionnel possède les mêmes gradients que la régression du champ marginal exact [33]. Deux propriétés motivent ce choix face aux modèles de diffusion : la cible de régression est une différence directe $X_1 - X_0$, sans paramétrisation de score ni pondération de bruit, et les chemins d'échantillonnage quasi rectilignes qu'induit l'interpolation linéaire autorisent une intégration précise en une dizaine de pas [35, 44], là où les modèles de diffusion en requièrent typiquement bien davantage ; cette économie est déterminante pour générer les trajectoires à la cadence de replanification visée.

Dans le cadre de ce travail, un échantillon X_1 est un segment de trajectoire de l'outil sur un horizon de $H = 16$ pas, $X_1 = [\mathbf{a}_1, \dots, \mathbf{a}_H] \in \mathbb{R}^{H \times 10}$. Chaque action $\mathbf{a}_k \in \mathbb{R}^{10}$ regroupe la position de l'outil (trois composantes), son orientation (six composantes) et l'état binaire de la pince (une composante), tels que définis à la Section 4.4.2. L'orientation adopte la représentation 6D continue de Zhou et al. [107], soit les deux premières colonnes de la matrice de rotation : contrairement au quaternion, cette représentation est exempte de discontinuités, ce qui en facilite la régression par un réseau de neurones ; le quaternion consigné dans les données prétraitées y est converti au chargement. Le champ de vitesses est enfin conditionné sur l'observation c de la scène, $u_t^\theta(X_t \mid c)$, de sorte que le modèle apprend la distribution

conditionnelle des segments experts $p_1(X_1 | c)$ plutôt que leur distribution marginale.

4.5.2 Architecture du modèle

Le modèle réalise le champ de vitesses conditionnel $u_t^\theta(X_t | c)$: il reçoit l’observation c , le segment intermédiaire X_t et le temps d’écoulement t , et produit une vitesse à dix dimensions pour chacune des H actions du segment.

L’observation regroupe le nuage de points conditionnel de la Section 4.4.2 (1024×3) et la proprioception des deux derniers pas, soit deux vecteurs à dix dimensions rassemblant la pose de l’outil et l’état binaire de la pince. Conformément au centrage du prétraitement, toutes les positions, celles du nuage comme celles des poses observées et générées, sont exprimées relativement à la position courante de l’outil, les orientations demeurant dans le repère de base. Avant d’entrer dans le réseau, les positions et l’état de la pince sont normalisés linéairement dans $[-1, 1]$ à partir des minima et maxima du jeu d’entraînement ; la représentation 6D de l’orientation, déjà bornée dans $[-1, 1]$, est laissée telle quelle.

Deux encodeurs transforment cette observation en un vecteur de conditionnement. Le nuage de points traverse l’encodeur léger de *3D Diffusion Policy* [29] : un perceptron multicouche partagé entre les points ($3 \rightarrow 64 \rightarrow 128 \rightarrow 256$, normalisation de couche et ReLU), suivi d’un max-pooling sur l’ensemble des points, qui rend le descripteur invariant à leur ordre, puis d’une projection linéaire vers $\mathbb{R}^{64}$. La proprioception traverse quant à elle un perceptron ($20 \rightarrow 128 \rightarrow 64$, activation Mish). Les deux descripteurs concaténés forment le vecteur de conditionnement $c \in \mathbb{R}^{128}$.

Le temps d’écoulement t doit également être fourni au réseau, le champ de vitesses (4.9) en dépendant explicitement. Un scalaire brut constituerait toutefois un signal trop pauvre pour moduler l’ensemble du réseau : t est plutôt encodé par le plongement sinusoïdal introduit avec l’architecture Transformer [108],

$$\gamma(t) = [\sin(\omega_1 t'), \dots, \sin(\omega_{d/2} t'), \cos(\omega_1 t'), \dots, \cos(\omega_{d/2} t')], \quad \omega_i = 10000^{-\frac{i-1}{d/2-1}} \quad (4.10)$$

où $d = 256$ est la dimension du plongement et $t' = 1000t$ une mise à l’échelle qui ramène l’intervalle $[0, 1]$ à la plage d’indices pour laquelle ces fréquences ont été conçues. La progression géométrique des pulsations ω_i fait varier chaque paire sinus-cosinus à une échelle temporelle différente : les basses fréquences distinguent grossièrement le début de la fin de l’écoulement, tandis que les hautes fréquences en résolvent les variations fines, le tout en demeurant une fonction lisse de t . Ce plongement est raffiné par un perceptron ($256 \rightarrow 1024 \rightarrow 256$, activation Mish), puis concaténé au vecteur c pour constituer le conditionnement global $c_g \in \mathbb{R}^{384}$

(128+256) : c'est par ce vecteur que la dépendance du champ u_t^θ au temps t et à l'observation c entre dans le réseau de vitesse.

Le réseau de vitesse est le U-Net 1D conditionnel popularisé par *Diffusion Policy* [25], illustré à la Figure 4.7. Sur cette figure, chaque triplet indique la forme du tenseur qui transite entre les blocs : B est la taille du lot, le deuxième terme le nombre de canaux et le troisième la longueur temporelle. Le segment $X_t \in \mathbb{R}^{H \times 10}$, noté x_t sur la figure, y est traité comme un signal unidimensionnel de longueur H à dix canaux, et la sortie $\mathbf{v}_t$, de même forme, est l'estimation de la vitesse $u_t^\theta(X_t \mid c)$ pour chacune des H actions. Le réseau comprend trois niveaux de résolution (256, 512 puis 1024 canaux, la longueur temporelle étant divisée par deux entre les niveaux), un goulot central, puis une remontée symétrique dont chaque niveau concatène les cartes du niveau descendant correspondant (connexions saut, opérateur C sur la figure). Tous les niveaux sont constitués de blocs résiduels conditionnels, détaillés à la Figure 4.8 : deux convolutions 1D, chacune suivie d'une normalisation de groupe et de l'activation Mish (4.11),

$$\text{Mish}(x) = x \cdot \tanh(\ln(1 + e^x)) \quad (4.11)$$

le conditionnement global c_g y étant appliqué par modulation affine de type FiLM : un gain $\gamma \in \mathbb{R}^{C_{\text{out}}}$ et un biais $\beta \in \mathbb{R}^{C_{\text{out}}}$, prédits par un perceptron à partir de c_g , modulent canal par canal les cartes de caractéristiques entre les deux convolutions. Sur la Figure 4.8, x désigne la carte de caractéristiques entrant dans le bloc, de forme (B, C_{in}, H) , où C_{in} et C_{out} sont les nombres de canaux à l'entrée et à la sortie du bloc et H la longueur temporelle au niveau de résolution courant, divisée par deux à chaque descente ; la connexion résiduelle traverse une convolution 1×1 lorsque $C_{\text{in}} \neq C_{\text{out}}$ afin d'accorder les dimensions. Cette injection du conditionnement à tous les niveaux de résolution permet à l'observation d'influencer autant la forme d'ensemble du segment que ses détails locaux.

Le choix de ce U-Net convolutif plutôt que d'une tête Transformer, l'autre architecture de débruitage proposée par *Diffusion Policy* [25], tient à trois considérations. D'abord, Chi et al. rapportent que la variante convolutive s'entraîne de façon stable et convient à la plupart des tâches sans réglage particulier, alors que la variante Transformer est nettement plus sensible aux hyperparamètres [25] ; avec un jeu de données limité à quelques dizaines de démonstrations par tâche, cette robustesse en régime de faibles données prime. Ensuite, les biais inductifs des convolutions temporelles, localité et régularité, correspondent à la nature des données : les trajectoires démontrées sont lisses et échantillonnées à 10 Hz, sans les changements d'action à haute fréquence pour lesquels la variante Transformer est recommandée. Enfin, l'horizon généré est court ($H = 16$) : l'atout principal de l'auto-attention, la modélisation des dépendances à longue portée, n'apporte rien ici, le goulot du U-Net couvrant déjà

l’horizon entier après deux sous-échantillonnages.

À l’inférence, la génération d’un segment consiste à intégrer l’équation (4.7) de $t = 0$ à $t = 1$ à partir d’un tirage $X_0 \sim \mathcal{N}(\mathbf{0}, \mathbf{I})$, par le schéma d’Euler explicite :

$$X_{t+\Delta t} = X_t + \Delta t u_t^\theta(X_t \mid c), \quad \Delta t = 1/N. \quad (4.12)$$

Le segment obtenu est dénormalisé puis converti en consignes exécutables : les positions absolues sont récupérées en réappliquant la position courante de l’outil, inverse du centrage en translation ; la matrice de rotation de chaque action est reconstruite par orthonormalisation de Gram–Schmidt de ses six composantes d’orientation [107] ; et l’état binaire de la pince est obtenu par seuillage de la dixième composante à 0,5. Le résultat est une séquence de $H = 16$ poses de l’outil accompagnées de la commande de la pince, transmise à la chaîne de conversion et d’exécution de la section suivante.

4.5.3 Entraînement du modèle

Les trajectoires prétraitées sont découpées en échantillons par fenêtre glissante : à chaque pas d’une trajectoire correspond un échantillon regroupant le nuage conditionnel courant, la proprioception des deux derniers pas et le segment des $H = 16$ actions à venir, soit 1,6 s à la cadence d’enregistrement de 10 Hz. Les trajectoires sont réparties entre entraînement (80 %) et validation (20 %).

Chaque échantillon d’entraînement subit une augmentation aléatoire qui préserve la cohérence géométrique entre le nuage et les poses : une rotation de la scène complète autour de la verticale, tirée uniformément dans $\pm 15^\circ$ et appliquée simultanément au nuage, aux positions et aux composantes d’orientation ; une translation uniforme de ± 1 cm appliquée au nuage et aux positions ; et un bruit gaussien d’écart-type 5 mm, tronqué à ± 1 cm, ajouté aux points du nuage pour imiter le bruit du capteur. L’état binaire de la pince n’est pas modifié par l’augmentation.

Le Tableau 4.1 rassemble les hyperparamètres d’entraînement, identiques pour les deux tâches. L’optimisation minimise la perte (4.9) avec AdamW, un pas d’apprentissage suivant un recuit en cosinus après une période d’échauffement et un écrêtage du gradient à une norme unitaire. Une moyenne mobile exponentielle (EMA) des poids du modèle est maintenue tout au long de l’entraînement et sert tant à l’évaluation qu’au déploiement, ce qui lisse les oscillations des derniers pas d’optimisation ; le point de contrôle retenu est celui qui minimise la perte de validation évaluée avec ces poids moyennés.

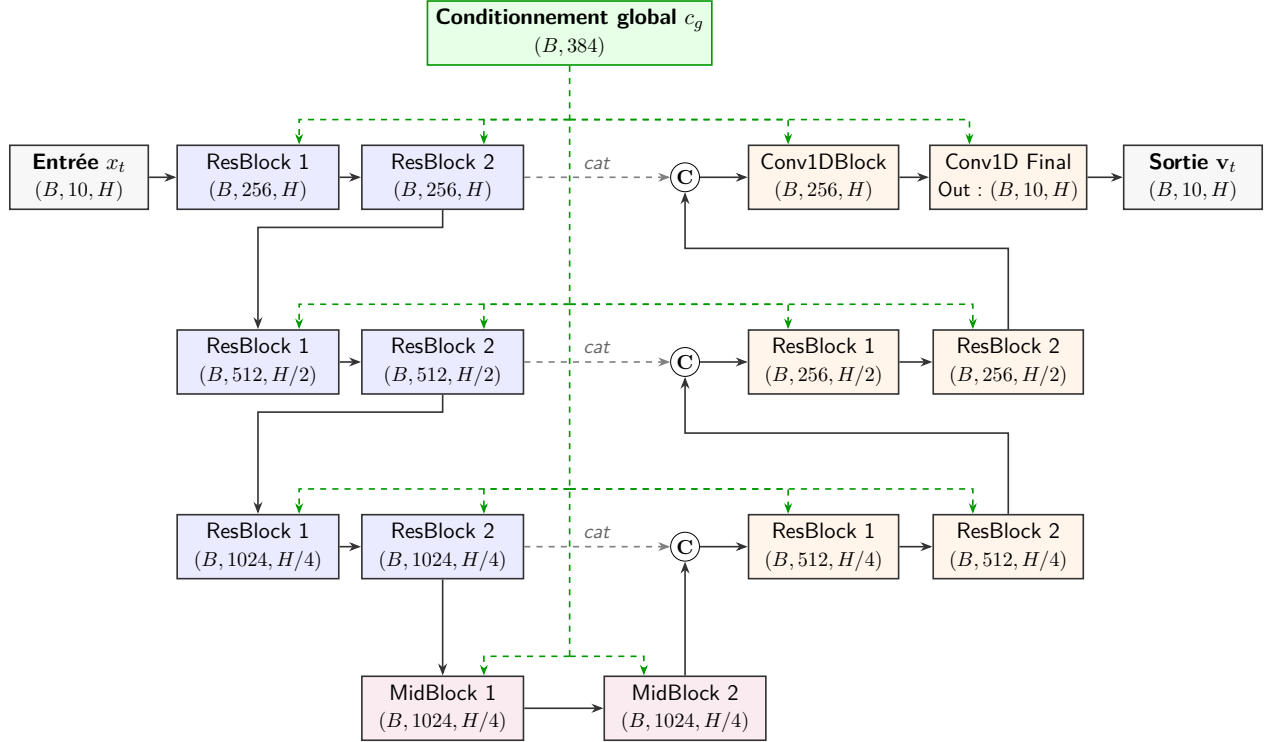


Figure 4.7 Architecture du réseau de vitesse **ConditionalUnet1D** : encodeur-décodeur 1D à trois niveaux de résolution, connexions saut et injection du conditionnement global dans chaque bloc résiduel. Chaque triplet indique la forme du tenseur transitant entre les blocs : taille de lot B , nombre de canaux, longueur temporelle ($H = 16$).

4.6 Conversion et exécution de la trajectoire nominale

La sortie du modèle de la section précédente est une séquence de poses de l'outil dans l'espace cartésien ; le robot, lui, est asservi dans l'espace articulaire. Cette section décrit la chaîne qui transforme chaque segment généré en une trajectoire articulaire exécutable : le passage des poses de l'outil aux poses du TCP, leur conversion en configurations articulaires par cinématique inverse, puis le réajustement temporel qui leur affecte des instants de passage produisant un profil de vitesse régulier.

4.6.1 Conversion outil–TCP

Le modèle de Flow Matching est entraîné et échantillonné dans le repère de l'outil $\mathcal{F}_o$: pour la tâche d'alimentation, les poses générées sont celles de la pointe de la fourchette, soit l'élément qui interagit physiquement avec la cible. Le contrôleur du robot n'asservit toutefois pas ce repère, mais celui du TCP $\mathcal{F}_e$. La pose du TCP à commander s'obtient en inversant

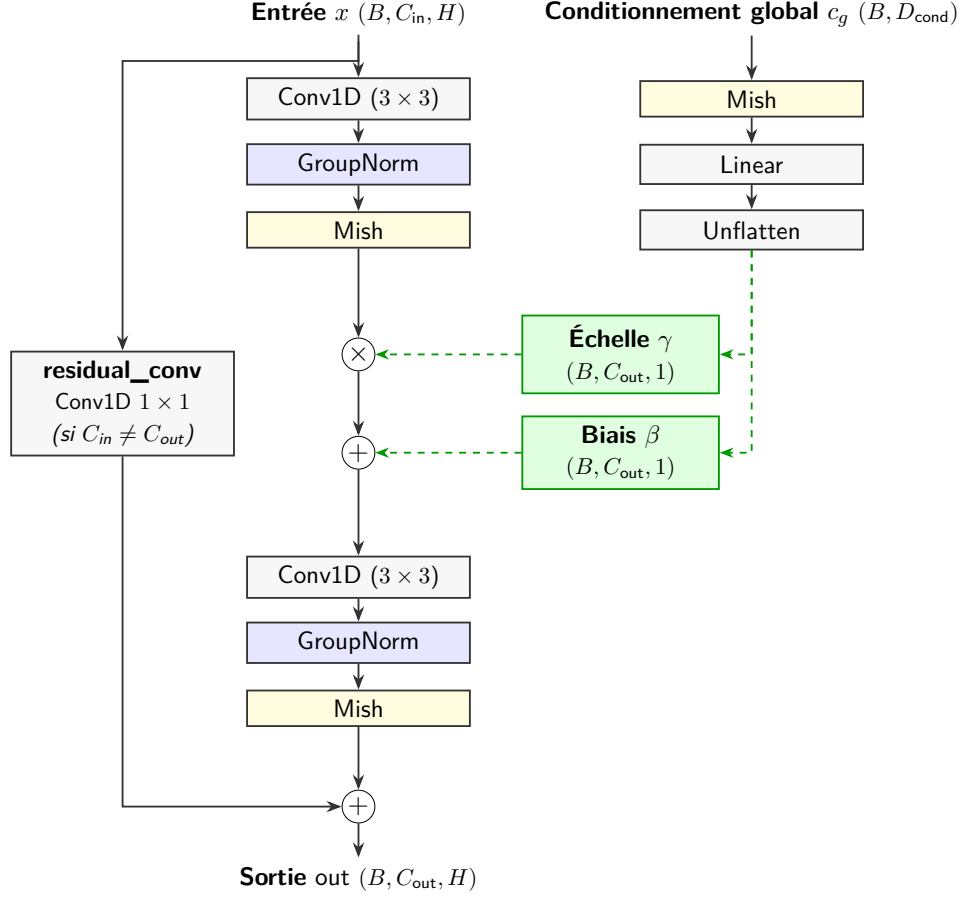


Figure 4.8 Architecture du bloc résiduel conditionnel (**ConditionalResidualBlock1D**) du réseau de vitesse, avec sa modulation affine par le conditionnement global. B désigne la taille du lot, C_{in} et C_{out} les nombres de canaux à l’entrée et à la sortie du bloc, H la longueur temporelle du niveau de résolution courant et $D_{cond} = 384$ la dimension du conditionnement global c_g .

la relation (4.5) utilisée au prétraitement (Section 4.4.2) :

$$\mathbf{T}_{e,k}^b = \mathbf{T}_{o,k}^b \left(\mathbf{T}_o^e \right)^{-1}, \quad k = 1, \dots, H, \quad (4.13)$$

où $\mathbf{T}_{o,k}^b$ est la pose homogène de l’outil au pas k , assemblée à partir de la position et de la matrice de rotation issues de l’inférence (Section 4.5), et $\mathbf{T}_o^e$ la transformation rigide constante entre le TCP et l’outil, illustrée à la Figure 4.6 et consignée dans les fichiers de trajectoire (Section 4.4.2). Cette transformation étant connue et invariante, la conversion est exacte et ne coûte qu’un produit matriciel par pose ; pour la tâche de saisie et dépôt, l’outil est le TCP lui-même ($\mathbf{T}_o^e = \mathbf{I}$). Les poses ainsi obtenues constituent les poses désirées du TCP, notées $\mathbf{T}_{e,k}^{b\star}$ dans la sous-section suivante.

Tableau 4.1 Hyperparamètres d’entraînement du modèle de Flow Matching.

Hyperparamètre	Valeur
Horizon de prédiction H / horizon d’observation	16 / 2
Dimension des actions et de la proprioception	10
Points du nuage conditionnel	1024
Taille de lot	128
Nombre d’époques	500
Optimiseur	AdamW
Pas d’apprentissage initial	1×10^{-4}
Régularisation (<i>weight decay</i>)	1×10^{-6}
Ordonnancement du pas d’apprentissage	cosinus, 500 pas d’échauffement
Écrêtage du gradient (norme)	1,0
Taux de décroissance EMA (maximal)	0,9999
Pas d’intégration à l’inférence N	10

4.6.2 Cinématique inverse pour obtenir la trajectoire articulaire nominale

La trajectoire générée par *Flow Matching* décrit le mouvement du lien contrôlé dans l’espace cartésien : sur l’horizon H , le modèle produit une séquence de poses que la conversion outil–TCP de la sous-section précédente ramène en poses désirées du TCP, $\mathbf{T}_{e,k}^{b\star} \in SE(3)$, $k = 1, \dots, H$. Le robot étant commandé dans l’espace articulaire, chaque pose doit être convertie en une configuration $\mathbf{q}_k \in \mathbb{R}^7$ par cinématique inverse. Seules les sept articulations du bras interviennent dans la tâche : les deux articulations de la pince complètent le vecteur de configuration du modèle mais n’influent pas sur la pose du TCP, et la préhension est commandée séparément. Le manipulateur disposant de sept articulations pour une tâche de pose à six degrés de liberté, le problème est redondant et son issue doit être fixée par un critère explicite.

La résolution s’effectue, pour chaque point de passage, par une descente de Gauss–Newton amortie sur l’erreur de pose, implémentée en C++ avec la bibliothèque *Pinocchio*. À l’itération courante, l’écart entre la pose atteinte $\mathbf{T}_e^b(\mathbf{q})$ et la pose désirée $\mathbf{T}_{e,k}^{b\star}$ est mesuré dans l’algèbre de Lie de $SE(3)$, via le logarithme du groupe,

$$\mathbf{e}(\mathbf{q}) = \log\left((\mathbf{T}_{e,k}^{b\star})^{-1} \mathbf{T}_e^b(\mathbf{q})\right) \in \mathbb{R}^6.$$

La Jacobienne géométrique du repère TCP, $\mathbf{J}(\mathbf{q}) \in \mathbb{R}^{6 \times 7}$, donne le torseur cinématique (vitesses linéaire et angulaire) du TCP produit par une vitesse articulaire. Au premier ordre, elle relie donc un incrément $\Delta\mathbf{q}$ à la variation de l’erreur de pose qu’il induit, $\mathbf{e}(\mathbf{q} + \Delta\mathbf{q}) \approx$

$\mathbf{e}(\mathbf{q}) + \mathbf{J}(\mathbf{q}) \Delta \mathbf{q}$. L'incrément articulaire combine alors une tâche principale, celle de réduire $\mathbf{e}$, ainsi qu'une tâche secondaire confinée au noyau de $\mathbf{J}$, c'est-à-dire aux mouvements articulaires qui laissent la pose du TCP inchangée :

$$\Delta \mathbf{q} = \underbrace{-\mathbf{J}^+ \mathbf{e}}_{\text{suivi de pose}} + \underbrace{(\mathbf{I} - \mathbf{J}^+ \mathbf{J}) \left(-k_0(\mathbf{q} - \mathbf{q}_{\text{ref}}) \right)}_{\text{résolution de la redondance}}, \quad \mathbf{J}^+ = \mathbf{J}^\top (\mathbf{J} \mathbf{J}^\top + \lambda \mathbf{I})^{-1}.$$

Le premier terme, $-\mathbf{J}^+ \mathbf{e}$, est la pseudo-inverse amortie de $\mathbf{J}$ [109, 110] appliquée à l'erreur : l'amortissement λ régularise le voisinage des singularités, où la pseudo-inverse non amortie produirait des pas divergents. Le second terme rapproche la configuration d'une posture de référence $\mathbf{q}_{\text{ref}}$ au moyen de l'objectif $-k_0(\mathbf{q} - \mathbf{q}_{\text{ref}})$, que le projecteur $\mathbf{I} - \mathbf{J}^+ \mathbf{J}$ restreint au noyau de $\mathbf{J}$ [111] : la résolution de la redondance n'altère ainsi pas le suivi de la pose, au premier ordre et hors du voisinage immédiat des singularités. L'incrément résultant est borné en norme ($\|\Delta \mathbf{q}\| \leq \delta_{\text{max}} = 0,2 \text{ rad}$), puis intégré sur la variété des configurations, $\mathbf{q} \leftarrow \mathbf{q} \oplus \Delta \mathbf{q}$, jusqu'à ce que $\|\mathbf{e}\| < \varepsilon = 10^{-5}$ ou que 50 itérations soient atteintes.

La référence $\mathbf{q}_{\text{ref}}$ est prise égale à la configuration d'amorçage (*warm-start*), elle-même propagée le long de l'horizon : la première pose est résolue à partir de la configuration mesurée du robot, et chaque pose suivante à partir de la solution de la précédente. Ce chaînage assure simultanément une résolution cohérente de la redondance en assurant que les $\mathbf{q}_k$ restent voisins et la continuité de la trajectoire articulaire. Un déroulement final ramène les écarts successifs dans $[-\pi, \pi]$ pour éliminer tout saut de 2π . La robustesse numérique repose sur une projection de la rotation cible sur $SO(3)$ et sur le rejet des configurations dégénérées (déterminant ou trace aberrants, échec du logarithme), auquel cas la configuration d'amorçage est conservée.

Le solveur produit ainsi la trajectoire articulaire nominale $\mathbf{q}_{1:H}$, qui est la seule grandeur retenue en aval : elle est publiée, puis ré-échelonnée temporellement (Section 4.6.3) avant d'alimenter le filtre de sécurité.

4.6.3 Réajustement temporel de la trajectoire articulaire

La cinématique inverse fournit une suite de configurations articulaires $\mathbf{q}_1, \dots, \mathbf{q}_H$, mais sans horodatage : le modèle de *Flow Matching* produit un nombre fixe de points de passage dont l'espacement dans l'espace articulaire n'est pas uniforme et varie aussi bien à l'intérieur d'un plan que d'un plan à l'autre. Leur affecter une cadence constante, par exemple un point toutes les 0,1 s, ferait alors varier fortement la vitesse articulaire réelle : élevée entre deux configurations éloignées, faible entre deux configurations voisines, d'où un profil de vitesse irrégulier et saccadé. Le réajustement attribue au contraire à chaque segment une durée

proportionnelle à sa longueur articulaire, de sorte que la vitesse reste approximativement constante d'un segment à l'autre.

La longueur d'un segment est mesurée par une norme articulaire pondérée,

$$\ell_i = \left\| \mathbf{W} (\mathbf{q}_{i+1} - \mathbf{q}_i) \right\|, \quad \mathbf{W} = \text{diag}(\mathbf{w}),$$

où les poids $\mathbf{w}$ autorisent à pénaliser différemment les articulations (tous égaux à l'unité dans la configuration retenue, soit la norme euclidienne articulaire). Pour une vitesse articulaire cible v^* (fixée à 0,3 rad/s), la durée de chaque segment et les instants de passage valent

$$\Delta t_i = \frac{\max(\ell_i, \ell_{\min})}{v^*}, \quad t_1 = 0, \quad t_{i+1} = t_i + \Delta t_i,$$

chaque segment étant alors parcouru à la vitesse articulaire constante v^* . Le plancher $\ell_{\min} = f_{\min} L / (H - 1)$, fraction $f_{\min} = 0,02$ de la longueur totale $L = \sum_i \ell_i$ rapportée au nombre de segments, empêche un segment quasi stationnaire de se voir attribuer une durée nulle, qui ferait diverger la vitesse définie ci-dessous.

La trajectoire ré-échelonnée $(\mathbf{q}_i, t_i)$ est publiée sous forme de positions horodatées, sans vitesse explicite. L'exécuteur de trajectoire l'interpole à la fréquence de commande et en publie la vitesse feedforward par différenciation finie :

$$\dot{\mathbf{q}}_{\text{ff}}(t) = \frac{\mathbf{q}_{i+1} - \mathbf{q}_i}{t_{i+1} - t_i}, \quad t \in [t_i, t_{i+1}]. \quad (4.14)$$

Cette vitesse ne provient donc pas d'un second calcul de cinématique inverse, mais de la simple différenciation des positions articulaires aux instants imposés par le réajustement. Par construction de ces instants, sa norme est approximativement constante et proche de v^* malgré des points de passage non équidistants. Elle représente l'intention instantanée de la trajectoire générée par *Flow Matching* et fournit au filtre de sécurité une référence régulière, plutôt qu'une différenciation bruitée d'une consigne de position échantillonnée à cadence fixe.

L'exécuteur n'avance pas le long de cette trajectoire au rythme de l'horloge murale, mais d'une *horloge de progression* : l'abscisse temporelle courante n'est incrémentée que si le robot suit effectivement la consigne, et son avancement est suspendu dans deux cas. Le premier est un retard de poursuite excessif : si l'écart articulaire entre la consigne interpolée et la configuration mesurée dépasse une limite (0,25 rad), la consigne attend le robot au lieu de creuser l'erreur. Le second est signalé par le filtre de sécurité lui-même : tant que la barrière publiée est en violation ($h < 0$), la progression est gelée. Sans ce gel, la consigne nominale continuerait de défilier pendant qu'une déviation de sécurité retient le robot : l'erreur de suivi

s'accumulerait, le terme de retour saturerait contre la contrainte et, à la libération, le robot rejoindrait la trajectoire loin en aval, court-circuitant le segment que le filtre venait de lui faire éviter. Avec lui, la trajectoire planifiée reprend exactement là où elle a été interrompue.

4.7 Filtre de sécurité réactif SDF-CBF

Le modèle génératif est entraîné sur des démonstrations sans obstacle et exécuté en boucle ouverte : il n'offre aucune garantie vis-à-vis d'un environnement non vu à l'entraînement. Plutôt que de contraindre le processus de génération (au prix d'une dérive distributionnelle et d'un coût de calcul accru, voir le Chapitre 2), la sécurité est déléguée à un filtre réactif agissant sur la vitesse articulaire à la fréquence de commande. Le robot étant protégé sur l'ensemble de ses liens, chaque lien l menacé par un obstacle impose sa propre condition de sécurité ; à chaque cycle de commande, le filtre retient la vitesse la plus proche de l'intention du modèle qui les respecte toutes :

$$\dot{\mathbf{q}}^* = \arg \min_{\dot{\mathbf{q}} \in \mathbb{R}^7} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}}\|_{\mathbf{W}}^2 \quad \text{s.c.} \quad \nabla h_l(\mathbf{q})^\top \dot{\mathbf{q}} \geq b(h_l(\mathbf{q})), \quad \forall l \in \mathcal{A}. \quad (4.15)$$

où $\mathbf{q} \in \mathbb{R}^7$ est la configuration articulaire mesurée, $\dot{\mathbf{q}}_{\text{base}}$ la vitesse nominale (l'intention du modèle génératif), $\mathcal{A}$ l'ensemble des liens dont la contrainte est active au cycle courant et $\|\cdot\|_{\mathbf{W}}$ une norme articulaire pondérée par une métrique $\mathbf{W} \succ 0$. Chaque contrainte délimite un demi-espace de vitesses admissibles : la commande retenue est la projection, au sens de $\mathbf{W}$, de $\dot{\mathbf{q}}_{\text{base}}$ sur leur intersection. La métrique ne change pas l'ensemble admissible, elle ne modifie donc pas la sécurité obtenue. Elle influence la répartition articulaire de la correction, c'est-à-dire de ce que le filtre sacrifie lorsqu'il doit dévier de l'intention du modèle. Ce problème comporte trois ingrédients, qui structurent la section. Les fonctions barrière $h_l(\mathbf{q})$, qui mesurent la distance signée entre chaque lien et les points obstacles $\mathbf{p}_i \in \mathbb{R}^3$ de la carte persistante, reposent sur un modèle de distance du robot appris hors ligne (Section 4.7.1), à partir duquel elles sont assemblées, tout comme leur gradient ∇h_l obtenu en ligne (Sections 4.7.2 et 4.7.3). Le second membre $b(h_l)$ et la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$ sont définis à la Section 4.7.4. Enfin, la Section 4.7.5 résout la projection et définit la métrique $\mathbf{W}$, décrit le post-traitement qui transforme cette solution en commande exécutable $\dot{\mathbf{q}}_{\text{final}}$; la Section 4.7.6 traite les minima locaux induits par le filtre, et la Section 4.7.7 délimite la portée des garanties obtenues.

4.7.1 Modèle de distance du robot appris hors ligne

Le filtre doit connaître, en quelques millisecondes et pour des milliers de points simultanément, la distance signée entre tout point de l'espace et la surface du robot ainsi que leur gradient. Suivant l'approche *Robot Distance Fields* [80], également exploitée pour l'évitement réactif de collisions dans un cadre de commande à priorité de tâches [82], le SDF de chaque lien l est approximé par une base tensorielle de polynômes de Bernstein dans un repère normalisé propre au lien. La géométrie maillée du lien est d'abord centrée et mise à l'échelle :

$$\bar{\mathbf{x}} = \frac{\mathbf{x} - \mathbf{o}_l}{s_l}, \quad \mathbf{o}_l = \text{centre de la boîte englobante}, \quad s_l = \max_{\mathbf{v} \in \mathcal{V}_l} \|\mathbf{v} - \mathbf{o}_l\|, \quad (4.16)$$

où $\mathcal{V}_l$ est l'ensemble des sommets du maillage : le lien est ainsi contenu dans la boule unitaire, donc dans le cube $[-1, 1]^3$ qui sert de domaine d'approximation. Sur ce domaine, le SDF normalisé est une combinaison linéaire de n^3 fonctions de base :

$$\bar{d}_l(\bar{\mathbf{x}}) = \sum_{a,b,c=0}^{n-1} w_{abc}^{(l)} B_a(u_x) B_b(u_y) B_c(u_z) = \Phi(\bar{\mathbf{x}}) \mathbf{w}_l \quad (4.17)$$

$$B_a(u) = \binom{n-1}{a} u^a (1-u)^{n-1-a} \quad (4.18)$$

$$\mathbf{u} = \frac{\bar{\mathbf{x}} + 1}{2} \quad (4.19)$$

et la distance métrique s'en déduit par $d_l = s_l \bar{d}_l$. Les polynômes de Bernstein étant définis sur $[0, 1]$, le changement de variable affine $\mathbf{u} = (\bar{\mathbf{x}} + 1)/2$ y ramène la coordonnée normalisée : paramétrer la base sur $[-1, 1]^3$ ou sur $[0, 1]^3$ est strictement équivalent, et la convention retenue ici est celle de l'implémentation de référence de [80], qui normalise de même chaque lien dans $[-1, 1]^3$ avant d'évaluer la base sur $[0, 1]$. Chacun des neuf liens protégés (liens 2 à 7, la main et les deux doigts de la pince) utilise $n = 12$ fonctions par axe, soit 1728 coefficients par lien. Trois propriétés motivent ce choix de représentation : le modèle est polynomial, donc $\mathcal{C}^\infty$ et de gradient analytique exact ; il est linéaire en ses paramètres $\mathbf{w}_l$, ce qui rend l'ajustement convexe ; et son évaluation se réduit à des produits tensoriels, massivement parallélisables sur GPU.

L'apprentissage des poids procède en deux étapes par lien. La première reproduit le protocole original de [80] : un jeu d'étiquettes de distance signée exacte est généré sur le maillage (points concentrés près de la surface et points uniformes dans le cube normalisé), puis, la sortie du modèle étant linéaire en $\mathbf{w}_l$, les poids sont ajustés par *moindres carrés récursifs* avec régularisation de type *ridge*. Cette étape n'est pas une contribution de ce mémoire ; son protocole d'échantillonnage, ses équations de mise à jour et ses hyperparamètres sont

reproduits à l'Annexe B. La régression de valeur qui en résulte fournit une distance précise, mais ne contraint pas le gradient : près de la surface, sa direction reste bruitée et sa norme s'écarte de l'unité.

Or le filtre de sécurité consomme précisément ce gradient : ∇h fixe la *direction* de la correction de la projection (4.15) et sa *norme* en fixe l'amplitude. Une seconde étape de raffinement géométrique, propre à ce mémoire, est donc appliquée. Un jeu de supervision dédié de 20 000 *rayons normaux vérifiés* est construit : des points de surface sont déplacés le long de la normale de leur facette d'un décalage δ tiré aléatoirement, concentré dans la coquille critique autour de la marge de sécurité d_{safe} . Chaque candidat n'est retenu que s'il est vérifié contre le maillage exact : le point doit être extérieur, sa distance exacte recalculée doit coïncider avec le décalage imposé, et la direction exacte du gradient ne doit pas s'écarter sensiblement de la normale de départ, ce qui élimine les rayons corrompus par l'axe médian ou la courbure. Chaque rayon fournit ainsi un triplet exact (point, distance, direction sortante $\hat{\mathbf{n}}$) ; la répartition des décalages et les tolérances de vérification sont détaillées à l'Annexe B. Les poids sont alors affinés par descente de gradient sur la fonction de coût

$$\begin{aligned} \mathcal{L} = & \mathcal{L}_{\text{près}} + \mathcal{L}_{\text{unif}} + \lambda_{\text{ray}} \mathcal{L}_{\text{ray}} + \lambda_{\text{eik}} \mathbb{E}[(\|\nabla \bar{d}\| - 1)^2] \\ & + \mathbb{E}\left[\omega \left(\lambda_{\text{dir}} (1 - \widehat{\nabla \bar{d}} \cdot \hat{\mathbf{n}}) + \lambda_{\text{vec}} \|\nabla \bar{d} - \hat{\mathbf{n}}\|^2 \right)\right] \end{aligned} \quad (4.20)$$

où les trois premiers termes sont les erreurs quadratiques de valeur (points près de la surface, uniformes et rayons), le terme eikonal impose $\|\nabla \bar{d}\| = 1$ partout, et les deux derniers alignent la direction du gradient sur la direction exacte des rayons, avec une pondération ω qui décroît avec la distance et concentre l'effort près de la surface ; les poids relatifs λ sont donnés à l'Annexe B. Le gradient $\nabla \bar{d}$ étant la dérivée analytique d'un polynôme, aucune approximation numérique n'intervient.

Les modèles finaux de chaque lien enchaînent ainsi les deux étapes, la régression de valeur servant d'initialisation au raffinement géométrique ; les hyperparamètres complets d'optimisation sont rassemblés à l'Annexe B. La précision de distance, la direction du gradient et la propriété eikonale qui en résultent sont caractérisées expérimentalement au Chapitre 5.

4.7.2 Fonction barrière

À l'exécution, chaque point obstacle $\mathbf{p}_i$ est ramené dans le repère normalisé de chaque lien protégé via la cinématique directe, puis évalué par le modèle appris :

$$\mathbf{x}_{l,i}(\mathbf{q}) = \mathbf{T}_l^b(\mathbf{q})^{-1} \mathbf{p}_i, \quad d_{l,i}(\mathbf{q}) = s_l \Phi\left(\frac{\mathbf{x}_{l,i} - \mathbf{o}_l}{s_l}\right) \mathbf{w}_l.$$

Pour les points situés hors du domaine d'approximation $[-1, 1]^3$, la coordonnée normalisée est bornée au cube et la distance euclidienne au point borné est ajoutée — une correction conservatrice qui prolonge le modèle hors de son domaine d'entraînement. Cette formulation fournit une distance différentiable par rapport à $\mathbf{q}$, interrogeable en parallèle sur GPU pour l'ensemble des points obstacles.

Évaluer cette distance sur la totalité de la carte persistante à chaque cycle serait inutilement coûteux : seuls les points les plus proches du robot peuvent activer la contrainte. La carte est donc d'abord élaguée par un préfiltre sphérique autour des liens protégés, puis les K points de plus faible SDF corps-complet sont conservés :

$$\mathcal{P}_{\text{CBF}} = \text{TopK}_K \left(\min_l d_l(\mathbf{q}, \mathbf{p}_i) \right), \quad K = 100.$$

Cette sélection est mise à jour à une fréquence inférieure à celle de la boucle de commande (30 Hz contre 100 Hz). L'obsolescence maximale de l'ensemble actif qui en résulte (≈ 33 ms) est prise en compte dans le budget de la marge de sécurité d_{safe} (Section 4.7.7).

Les distances $d_{l,i}$ des points retenus sont alors agrégées, lien par lien, en une barrière scalaire. Le minimum sur les points obstacles y est approché par un soft-min de température α , formulé de manière numériquement stable autour de $m_l = \min_i d_{l,i}$:

$$h_l(\mathbf{q}) = m_l - \alpha \log \left[\sum_{i=1}^N \exp \left(-\frac{d_{l,i} - m_l}{\alpha} \right) \right] - d_{\text{safe}}.$$

Chaque lien protégé possède ainsi sa propre barrière h_l et son propre ensemble admissible $\{\mathbf{q} \mid h_l(\mathbf{q}) \geq 0\}$; l'ensemble sûr du robot en est l'intersection,

$$\mathcal{C} = \bigcap_l \{\mathbf{q} \mid h_l(\mathbf{q}) \geq 0\}.$$

Réduire cette famille à la seule barrière minimale ne contraindrait que le lien instantanément le plus proche et laisserait les autres liens libres de progresser vers leurs propres obstacles, au prix d'une commutation brutale du lien sélectionné dès qu'un second lien devient critique. Le filtre conserve donc les K_a liens de plus faible h_l et traite chacun comme une contrainte distincte ; leur assemblage et leur résolution conjointe sont détaillés à la Section 4.7.5. Le soft-min introduit un biais conservateur borné par $\alpha \log N$; en notant h_{soft} la valeur brute issue du soft-min, la valeur compensée $h_{\text{corr}} = h_{\text{soft}} + \alpha \log N$ est utilisée pour l'interprétation des mesures — c'est sous ces deux noms que la barrière apparaît au Chapitre 5. Ce terme étant indépendant de $\mathbf{q}$, le gradient n'est pas affecté.

Cette compensation corrige toutefois l'interprétation, pas la décision : dans le problème (4.15), c'est la valeur de h_l qui fixe la bande d'activation et le second membre $b(h_l)$. Or le biais effectif vaut $\alpha \log M_{\text{eff}}$, où M_{eff} est le nombre de points dont la distance se loge à moins de α du minimum. Face à un nuage d'obstacle dense (un obstacle synthétique de 1000 points, ou une surface plane vue de près), de nombreux points se massent près du point le plus proche et ce biais atteint l'ordre de la marge d_{safe} elle-même : la barrière rapportée passe sous zéro alors que la distance réelle reste positive, et le filtre déclenche à tort son régime de récupération. Les rôles de la valeur et du gradient sont donc dissociés dans la configuration finale du système : la valeur transmise au second membre et aux diagnostics est le minimum exact par lien, $h_l = \min_i d_{l,i} - d_{\text{safe}}$, disponible sans calcul supplémentaire puisque le champ $d_{l,i}$ est de toute façon évalué point par point, tandis que le soft-min est conservé exclusivement pour le gradient, dont il garantit la continuité lors des commutations du point dominant (Section 4.7.3). La barrière décisionnelle est ainsi exempte du biais d'encombrement, sans réintroduire la discontinuité de gradient du minimum exact.

4.7.3 Gradient analytique de la contrainte

La projection (4.15) repose entièrement sur le gradient $\nabla_{\mathbf{q}} h$, qui en fixe à la fois la direction et l'amplitude. On l'obtient en forme close, par dérivation exacte de chaque maillon de la chaîne, ce qui en garantit le coût borné qu'exige le temps réel. Pour un point obstacle donné, la distance résulte de la composition de trois applications,

$$\mathbf{q} \xrightarrow{\text{cinématique}} \mathbf{x}_{l,i} \xrightarrow{\text{normalisation}} \mathbf{u} \xrightarrow{\text{SDF de Bernstein}} \bar{d}_l,$$

toutes trois dérivables analytiquement. Les différentielles correspondantes sont établies successivement, puis recombinaées par la règle de chaîne et agrégées à travers le soft-min. Cette dérivation distingue le présent filtre de l'implémentation de référence de [80], qui obtient le gradient articulaire par différences finies sur la cinématique directe : la forme close en fournit la valeur exacte, sans choix d'un pas de perturbation et à coût indépendant du nombre d'articulations perturbées.

Tout d'abord, en ce qui concerne la différentielle cinématique, l'obstacle est fixe dans le repère de base $\mathcal{F}_b$ ($\dot{\mathbf{p}}_i = \mathbf{0}$), mais paraît mobile dans le repère $\mathcal{F}_l$ que les articulations déplacent. Sa position locale $\mathbf{x}_{l,i} = (\mathbf{R}_l^b)^\top (\mathbf{p}_i - \mathbf{d}_l^b)$ vérifie la relation de fermeture $\mathbf{p}_i = \mathbf{d}_l^b + \mathbf{R}_l^b \mathbf{x}_{l,i}$, où $\mathbf{R}_l^b(\mathbf{q})$ et $\mathbf{d}_l^b(\mathbf{q})$ sont la rotation et la translation de $\mathbf{T}_l^b(\mathbf{q})$. En dérivant cette relation par rapport au temps et en annulant $\dot{\mathbf{p}}_i$:

$$\mathbf{0} = \dot{\mathbf{d}}_l^b + \dot{\mathbf{R}}_l^b \mathbf{x}_{l,i} + \mathbf{R}_l^b \dot{\mathbf{x}}_{l,i}.$$

Les blocs linéaire et angulaire de la Jacobienne géométrique du lien, $\mathbf{J}_{L,l}^b$ et $\mathbf{J}_{A,l}^b$, relient ces dérivées à la vitesse articulaire par $\dot{\mathbf{d}}_l^b = \mathbf{J}_{L,l}^b \dot{\mathbf{q}}$ et $\dot{\mathbf{R}}_l^b = [\mathbf{J}_{A,l}^b \dot{\mathbf{q}}]_{\times} \mathbf{R}_l^b$. En les substituant et en isolant le terme de vitesse locale :

$$\mathbf{R}_l^b \dot{\mathbf{x}}_{l,i} = -\mathbf{J}_{L,l}^b \dot{\mathbf{q}} - [\mathbf{J}_{A,l}^b \dot{\mathbf{q}}]_{\times} \mathbf{R}_l^b \mathbf{x}_{l,i}.$$

Une multiplication à gauche par $(\mathbf{R}_l^b)^{\top}$, suivie des identités du produit vectoriel $\mathbf{R}^{\top}[\mathbf{a}]_{\times} \mathbf{R} = [\mathbf{R}^{\top} \mathbf{a}]_{\times}$ et $[\mathbf{a}]_{\times} \mathbf{b} = -[\mathbf{b}]_{\times} \mathbf{a}$, isole la Jacobienne recherchée $\mathbf{J}_{x_{l,i}} = \partial \mathbf{x}_{l,i} / \partial \mathbf{q}$, telle que $\dot{\mathbf{x}}_{l,i} = \mathbf{J}_{x_{l,i}} \dot{\mathbf{q}}$:

$$\boxed{\mathbf{J}_{x_{l,i}}(\mathbf{q}) = -(\mathbf{R}_l^b)^{\top} \mathbf{J}_{L,l}^b + [\mathbf{x}_{l,i}]_{\times} (\mathbf{R}_l^b)^{\top} \mathbf{J}_{A,l}^b}$$

le premier terme traduisant la translation du repère du lien, le second sa rotation. La matrice antisymétrique associée au produit vectoriel y vaut

$$[\mathbf{x}_{l,i}]_{\times} = \begin{bmatrix} 0 & -x_3 & x_2 \\ x_3 & 0 & -x_1 \\ -x_2 & x_1 & 0 \end{bmatrix}, \quad \mathbf{x}_{l,i} = (x_1, x_2, x_3)^{\top}.$$

La deuxième application, la normalisation qui ramène l'obstacle dans le cube d'apprentissage, $\mathbf{u} = (\mathbf{x}_{l,i} - \mathbf{o}_l) / (2s_l) + \frac{1}{2} \mathbf{1}$, est affine ; sa Jacobienne est la matrice diagonale constante

$$\frac{\partial \mathbf{u}}{\partial \mathbf{x}_{l,i}} = \frac{1}{2s_l} \mathbf{I}_3.$$

La troisième différentielle, le gradient du SDF dans l'espace normalisé, découle de la dérivation exacte de la base tensorielle. La dérivée d'un polynôme de Bernstein de degré $n-1$ se ramène à la base de degré immédiatement inférieur par la règle d'abaissement

$$\frac{dB_a(u)}{du} = (n-1) \left(B_{a-1}^{(n-2)}(u) - B_a^{(n-2)}(u) \right),$$

d'où, en dérivant le produit tensoriel selon chacun des trois axes, le gradient $\nabla_{\mathbf{u}} \bar{d}_l \in \mathbb{R}^3$:

$$\nabla_{\mathbf{u}} \bar{d}_l = \begin{bmatrix} \sum_{a,b,c} w_{abc}^{(l)} B'_a(u_x) B_b(u_y) B_c(u_z) \\ \sum_{a,b,c} w_{abc}^{(l)} B_a(u_x) B'_b(u_y) B_c(u_z) \\ \sum_{a,b,c} w_{abc}^{(l)} B_a(u_x) B_b(u_y) B'_c(u_z) \end{bmatrix}.$$

La composition des trois différentielles, en tenant compte de la mise à l'échelle métrique $d_{l,i} = s_l \bar{d}_l$, fournit le gradient articulaire d'un point obstacle : le facteur s_l se simplifie avec

celui de la normalisation, de sorte que le gradient spatial métrique se réduit à $\nabla_{\mathbf{x}} d_{l,i} = \frac{1}{2} \nabla_{\mathbf{u}} \bar{d}_l$. Cette expression vaut pour les points dont la coordonnée normalisée reste dans le cube $[-1, 1]^3$; pour la correction conservatrice appliquée hors de ce domaine (Section 4.7.2), la composante polynomiale s'annule sur les axes bornés au cube et le gradient s'y réduit à la direction radiale unitaire allant du point borné vers l'obstacle, dérivée exacte de la distance euclidienne ajoutée. Les contraintes effectivement actives portant sur des points proches de la surface du robot, c'est néanmoins la branche intérieure qui gouverne la projection. Plutôt que d'assembler explicitement la Jacobienne $\mathbf{J}_{x_{l,i}}$ pour chacun des points — ce qui matérialiserait un tenseur coûteux —, on applique les mêmes identités du produit vectoriel pour réécrire $\nabla_{\mathbf{q}} d_{l,i} = \mathbf{J}_{x_{l,i}}^\top \nabla_{\mathbf{x}} d_{l,i}$ directement dans le repère de base. En notant $\mathbf{g}_{l,i} = \mathbf{R}_l^b \nabla_{\mathbf{x}} d_{l,i}$ le gradient spatial exprimé dans $\mathcal{F}_b$ et $\mathbf{r}_{l,i} = \mathbf{p}_i - \mathbf{d}_l^b$ le vecteur reliant l'origine du repère du lien à l'obstacle :

$$\nabla_{\mathbf{q}} d_{l,i} = -(\mathbf{J}_{L,l}^b)^\top \mathbf{g}_{l,i} - (\mathbf{J}_{A,l}^b)^\top (\mathbf{r}_{l,i} \times \mathbf{g}_{l,i}).$$

Cette forme, retenue dans l'implémentation, ne requiert que deux produits matrice-vecteur et un produit vectoriel par lien, et évite de construire la Jacobienne point par point.

Il reste à propager ce résultat à travers l'agrégation soft-min, conduite indépendamment pour chaque lien. La dérivation du Log-Sum-Exp fait apparaître les poids du soft-max, ω_i , qui agissent comme un sélecteur lisse du point obstacle dominant :

$$\omega_i = \frac{\exp(-d_{l,i}/\alpha)}{\sum_{j=1}^N \exp(-d_{l,j}/\alpha)} = \text{softmax}_i(-d_{l,i}/\alpha),$$

et le gradient de la barrière d'un lien l est la combinaison convexe correspondante :

$$\nabla_{\mathbf{q}} h_l = \sum_{i=1}^N \omega_i \nabla_{\mathbf{q}} d_{l,i} = \sum_{i=1}^N \omega_i \mathbf{J}_{x_{l,i}}(\mathbf{q})^\top \nabla_{\mathbf{x}} d_{l,i}.$$

Chaque facteur étant la dérivée exacte d'un polynôme ou d'une transformation rigide, ce gradient est obtenu en forme close, sans aucune approximation numérique. Une seule évaluation vectorisée produit conjointement les barrières h_l et les gradients $\nabla_{\mathbf{q}} h_l$ de tous les liens protégés, qui alimentent directement les lignes de la contrainte de sécurité résolue à la Section 4.7.5. Le soft-min sur les points lisse la direction de chaque gradient en pondérant les points obstacles les plus proches, au prix du biais déjà mentionné. Le tableau 4.2 présente la comparaison du temps de calcul en millisecondes de manière analytique du gradient pour L nombre de liens avec le temps de calcul obtenue par différentiation automatique via la fonction `torch.autograd` de la librairie PyTorch.

Tableau 4.2 Temps d'évaluation conjointe des barrières h_l et de la Jacobienne de contrainte complète ($L \times 7$) pour 100 points obstacles, en fonction du nombre L de liens protégés inclus : gradient analytique en forme close contre différentiation automatique (moyenne $\pm$ écart-type en ms, 100 répétitions), avec l'erreur absolue maximale entre les deux Jacobiennes.

Liens L	Analytique [ms]	Diff. automatique [ms]	Erreur max. gradient
1	5.596 ± 0.413	7.180 ± 0.113	5.960×10^{-8}
2	5.549 ± 0.345	8.010 ± 0.389	2.235×10^{-7}
3	5.432 ± 0.149	8.836 ± 0.457	2.373×10^{-7}
4	5.444 ± 0.106	9.789 ± 0.127	2.373×10^{-7}
5	5.538 ± 0.398	10.349 ± 0.321	2.980×10^{-7}
6	7.644 ± 0.156	12.257 ± 0.517	2.980×10^{-7}
7	7.792 ± 0.315	12.899 ± 0.288	2.980×10^{-7}
8	7.698 ± 0.172	13.479 ± 0.233	2.980×10^{-7}

4.7.4 Contrainte de sécurité et vitesse nominale

Le problème (4.15) requiert encore ses deux entrées : le second membre $b(h)$, qui délimite les vitesses admissibles, puis la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$, centre de l'objectif. Pour la première, la condition CBF classique impose $\nabla h^\top \dot{\mathbf{q}} \geq -\kappa h$. Avec une barrière issue de la perception, le terme linéaire κh présente deux inconvénients pratiques : loin de l'obstacle il autorise des vitesses d'approche arbitrairement grandes, et en violation il exige des vitesses de récupération proportionnelles à la pénétration, amplifiant le bruit de h . La contrainte implémentée sature ces deux régimes :

$$\nabla h(\mathbf{q})^\top \dot{\mathbf{q}} \geq b(h), \quad b(h) = m_{\text{CBF}} + \begin{cases} -v_{\text{in}} \min(h/h_{\text{act}}, 1), & h > 0, \\ v_{\text{rec}} \min(-h/d_{\text{rec}}, 1), & h \leq 0, \end{cases}$$

la contrainte n'étant évaluée que dans la bande d'activation $h \leq h_{\text{act}}$. Loin de la frontière, la vitesse d'approche normale est ainsi bornée par v_{in} ; à la frontière ($h = 0$), toute vitesse normale entrante est interdite ; en violation, une vitesse de sortie proactive, croissant avec la pénétration jusqu'à v_{rec} à la profondeur de saturation d_{rec} , est exigée. Cette récupération proactive est complétée par l'étape de réparation décrite à la Section 4.7.5, qui réévalue cette même contrainte saturée sur la commande filtrée et corrige la violation résiduelle que la saturation et le filtrage réintroduisent. Cette formulation appartient à la famille $\nabla h^\top \dot{\mathbf{q}} \geq -\gamma(h)$ avec γ une fonction de classe $\mathcal{K}$ saturée.

Telle quelle, cette contrainte suppose un environnement immobile. La condition d'invariance porte en effet sur la dérivée totale de la barrière, $\dot{h}_l = \nabla h_l^\top \dot{\mathbf{q}} + \partial h_l / \partial t$, dont le second

terme est la contribution propre du mouvement des obstacles; le second membre $b(h_l)$ ne borne que le premier. Un obstacle qui approche consomme donc la bande de freinage à un rythme que la commande ne contrôle pas. Ce terme temporel est estimé en ligne, sans modèle de mouvement, comme le résidu entre la variation mesurée de chaque barrière et la part imputable au déplacement du robot lui-même :

$$\hat{d}_{l,k} = \frac{h_l(t_k) - h_l(t_{k-1})}{\Delta t} - \nabla h_l^\top \frac{\mathbf{q}_k - \mathbf{q}_{k-1}}{\Delta t},$$

puis lissé par un filtre exponentiel de constante τ_{env} en une estimée $\bar{d}_l$. L'estimateur est remis à zéro dès qu'un échantillon est invalide, soit qu'une variation instantanée invraisemblable trahisse une commutation de la sélection des points critiques, soit que le lien se trouve loin de tout obstacle : un resserrement périmé ne doit pas survivre à sa cause. Chaque demi-espace actif est alors resserré du terme

$$\eta_l = \text{clip}\left(-\bar{d}_l - \varepsilon_{\text{env}}, 0, \eta_{\text{max}}\right), \quad \nabla h_l^\top \dot{\mathbf{q}} \geq b(h_l) + \eta_l,$$

qui exige du robot qu'il cède le passage proportionnellement à la vitesse d'approche estimée pendant qu'il se trouve encore à l'extérieur de la marge, au lieu de ne réagir qu'à la décroissance constatée de h_l ; la réévaluation finale de la Section 4.7.5 applique le même second membre resserré. L'estimation est conduite par lien, et non sur le minimum global : un obstacle approchant du coude est détecté même lorsque l'outil, au contact de la cible, maintient le minimum global près de zéro, et le resserrement ne s'applique qu'à la contrainte du lien concerné. Dans une scène immobile, $\bar{d}_l \approx 0$ et la zone morte ε_{env} annule exactement le terme : la contrainte quasi statique originale est retrouvée et le mécanisme est inerte.

La seconde entrée du problème (4.15) est la vitesse que le système *souhaite* exécuter. Celle-ci combine le feedforward retimé $\dot{\mathbf{q}}_{\text{ff}}$ (Section 4.6.3) et une correction de suivi proportionnelle saturée :

$$\dot{\mathbf{q}}_{\text{fb}}^0 = \text{sat}\left(K_p(\mathbf{q}_{\text{nom}} - \mathbf{q}_{\text{cur}}), \dot{q}_{\text{fb,max}}\right).$$

À proximité de la frontière, ce terme de suivi est cependant restreint à sa composante tangentielle d'avancement. En effet, lorsque le filtre dévie le robot de la trajectoire nominale pour des raisons de sécurité, un retour proportionnel complet pointe directement vers la trajectoire — c'est-à-dire contre la correction de sécurité — et les deux termes s'annulent en régime permanent, immobilisant le robot contre la frontière. Avec $\hat{\mathbf{t}} = \dot{\mathbf{q}}_{\text{ff}} / \|\dot{\mathbf{q}}_{\text{ff}}\|$ la direction

d'avancement nominale :

$$\dot{\mathbf{q}}_{\text{fb}} = \begin{cases} \max(0, \hat{\mathbf{t}}^\top \dot{\mathbf{q}}_{\text{fb}}^0) \hat{\mathbf{t}}, & h \leq h_{\text{act}}, \\ \dot{\mathbf{q}}_{\text{fb}}^0, & \text{sinon.} \end{cases}$$

La vitesse nominale brute $\dot{\mathbf{q}}_{\text{raw}} = \dot{\mathbf{q}}_{\text{ff}} + \dot{\mathbf{q}}_{\text{fb}}$ est ensuite lissée par un filtre passe-bas de constante τ_{nom} :

$$\gamma_{\text{nom}} = \frac{\Delta t}{\Delta t + \tau_{\text{nom}}}, \quad \dot{\mathbf{q}}_{\text{base},k} = (1 - \gamma_{\text{nom}}) \dot{\mathbf{q}}_{\text{base},k-1} + \gamma_{\text{nom}} \dot{\mathbf{q}}_{\text{raw},k},$$

puis saturée aux limites de vitesse articulaire et annulée sous un seuil de maintien (évitant toute dérive en station). Le résultat $\dot{\mathbf{q}}_{\text{base}}$ est la *vitesse nominale filtrée* : c'est l'entrée du filtre de sécurité, et la référence par rapport à laquelle toutes les interventions de celui-ci sont mesurées au Chapitre 5.

4.7.5 De la vitesse nominale à la commande sécurisée

Les entrées du problème (4.15) sont désormais toutes définies : les barrières et leurs gradients (Sections 4.7.2 et 4.7.3), le second membre resserré et la vitesse nominale filtrée $\dot{\mathbf{q}}_{\text{base}}$ (Section 4.7.4). Il reste à le résoudre, puis à transformer sa solution en commande exécutable.

La projection CBF calcule la vitesse admissible la plus proche de $\dot{\mathbf{q}}_{\text{base}}$ au sens de la métrique $\mathbf{W}$, c'est-à-dire sa projection sur l'intersection des demi-espaces actifs (4.15) :

$$\dot{\mathbf{q}}_{\text{proj}} = \arg \min_{\dot{\mathbf{q}}} \frac{1}{2} \|\dot{\mathbf{q}} - \dot{\mathbf{q}}_{\text{base}}\|_{\mathbf{W}}^2 \quad \text{s.c.} \quad \nabla h_l^\top \dot{\mathbf{q}} \geq b(h_l) + \eta_l, \quad \forall l \in \mathcal{A}.$$

La correction de sécurité $\Delta \dot{\mathbf{q}}_{\text{CBF}} \equiv \dot{\mathbf{q}}_{\text{proj}} - \dot{\mathbf{q}}_{\text{base}}$ n'est pas une entrée du calcul mais la quantité *dérivée* qui mesure l'intervention du filtre. Cette dernière est nulle dès que toutes les contraintes sont satisfaites et de norme égale à l'intensité de la correction.

La brique élémentaire de la résolution est la projection sur un seul demi-espace, qui admet une forme close pour toute métrique $\mathbf{W}$. Pour le lien l :

$$\dot{\mathbf{q}} \leftarrow \dot{\mathbf{q}} + \omega \frac{\max(0, b(h_l) + \eta_l - \nabla h_l^\top \dot{\mathbf{q}})}{\nabla h_l^\top \mathbf{W}^{-1} \nabla h_l + \varepsilon} \mathbf{W}^{-1} \nabla h_l.$$

Si la contrainte du lien est déjà respectée, le terme $\max(0, \cdot)$ s'annule et la vitesse reste inchangée ; sinon, elle est déplacée le long de $\mathbf{W}^{-1} \nabla h_l$ pour ramener le produit $\nabla h_l^\top \dot{\mathbf{q}}$ sur sa borne, le facteur de relaxation ω pondérant ce pas. Quelle que soit la métrique, le demi-espace est donc satisfait identiquement : $\mathbf{W}$ ne modifie que la direction articulaire dans laquelle la

correction est réalisée. Lorsqu'un *seul* lien est actif, une application de cette mise à jour résout exactement (4.15), et $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ se réduit à cette expression close.

Le choix de $\mathbf{W}$ répond à une question que la formulation euclidienne tranche mal : quel mouvement sacrifier quand la sécurité exige de dévier de l'intention du modèle. Avec $\mathbf{W} = \mathbf{I}$, la correction est colinéaire à ∇h_l et se dépose sur les articulations offrant le plus grand bras de levier sur la barrière, sans égard à leur effet sur la tâche : lorsqu'un lien du corps (typiquement le coude) rencontre un obstacle, c'est l'épaule entière qui recule et, avec elle, l'effecteur : la trajectoire exécutée du lien contrôlé s'écarte fortement de la trajectoire nominale alors même que la contrainte ne portait pas sur lui. La métrique de tâche retenue mesure au contraire le coût d'une vitesse articulaire par le déplacement qu'elle inflige au lien contrôlé :

$$\mathbf{W} = \mathbf{J}_p^\top \mathbf{J}_p + w_{\text{rot}} \mathbf{J}_o^\top \mathbf{J}_o + \lambda \mathbf{I}, \quad \|\dot{\mathbf{q}}\|_{\mathbf{W}}^2 = \|\mathbf{J}_p \dot{\mathbf{q}}\|^2 + w_{\text{rot}} \|\mathbf{J}_o \dot{\mathbf{q}}\|^2 + \lambda \|\dot{\mathbf{q}}\|^2,$$

où $\mathbf{J}_p$ et $\mathbf{J}_o$ sont les blocs linéaire et angulaire de la Jacobienne géométrique du lien contrôlé. Parmi toutes les vitesses satisfaisant le même demi-espace, la projection retient alors celle qui déplace le moins le lien contrôlé : la correction se résout, autant que la cinématique le permet, dans le noyau de la Jacobienne de tâche, le même levier de redondance que la cinématique inverse exploite pour sa tâche secondaire. Le poids $w_{\text{rot}} = 0,01$ règle l'arbitrage position–orientation (un radian de rotation du repère de tâche y coûte autant que dix centimètres de translation) et la régularisation $\lambda = 0,01$ maintient $\mathbf{W}$ inversible en attribuant un coût résiduel aux mouvements du noyau, faible devant le coût de tâche. La métrique est un préconditionneur lentement variable : elle est rafraîchie à 30 Hz dans le fil de perception, par différences finies sur la cinématique directe, et la boucle de commande à 100 Hz la lit sans synchronisation, l'obsolescence maximale (≈ 33 ms, soit moins de 0,025 rad de mouvement articulaire) étant sans effet mesurable sur la projection.

Avec plusieurs liens actifs, l'intersection des demi-espaces n'admet plus de forme fermée. Projeter sur le demi-espace d'un lien pouvant re-violer celui d'un lien déjà traité, on balaie $\mathcal{A}$ à plusieurs reprises par la méthode de Gauss–Seidel, aussi connue sous le nom de *projections sur ensembles convexes*. Les gradients étant figés à la configuration courante, la suite des projections converge vers une vitesse qui satisfait toutes les contraintes simultanément. Comme un lien déjà satisfait laisse $\dot{\mathbf{q}}$ inchangé, compléter $\mathcal{A}$ à un effectif fixe est sans effet sur la solution : la sélection $\text{top-}K_a$ et la boucle de N_{it} balayages ont une forme constante, capturée une seule fois dans un graphe CUDA puis exécutée intégralement sur le GPU à chaque cycle de commande, sans masquage dynamique ni synchronisation GPU–CPU. C'est cette structure à forme fixe qui rend abordable, à la fréquence de commande, la contrainte conjointe de tous les liens protégés du bras et de l'effecteur.

La vitesse projetée est ensuite saturée aux limites articulaires :

$$\dot{\mathbf{q}}_{\text{proj,sat}} = \text{sat}(\dot{\mathbf{q}}_{\text{proj}}).$$

Cette vitesse saturée est filtrée par un second filtre passe-bas exponentiel de constante de temps τ_{safe} :

$$\dot{\mathbf{q}}_{f,k} = (1 - \gamma_{\text{safe}})\dot{\mathbf{q}}_{f,k-1} + \gamma_{\text{safe}}\dot{\mathbf{q}}_{\text{proj,sat},k},$$

avec

$$\gamma_{\text{safe}} = \frac{\Delta t}{\Delta t + \tau_{\text{safe}}}.$$

L'ordre saturation puis filtrage évite que le filtre considère une vitesse physiquement irréalisable.

La saturation et le filtrage peuvent toutefois réintroduire une légère violation de la contrainte CBF. La contrainte est donc réévaluée sur la vitesse filtrée, avec le même second membre saturé et resserré $b(h) + \eta$ que la projection principale (Section 4.7.4) :

$$c_f = \nabla h^\top \dot{\mathbf{q}}_f - b(h) - \eta.$$

Si $c_f < 0$ dans la bande d'activation, une réparation finale est appliquée :

$$\Delta \dot{\mathbf{q}}_{\text{rep}} = \frac{\max(0, -c_f)}{\nabla h^\top \mathbf{W}^{-1} \nabla h + \varepsilon} \mathbf{W}^{-1} \nabla h.$$

Cette réparation est une projection corrective sur la même contrainte CBF linéarisée : elle réutilise la valeur de barrière h , le gradient ∇h , la borne $b(h) + \eta$ et la métrique $\mathbf{W}$ de la projection principale, de sorte qu'elle répartit sa correction de la même manière que le solveur au lieu de défaire, par un pas euclidien, la préférence pour le noyau de tâche. La commande finale est ainsi certifiée contre exactement la même condition de sécurité que la solution du solveur.

Deux garde-fous bornent cette réparation. D'une part, sa norme est plafonnée : à Δv_{max} tant que la barrière est positive, et à un plafond distinct $\Delta v_{\text{max}}^{\text{rec}}$, plus permissif, lorsque $h < 0$ et qu'une sortie rapide de la zone de violation devient prioritaire. Ce plafond couvre notamment le scénario où la carte persistante fait apparaître de nouveaux points obstacles déjà très proches du robot, voire déjà dans la zone de violation : la violation instantanée $-c_f$ peut alors être grande, et une réparation non bornée se traduirait par une commande de vitesse violente en un seul cycle. D'autre part, la réparation est modulée par un seuil de fiabilité du gradient : son amplitude croît linéairement de zéro, lorsque $\|\nabla h\| = g_{\text{min}}$, jusqu'à sa pleine

valeur à $3g_{\min}$. La direction $\nabla h / \|\nabla h\|$ n'est en effet fiable que si la norme du gradient est significative : face à un obstacle mince, les gradients des points situés de part et d'autre du lien se compensent dans l'agrégation soft-min et $\|\nabla h\|$ s'effondre ; une réparation de pleine amplitude pousserait alors le bras dans une direction dépourvue de sens géométrique. Le filtre s'en remet dans ce cas à la vitesse issue de la projection pour le cycle courant, la violation résiduelle étant résorbée aux cycles suivants, dès que la géométrie redevient non dégénérée. La même limite $\Delta v_{\max}$ s'applique enfin à chaque pas de projection de la boucle de balayage elle-même : lorsque cette borne est atteinte, la solution retournée n'est plus la projection exacte de (4.15), mais une solution dont la correction par cycle est bornée, le résidu étant repris par la réévaluation finale ci-dessus puis par les cycles suivants.

La vitesse finale envoyée à la commande est donc

$$\dot{\mathbf{q}}_{\text{final}} = \text{sat}(\dot{\mathbf{q}}_f + \Delta \dot{\mathbf{q}}_{\text{rep}}).$$

Le contrôleur bas niveau du robot étant un contrôleur de *vitesse* articulaire, cette vitesse certifiée constitue directement la commande envoyée à l'actionneur :

$$\dot{\mathbf{q}}_{\text{cmd},k} = \dot{\mathbf{q}}_{\text{final},k},$$

sans étape d'intégration en une consigne de position. La commande de sécurité reste ainsi exactement la vitesse que la contrainte CBF a certifiée au cycle courant : aucune dynamique d'intégration ne s'interpose entre la projection et l'actionneur, et la garantie en temps discret se réduit au seul maintien de cette vitesse pendant une période de commande (bloqueur d'ordre zéro), déjà comptabilisé dans la marge d_{safe} (Section 4.7.7). Une référence de position intégrée $\mathbf{q}_{\text{ref},k} = \mathbf{q}_{\text{ref},k-1} + \dot{\mathbf{q}}_{\text{final},k} \Delta t$ est néanmoins maintenue en parallèle, à des fins de supervision et de visualisation, mais elle n'est pas l'entrée du contrôleur. Lorsque la contrainte est inactive et que la projection ne modifie pas la vitesse nominale, $\dot{\mathbf{q}}_{\text{final}}$ se réduit à la vitesse nominale lissée, transmise telle quelle.

4.7.6 Évitement des minima locaux du filtre

La projection est minimale par construction : elle annule exactement la composante de vitesse que la borne interdit, et ne recherche jamais davantage de dégagement que la contrainte n'en exige. Cependant, le modèle génératif étant aveugle aux obstacles, une trajectoire nominale générée alors que le robot est pris dans un minimum local réémet le même ordre à chaque replanification. L'équilibre entre cette intention et la correction qui l'annule immobilise alors le robot sur la frontière $h \approx 0$: la sécurité est préservée, mais la tâche ne progresse plus. Deux

situations de blocage sont distinguées, selon que la contrainte active porte sur un lien du corps ou sur le lien contrôlé lui-même. Elles sont traitées par deux termes complémentaires, tous deux ajoutés à la vitesse nominale en amont de la projection : chaque demi-espace actif les filtre au même titre que l'intention du modèle, de sorte qu'aucun des deux ne peut dégrader la sécurité.

Le premier terme, le dégagement postural, traite les contraintes portées par le corps du bras. Il exploite la même redondance que la métrique de tâche : puisque le manipulateur dispose d'un noyau de dimension un autour de la Jacobienne 6D du lien contrôlé, la posture peut se déformer sans que la tâche n'en souffre. La vitesse injectée pousse le corps dans la direction du noyau qui accroît la barrière critique,

$$\dot{\mathbf{q}}_{\text{clear}} = k_{\text{clear}} \rho(h); \frac{\mathbf{N} \nabla h}{\|\mathbf{N} \nabla h\|}, \quad \mathbf{N} = \mathbf{I} - \mathbf{J}^+ \mathbf{J},$$

où $\mathbf{N}$ est le projecteur amorti sur le noyau de la Jacobienne complète (position et orientation) du lien contrôlé, réutilisée du calcul de la métrique de tâche, $\rho(h) = \text{clip}\left((h_{\text{ns}} - h)/h_{\text{ns}}, 0, 1\right)$ une rampe de proximité et $a = \min\left(\|\mathbf{N} \nabla h\|/(0,3 \|\nabla h\|), 1\right)$ une rampe d'autorité proportionnelle à la fraction du gradient contenue dans le noyau. Dès que le lien critique entre dans la bande $h < h_{\text{ns}}$, la posture se déforme activement dans la direction qui accroît la barrière, à pose du lien contrôlé inchangée au premier ordre (pivot du coude) : le bras s'écarte de l'obstacle *tout en continuant de suivre* la trajectoire de tâche, au lieu de se garer contre la frontière. La rampe a éteint le terme lorsque le gradient ne possède aucune composante dans le noyau, c'est-à-dire précisément lorsque la contrainte porte sur le lien contrôlé et qu'aucune déformation posturale ne peut la desserrer ; la vitesse de pivot est bornée par le gain k_{clear} .

Le second terme, l'*échappement dirigé*, prend le relais dans ce cas résiduel : l'obstacle barre le chemin du lien contrôlé lui-même, typiquement lors du transport de l'objet saisi face à une paroi. Le gradient de barrière ne pousse alors qu'en recul, le retour de suivi, réduit à sa composante tangentielle près de la frontière (Section 4.7.4), s'annule face à un blocage frontal, et le noyau n'a aucune autorité sur une contrainte portée par la main. L'échappement s'active par hystérésis, afin de ne réagir qu'aux blocages avérés et non aux frôlements : la barrière doit descendre sous h_{trig} *pendant que* la projection intervient effectivement ($\|\dot{\mathbf{q}}_{\text{proj}} - \dot{\mathbf{q}}_{\text{base}}\|$ au-delà d'un seuil) durant plusieurs cycles consécutifs, et le terme se relâche dès que $h > h_{\text{rel}}$. Actif, il ajoute à la vitesse nominale l'amplification de sa propre composante tangentielle (gain k_{tan}), qui relance le glissement le long de la frontière, ainsi qu'un échappement dirigé

le long d'une direction cartésienne $\mathbf{e} \in \mathbb{R}^3$, $\|\mathbf{e}\| = 1$, élue en ligne,

$$\dot{\mathbf{q}}_{\text{esc}} = k_{\text{esc}} (1 - a) \frac{(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top) \mathbf{j}_{\mathbf{e}}}{\|(\mathbf{I} - \hat{\mathbf{n}}\hat{\mathbf{n}}^\top) \mathbf{j}_{\mathbf{e}}\|}, \quad \mathbf{j}_{\mathbf{e}} = \frac{\partial (\mathbf{e}^\top \mathbf{p}_{\text{TCP}})}{\partial \mathbf{q}} = \mathbf{J}_p^\top \mathbf{e}, \quad \hat{\mathbf{n}} = \frac{\nabla h}{\|\nabla h\|},$$

où $\mathbf{j}_{\mathbf{e}}$ est la dérivée articulaire de l'avancée du lien contrôlé le long de $\mathbf{e}$, obtenue de la Jacobienne de position $\mathbf{J}_p$ déjà différenciée pour la métrique de tâche. La direction est projetée sur le plan tangent à la barrière, de sorte que $\nabla h^\top \dot{\mathbf{q}}_{\text{esc}} = 0$: contourner l'obstacle ne coûte rien à la contrainte critique, et la projection filtre de toute façon ce terme contre les demi-espaces des autres liens. Son autorité $(1 - a)$ croît exactement là où celle du dégagement postural s'effondre : les deux mécanismes se complètent sans se disputer la même situation.

La direction $\mathbf{e}$ est élue par un *sondage de dégagement échantillonné*, exécuté dans le fil de prétraitement à ~ 30 Hz aux côtés de la métrique de tâche : le cycle de commande à 100 Hz n'en consomme que la ligne articulaire $\mathbf{J}_p^\top \mathbf{e}$ précalculée, et le coût du choix de direction est donc nul sur le chemin critique. K_{dir} directions unitaires sont échantillonnées dans le plan tangent à la normale d'obstacle locale, orientée du point le plus proche de la carte vers le lien contrôlé — approximation cartésienne de $\hat{\mathbf{n}}$, suffisante puisque la ligne consommée est de toute façon reprojetée contre le ∇h courant à chaque cycle. Le lien contrôlé y est virtuellement déplacé aux rayons $\delta_{1,2}$ et chaque direction reçoit un score : le dégagement minimal des positions virtuelles vis-à-vis de la carte d'obstacles, plafonné à $\bar{c}$ afin que le champ libre lointain n'écrase pas les autres termes, augmenté d'un alignement avec la vitesse de tâche nominale (poids w_{goal}) et d'une prime d'engagement envers la direction précédemment élue (poids w_{com}), puis pénalisé sous un plancher d'espace de travail z_{min} , la table n'appartenant pas à la carte d'obstacles. La direction gagnante est lissée exponentiellement (facteur $\beta_{\mathbf{e}}$) puis *gelée* pendant toute la manœuvre, l'hystérésis du déclencheur décidant seule de sa fin : sans prime d'engagement ni gel, deux issues de dégagement équivalentes — de part et d'autre d'un pilier, par exemple — entretiennent un cycle limite de bascule où le robot n'emprunte durablement aucune des deux. Le contournement systématique par le haut, seul comportement d'une version antérieure à direction fixe $\mathbf{e} = \hat{\mathbf{z}}$, redevient un cas particulier, élu de lui-même lorsque le sommet de l'obstacle est l'ouverture la plus dégagée.

Le sondage reste toutefois glouton, à horizon d'un pas : il ne voit pas au-delà d'une concavité. Une liste taboue l'en extrait. Pendant l'engagement, le progrès est mesuré comme le déplacement du lien contrôlé *le long de la direction élue*, et non comme l'accroissement de h : un échappement tangentiel réussi glisse précisément à $h \approx h_{\text{trig}}$ tout au long du contournement, la barrière ne remontant qu'une fois l'obstacle dépassé. Si ce progrès reste sous Δ_{min} pendant T_{tabu} — la projection écrête alors l'échappement : le lien est coincé contre une paroi de la ca-

tivité —, la direction courante est pénalisée (poids w_{tabu}) pendant T_{ttl} et le vote est rouvert. Le sondage essaie ainsi successivement les issues restantes, ce qui transforme l’heuristique gloutonne en recherche locale à mémoire. Une petite composante normale sortante n’est ajoutée qu’en violation ($h < 0$), et la norme totale de l’échappement est bornée par v_{esc} . Les termes de déblocage sont évalués à partir de l’état $(h, \nabla h)$ du cycle précédent, périmé d’une seule période de commande : les déclenchements étant temporisés par hystérésis, ce décalage est sans effet sur le comportement et permet de conserver une unique passe de solveur par cycle. Enfin, ces termes sont des heuristiques de déblocage et non des garanties : ils ne modifient ni les barrières ni leurs bornes, la propriété de sécurité de la section suivante reste entière, et leur effet sur la progression de la tâche est évalué au Chapitre 5.

4.7.7 Portée des garanties

Au-delà du solveur lui-même, il reste à délimiter ce que le filtre garantit réellement. L’invariance avant théorique de l’ensemble $\mathcal{C}$ suppose une dynamique en temps continu, un SDF exact, des obstacles correctement observés et immobiles, et un suivi parfait de la commande par le contrôleur bas niveau. L’implémentation s’écarte de ces hypothèses sur quatre points quantifiables : la discrétisation de la commande (100 Hz), l’obsolescence de l’ensemble des points critiques (mise à jour à 30 Hz), l’erreur d’approximation du SDF de Bernstein (caractérisée au Chapitre 5) et le biais conservateur du soft-min ($\alpha \log N$), éliminé de la barrière décisionnelle par la valeur exacte de la Section 4.7.2. S’y ajoute le cas dégénéré du gradient effondré face aux obstacles minces, où la réparation finale est volontairement atténuée (seuil g_{min} , Section 4.7.5) et la résorption de la violation reportée aux cycles suivants. Quant à l’hypothèse d’immobilité, le resserrement temporel η_l (Section 4.7.4) la tempère sans la lever : il fait céder le robot proportionnellement à la vitesse d’approche estimée de l’environnement, mais repose sur une estimation filtrée, retardée de la constante τ_{env} et plafonnée à η_{max} , et ne constitue pas une garantie formelle vis-à-vis d’obstacles mobiles ; la garantie énoncée demeure celle de l’environnement quasi statique, l’apport du terme face à un obstacle lentement mobile étant évalué empiriquement au Chapitre 5. Le système constitue donc un filtrage réactif de sécurité dont la garantie pratique repose sur le dimensionnement de la marge d_{safe} pour absorber la somme de ces termes, validé expérimentalement au Chapitre 5.

La portée de ces garanties est par ailleurs celle de la sécurité, non de la progression : l’invariance de $\mathcal{C}$ vaut pour une géométrie d’obstacle arbitraire, tandis que les termes de déblocage de la Section 4.7.6 demeurent des heuristiques qui n’étendent la vivacité qu’à certaines topologies. Le cas limite est l’obstacle concave dont la cavité fait face à la trajectoire nominale (croissant, parabole, pince). La projection préservant le mouvement tangentiel, l’intention

nominale glisse le long des parois et entonne le lien contrôlé vers le fond de la cavité ; à son voisinage, les normales opposées des parois s'annulent partiellement dans le gradient soft-min (le cas dégénéré évoqué plus haut, ici porté à son pire) et l'intersection des demi-espaces actifs ne laisse subsister que le recul ou le glissement : le robot s'immobilise sur la frontière $h \approx 0$. Le sondage tabou étend la vivacité à une classe plus large de ces géométries : les issues tangentes sont essayées successivement, chacune abandonnée dès que le déplacement qu'elle produit stagne, de sorte qu'une cavité dont l'ouverture est atteignable par une marche tangentielle monotone — bol, croissant, paroi simple, que l'ouverture se trouve au sommet, sur le côté ou en contrebas au-dessus du plancher $z_{\min}$ — finit par être contournée sans qu'aucune direction ne soit privilégiée a priori. La limite résiduelle est double. D'une part, les directions candidates vivent dans le plan tangent à la normale critique : le retrait le long de la normale n'est jamais proposé, précisément parce qu'il affronte de face l'intention nominale que l'échappement cherche à débloquer ; une cavité qui exige de reculer avant de contourner (poche profonde fermée au-dessus) reste hors d'atteinte. D'autre part, le progrès exigé est monotone par direction : un contournement qui demanderait de traverser un rétrécissement en réduisant temporairement le dégagement est écarté par le score avant d'être tenté. Lorsque toutes les issues tangentes ont été frappées de tabou, le système demeure indéfiniment en arrêt sûr : $h \geq 0$ est maintenu, mais le modèle génératif, aveugle aux obstacles, réémet la même intention à chaque replanification et la tâche ne progresse plus. Cette limite est assumée comme frontière du filtre : la recherche locale à mémoire repousse le seuil de blocage sans le supprimer, et la résolution d'un blocage au-delà exige une information globale sur la topologie de la scène — génération de trajectoires consciente des obstacles ou champ de circulation général — qui relève de la couche de planification et non du filtrage réactif.

Tableau 4.3 Paramètres du filtre de sécurité SDF-CBF.

Symbole	Description	Valeur
d_{safe}	marge de sécurité du SDF	15 mm
α	température du soft-min	0,002
K	points critiques sélectionnés	100
h_{act}	bande d'activation de la contrainte	40 mm
v_{in}	vitesse d'approche normale maximale ($\dot{h}$)	0,30 m/s
v_{rec}	vitesse de sortie exigée en violation ($\dot{h}$)	0,30 m/s
d_{rec}	profondeur de saturation en récupération	15 mm
Δv_{max}	correction de projection et de réparation maximale	0,30 rad/s
$\Delta v_{\text{max}}^{\text{rec}}$	réparation maximale en violation ($h < 0$)	0,85 rad/s
g_{min}	seuil de fiabilité du gradient de réparation	0,03
m_{CBF}	marge numérique de la contrainte	0,001 m/s
$K_p, \dot{q}_{\text{fb,max}}$	suivi proportionnel et saturation	5, 0,5 rad/s
$\tau_{\text{nom}}, \tau_{\text{safe}}$	constantes des filtres passe-bas	0,08 s, 0,08 s
Δq_{lead}	avance maximale de la référence	0,05 rad
$K_a, N_{\text{it}}, \omega$	liens contraints, passes POCS, relaxation	9, 6, 1,0
λ, w_{rot}	métrique de tâche : régularisation, poids de rotation	0,01, 0,01
$k_{\text{clear}}, h_{\text{ns}}$	dégagement postural : gain, bande d'activation	0,3 rad/s, 150 mm
$h_{\text{trig}}, h_{\text{rel}}$	échappement : seuils d'activation et de relâche	20 mm, 50 mm
$k_{\text{tan}}, k_{\text{esc}}, v_{\text{esc}}$	échappement : gains tangent, dirigé, norme max.	1,0, 0,2 rad/s, 0,15 rad/s
$K_{\text{dir}}, \delta_{1,2}, \bar{c}$	sondage : directions, rayons, plafond de dégagement	16, 50/100 mm, 0,20 m
$w_{\text{goal}}, w_{\text{com}}, z_{\text{min}}, \beta_{\text{e}}$	sondage : poids but/engagement, plancher, lissage	0,02, 0,02, 30 mm, 0,3
$T_{\text{tabu}}, \Delta_{\text{min}}, w_{\text{tabu}}, T_{\text{ttl}}$	tabou : délai, progrès minimal, pénalité, durée	1,5 s, 30 mm, 0,1, 8 s
$\tau_{\text{env}}, \varepsilon_{\text{env}}, \eta_{\text{max}}$	resserrement temporel : filtre, zone morte, plafond	0,1 s, 0,05 m/s, 0,5 m/s

CHAPITRE 5 RÉSULTATS ET DISCUSSION

Présentation avec CBF et sans CBF

Présentation de la détection avec SAM 3

Listes des figures pour les résultats

- Trajectoire de la pointe de la fourchette dans le nuage persistant map ;
 1. Nuage de points de la scène ;
 2. Trajectoire sans CBF
 3. Trajectoire avec CBF
 4. Cible.
- Évolution de $h(q)$ en fonction du temps ;
 1. h_{corr} ;
 2. ligne zéro.
 3. zones où le CBF est actif (en rouge par exemple).
- Distance obstacle-robot en fonction du temps
- Décomposition des vitesses en fonction du temps
 1. $|dq_{ff}|$;
 2. $|dq_{feedback}|$;
 3. $|dq_{base}|$
 4. $|dq_{cbf_{delta}}|$
 5. $|dq_{escape}|$
 6. $|dq_{safe}|$.
 7. Objectif : Expliquer le comportement du système et montré quand le FLOW Matching dirige le robot seul, quand le CBF intervient, quand l'échappement est nécessaire, si la vitesse corrigée devient trop agressive, etc.
- Effet du filtre passe bas
 1. Vitesse avant CBF brute
 2. Vitesse avant CBF filtrée
- Temps de calcul du pipeline. Faire un boxplot (ou un tableau) sur :
 1. Temps de détection avec SAM3

2. Temps de suivi avec SAM2
 3. Temps de mise à jour persistent map
 4. Temps de génération de trajectoire avec Flow Matching
 5. Temps de calcul de la cinématique inverse
 6. Temps de calcul des points critiques
 7. Temps de calcul de la correction de trajectoire
 8. Temps total du pipeline
- Résistance faces aux occlusions et pertes de détection.
 1. Détection dispo / indispo dans le temps
 2. Images présentant pertes de détection puis redétection
 - NE PAS OUBLIER LA DÉTECTION INITIALE DES FORK TEETH
 - Cas d'échec (comme la figure de Félix).

5.1 Protocol expérimental

5.1.1 Description des scénarios testés

Pick and place

Robot assisted feeding

5.1.2 Métriques utilisées

À compléter.

5.2 validation du modèle géométrique de sécurité

Cette section valide le modèle géométrique au cœur du filtre de sécurité : la SDF du robot approximée par polynômes de Bernstein (section 4.7.1). Trois propriétés sont évaluées par rapport à la vérité terrain calculée sur les maillages du robot : la précision de la distance, la qualité de la direction du gradient et le temps de calcul.

Le protocole est le suivant : pour chacun des six corps protégés, des rayons sont lancés depuis la surface le long de la normale sortante (40 rayons par corps), et la SDF est évaluée en 25 échantillons par rayon, entre 1 mm et 50 mm de la surface. Seuls les rayons *propres* sont conservés, c'est-à-dire ceux dont la distance exacte à l'union des corps coïncide avec l'avancée le long de la normale, ce qui garantit que l'axe des abscisses des figures correspond bien à la

distance réelle à la surface du robot. Les échantillons situés hors du domaine d’entraînement du modèle d’un corps (7,7 % des points, principalement au-delà de 35 mm) sont exclus, car le modèle y retourne une borne géométrique plutôt que la SDF apprise. Au total, 5541 points extérieurs vérifiés sont retenus.

5.2.1 Précision du SDF du robot

La figure 5.2 compare la distance prédite à la distance exacte. L’erreur absolue moyenne est de 1,96 mm avec un biais négligeable ($-0,02$ mm) et une corrélation $R = 0,9869$. La distribution des erreurs signées est centrée et symétrique, avec un 99^e centile de l’erreur absolue de 6,8 mm.

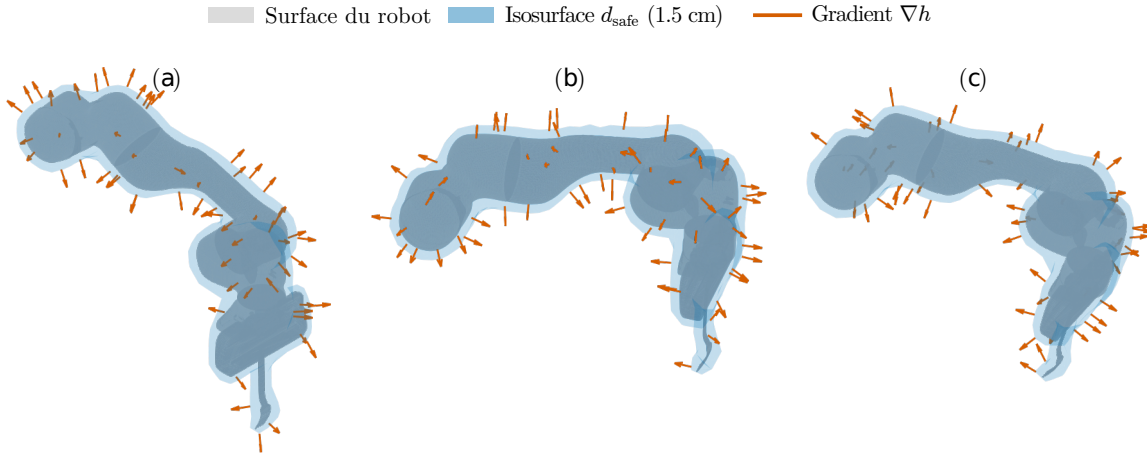


Figure 5.1 Précision de la distance du SDF de Bernstein. (a) Profil de distance le long des normales à la surface du robot ; (b) distribution de l’erreur signée par rapport à la distance exacte calculée sur les maillages.

La figure 5.8(a) présente l’erreur angulaire entre le gradient du modèle et la direction exacte (vecteur vers le point de surface le plus proche). L’erreur médiane globale est de $5,4^\circ$, et reste sous 10° pour la grande majorité des points dès 5 mm de la surface. La figure 5.8(b) montre la norme du gradient : une SDF exacte satisfait la propriété eikonale $\|\nabla h\| = 1$, et le modèle s’en approche au-delà de 10 mm, avec un affaissement vers 0,7 tout près de la surface, où le polynôme lisse le passage par zéro.

Ces résultats justifient la marge de sécurité $d_{\text{safe}} = 15$ mm retenue au chapitre précédent : l’erreur de distance au 99^e centile (6,8 mm) reste largement inférieure à la marge, de sorte qu’une sous-estimation de la distance par le modèle ne suffit pas à amener le robot au contact. De plus, à d_{safe} , la direction du gradient (erreur médiane $\approx 6^\circ$) et sa norme (≈ 1) sont déjà

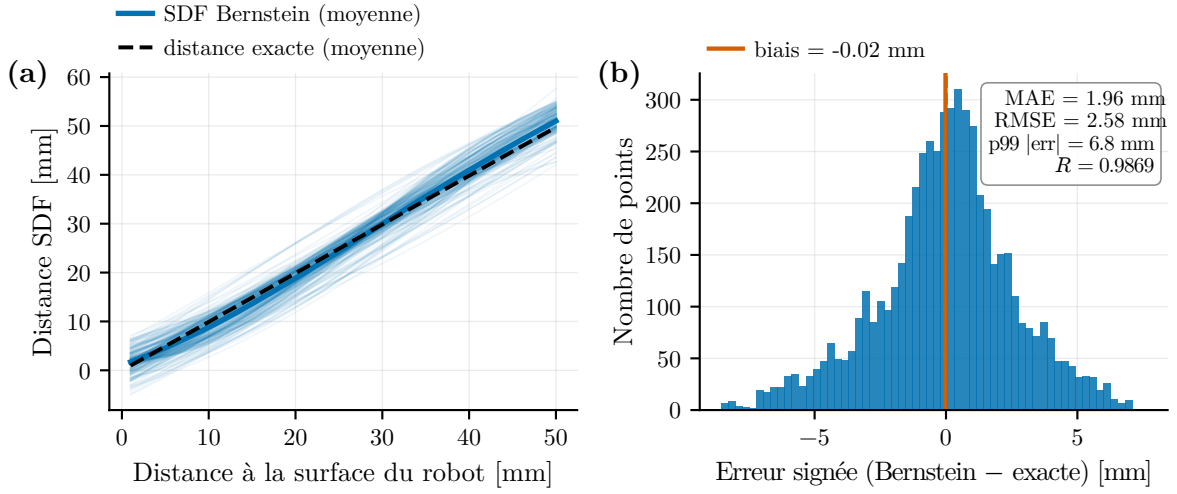


Figure 5.2 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).

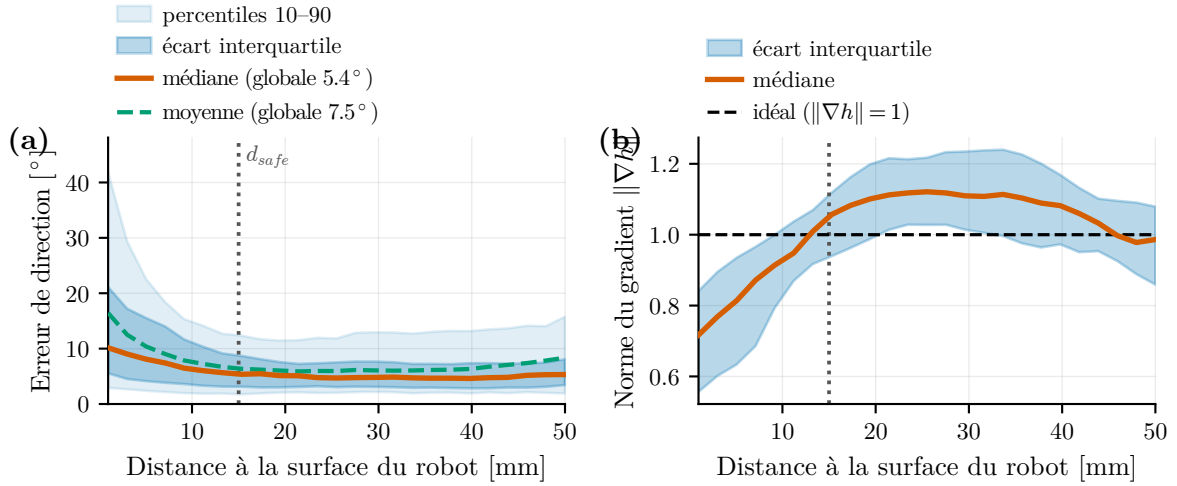


Figure 5.3 Trajectoire multimodale générée par *Flow Matching*.

bien conditionnées, ce qui assure que la contrainte Constraint Barrier Function (CBF) pousse le robot dans la bonne direction au moment où le filtre s'active.

5.2.2 Temps d'évaluation du SDF et du gradient

La figure 5.4 compare le temps d'évaluation conjointe de la distance et du gradient entre le modèle de Bernstein (analytique, sur GPU) et la vérité terrain par requête de proximité sur les maillages (**trimesh**, sur CPU). Le temps du modèle forme un plateau d'environ 5 ms jusqu'à quelques milliers de points : à ces tailles, le coût est dominé par des frais fixes par appel (cinématique directe, $\approx 2,5$ ms, lancement des noyaux GPU, évaluation du gradient et transferts mémoire), le coût par point étant négligeable tant que le GPU n'est pas saturé. La croissance ne devient visible qu'au-delà d'environ 4096 points, soit bien après le point d'opération du filtre ($K = 100$ points critiques). Au point d'opération, l'évaluation reste donc largement en deçà du budget de 13,3 ms imposé par la boucle de commande à 75 Hz, tandis que la requête sur maillage croît linéairement avec le nombre de points et dépasse ce budget d'un à deux ordres de grandeur, ce qui exclut son utilisation en ligne.

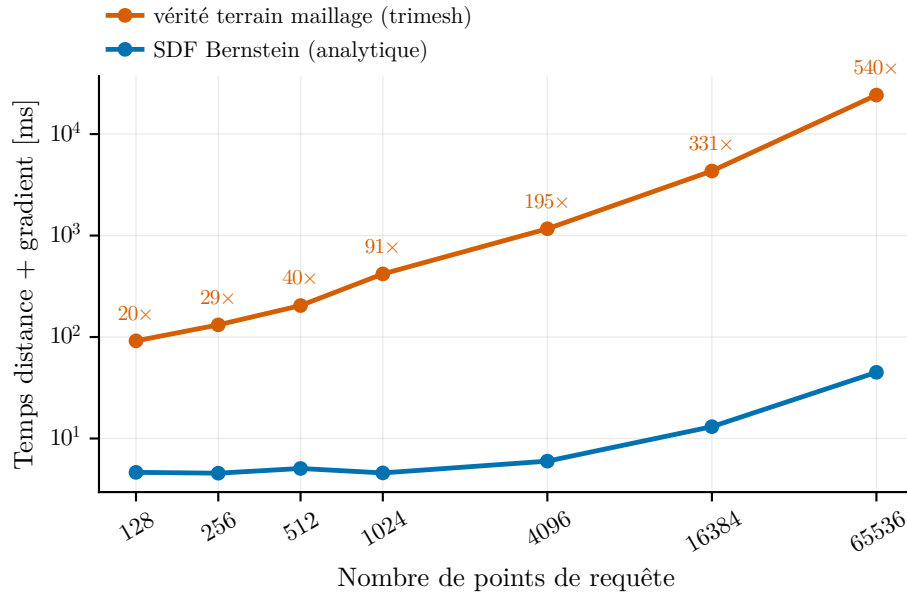


Figure 5.4 Temps de calcul de la distance et du gradient : modèle de Bernstein (GPU) contre vérité terrain sur maillages (CPU), en fonction du nombre de points de requête.

5.2.3 Visualisation des points critiques sélectionnés

Enveloppe de sécurité correspondant à l'isosurface ($d_{\text{robot}} = 1,5$ cm).

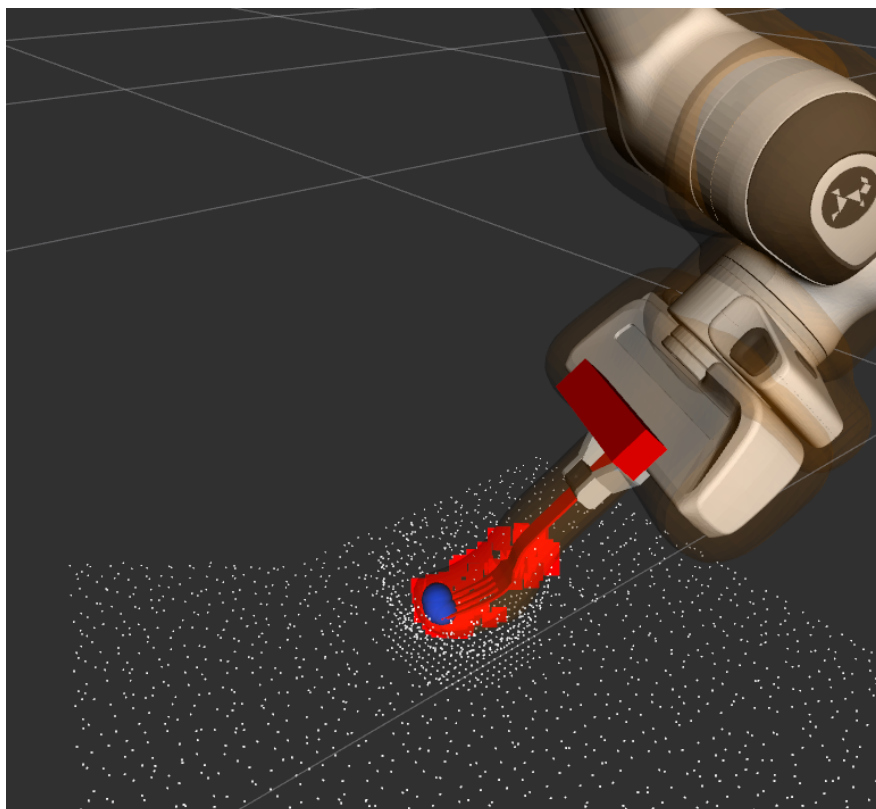


Figure 5.5 Sélection des points les plus proches pour contraindre le CBF

5.3 Qualité de la génération nominale

5.3.1 Trajectoire générée sans obstacle critique

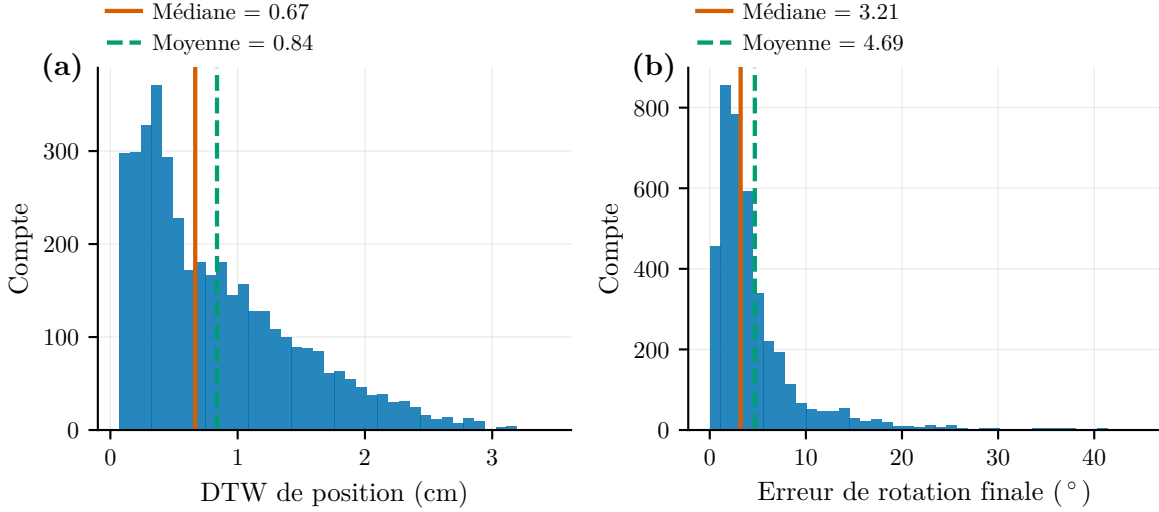


Figure 5.6 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).

5.3.2 Cohérence de la trajectoire avec la cible

À compléter.

À compléter.

À compléter.

À compléter.

5.4 Sécurité et évitement d'obstacle

5.4.1 Comparaison des trajectoires avec et sans SDF-CBF

5.4.2 Évolution de $h(q)$ et de la contrainte de sécurité

La Figure 5.12 présente l'évolution de la barrière h_{soft} et de sa valeur compensée h_{corr} au cours d'un essai représentatif. Les bandes vertes marquent les cycles où la projection corrige la vitesse ($\Delta \dot{\mathbf{q}}_{\text{CBF}} \neq 0$) ; les bandes jaunes, ceux où seule la réparation intervient — la vitesse

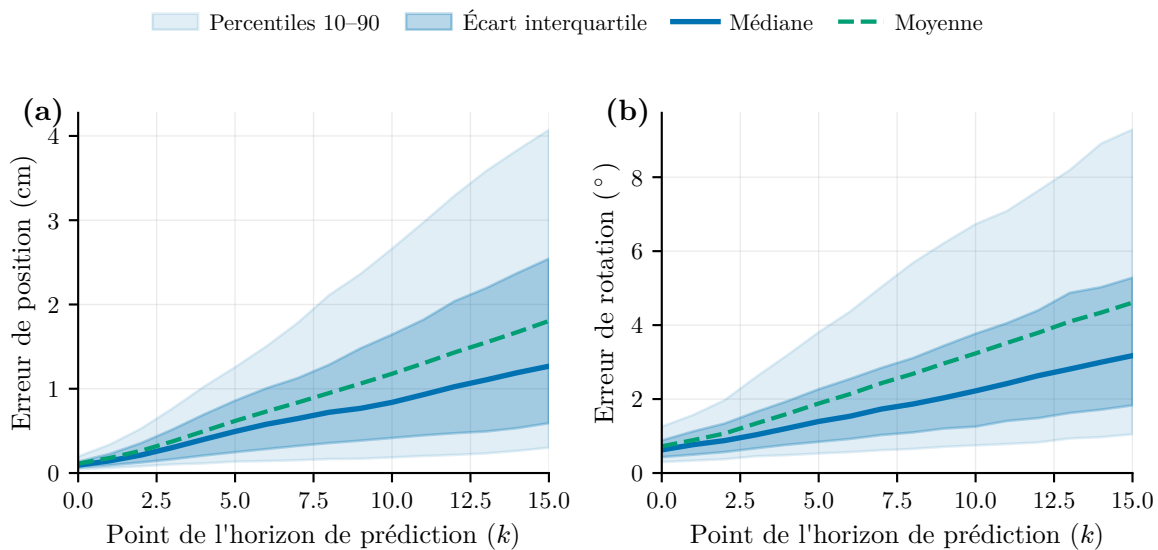


Figure 5.7 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

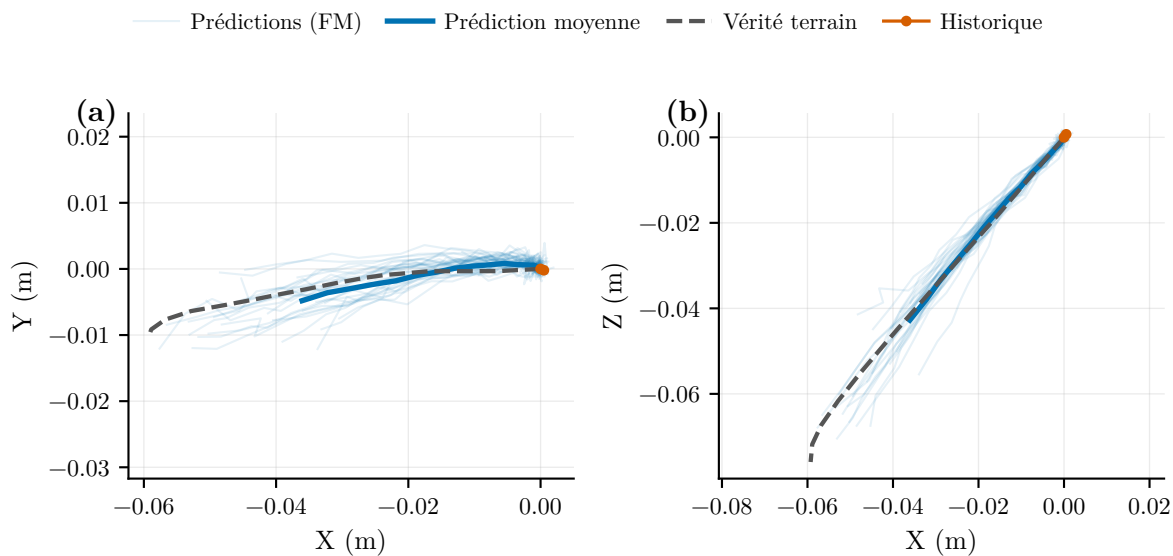


Figure 5.8 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

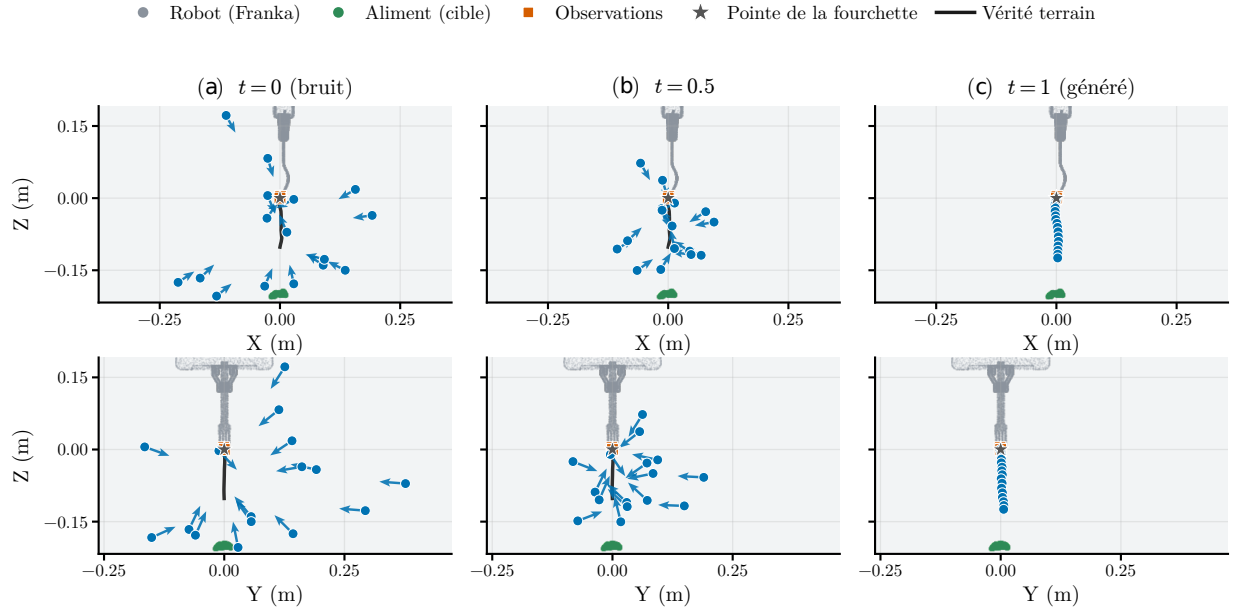


Figure 5.9 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

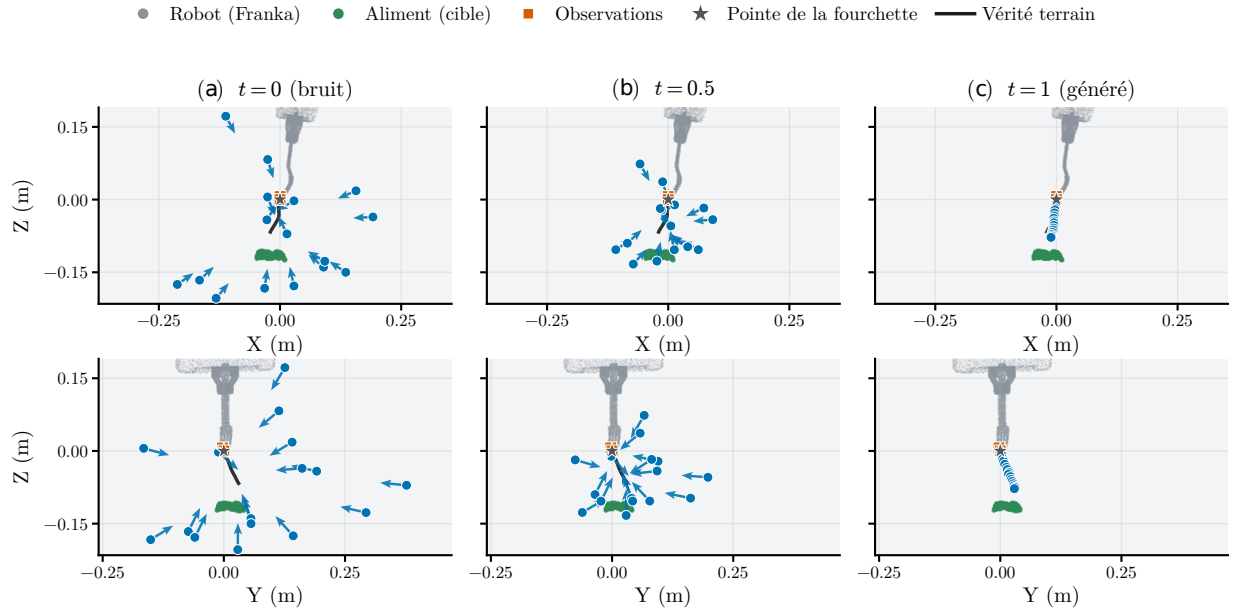


Figure 5.10 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact; (b) norme du gradient (propriété eikonale).

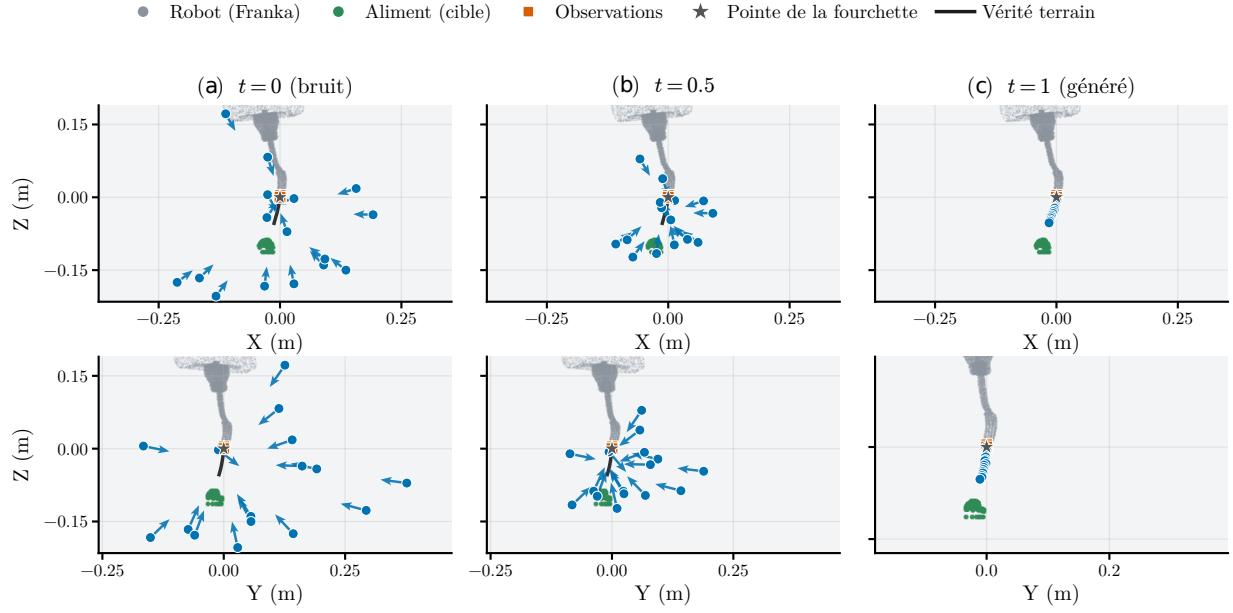


Figure 5.11 Qualité du gradient du SDF en fonction de la distance à la surface du robot. (a) Erreur de direction par rapport au gradient exact ; (b) norme du gradient (propriété eikonale).

instantanée satisfait la contrainte, mais la commande filtrée, qui porte la mémoire des cycles précédents, échoue à la revérification et est corrigée par $\Delta \dot{\mathbf{q}}_{\text{rep}}$. Après la phase d'approche, le système opère en régime de suivi de frontière : h_{corr} se stabilise à quelques millimètres de la frontière pendant que le robot progresse tangentiellement le long des surfaces vers la cible, conformément au comportement attendu de la projection préservant la composante tangentielle (Section 4.7.5).

La Figure 5.13 compare la valeur de la contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après filtrage et réparation). La courbe nominale plonge sous zéro dès que le robot opère à proximité des obstacles : laissée seule, la trajectoire générée par *Flow Matching* violerait continuellement la condition de sécurité. La courbe finale ne descend jamais sous $-m_{\text{CBF}}$ (pire valeur -5×10^{-4}) : le résidu introduit par le filtrage et les saturations est entièrement absorbé par la marge numérique, de sorte que la condition CBF proprement dite, $\nabla h^\top \dot{\mathbf{q}} + \kappa_{\text{eff}} h \geq 0$, est satisfaite à chaque cycle de l'essai. L'étape de réparation (Section 4.7.5) est loin d'être marginale dans ce résultat : elle intervient sur la quasi-totalité des cycles où la projection est active, le filtre de sortie érodant systématiquement une partie de la correction au sein du même cycle. L'écart entre les deux courbes constitue la mesure directe de l'intervention du filtre de sécurité.

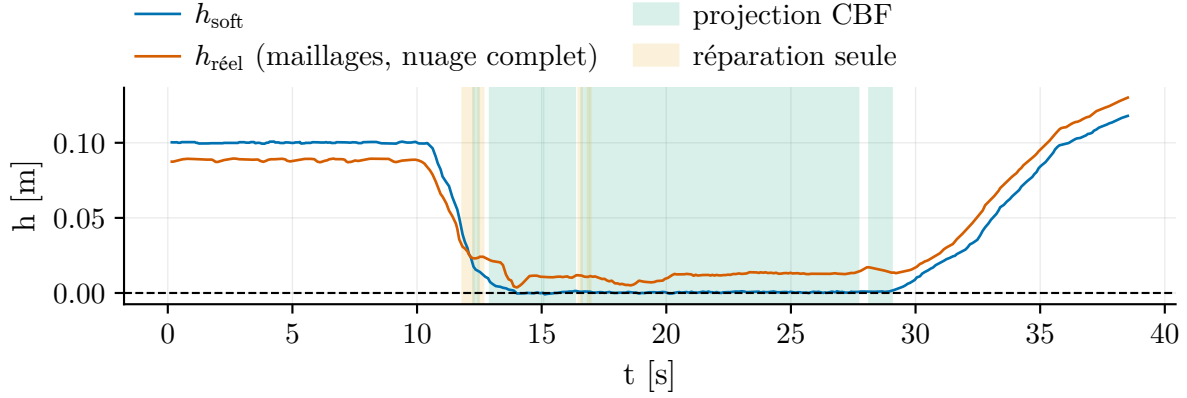


Figure 5.12 Évolution de la barrière h_{soft} et de sa valeur compensée h_{corr} pour une tâche en présence d'obstacles ; les bandes vertes indiquent les cycles où la projection corrige la vitesse, les bandes jaunes ceux où seule la réparation intervient.

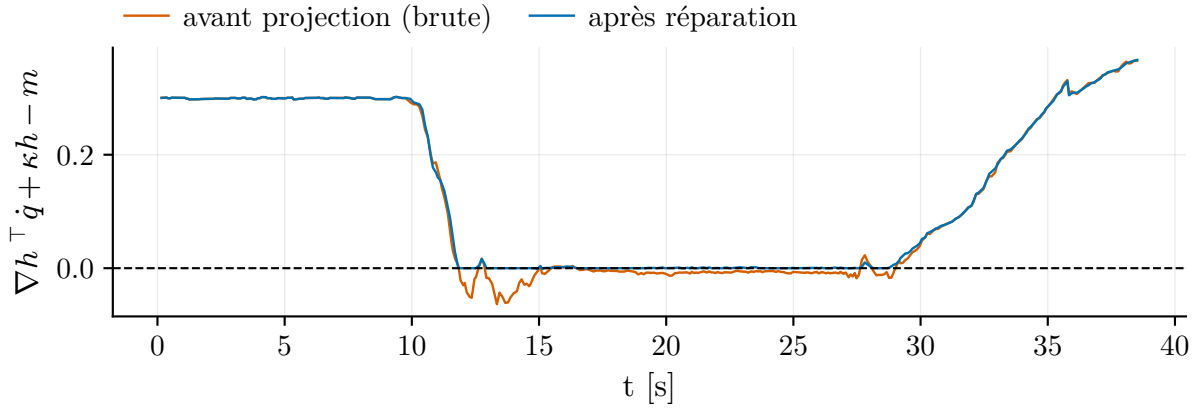


Figure 5.13 Contrainte de sécurité évaluée sur la vitesse nominale (avant projection) et sur la commande finale (après réparation). Les valeurs négatives de la courbe nominale correspondent aux violations qu'aurait commises la trajectoire générée sans filtre.

5.4.3 Déviation entre trajectoire nominale et trajectoire sécurisée

À compléter.

5.5 Analyse des vitesses et de la fluidité

5.5.1 Décomposition des vitesses articulaires

La Figure 5.14 déroule la chaîne complète des Sections 4.7.4 à 4.7.5. Le panneau (a) montre la construction de la vitesse nominale : le feedforward retimé $\dot{\mathbf{q}}_{\text{ff}}$, de norme constante par construction, la correction de suivi $\dot{\mathbf{q}}_{\text{fb}}$, et la vitesse nominale filtrée $\dot{\mathbf{q}}_{\text{base}}$ qui en résulte. Le panneau (b) montre ce que le filtre de sécurité en fait : les deux corrections, projection $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ et réparation $\Delta\dot{\mathbf{q}}_{\text{rep}}$, et la vitesse effectivement commandée $\dot{\mathbf{q}}_{\text{final}}$, superposées à $\dot{\mathbf{q}}_{\text{base}}$ comme référence commune aux deux panneaux. La réparation n'étant pas journalisée par le nœud, $\Delta\dot{\mathbf{q}}_{\text{rep}}$ est reconstituée en rejouant le filtre passe-bas de sortie sur les signaux enregistrés, reconstruction validée par un résidu médian de 10^{-4} rad/s sur les cycles sans réparation. Les normes sont exprimées dans l'espace articulaire (rad/s) ; la vitesse cartésienne de l'outil s'en déduit par la jacobienne et varie avec la configuration du bras.

Trois observations ressortent. Premièrement, $\|\dot{\mathbf{q}}_{\text{ff}}\|$ est constante à 0,25 rad/s sur toute la tâche : le retiming à vitesse articulaire constante (Section 4.6.3) remplit son rôle et fournit au filtre une référence régulière. Deuxièmement, la correction de projection est strictement nulle hors des zones d'activation : le filtre est transparent dès que la contrainte est satisfaite, ce qui confirme le mécanisme de transmission directe. On remarque d'ailleurs des épisodes où $\Delta\dot{\mathbf{q}}_{\text{rep}}$ est active alors que $\Delta\dot{\mathbf{q}}_{\text{CBF}}$ demeure nulle (vers 5 s et 22 s) : la barrière y effleure la frontière sans la presser — la projection juge la vitesse instantanée admissible, mais la commande filtrée, qui porte la mémoire des cycles précédents, échoue à la revérification et est corrigée par la réparation. Troisièmement, la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$ présente un bruit important dont l'analyse spectrale situe la puissance dominante à 12 Hz, soit exactement la fréquence de régénération du planificateur : chaque nouvelle trajectoire générée déplace légèrement la consigne, et le gain de suivi K_p transforme ces sauts en impulsions de vitesse.

5.5.2 Action du filtre de sécurité sur la vitesse commandée

La Figure 5.15 superpose la vitesse nominale $\dot{\mathbf{q}}_{\text{base}}$ (ce que le robot ferait en l'absence d'obstacle) et la vitesse commandée $\dot{\mathbf{q}}_{\text{final}}$. Sur les cycles où le filtre intervient effectivement (projection ou réparation), la surface entre les deux courbes est teintée selon le sens de l'intervention : freinage lorsque le filtre retire de la vitesse, poussée lorsqu'il en ajoute (récupération hors de

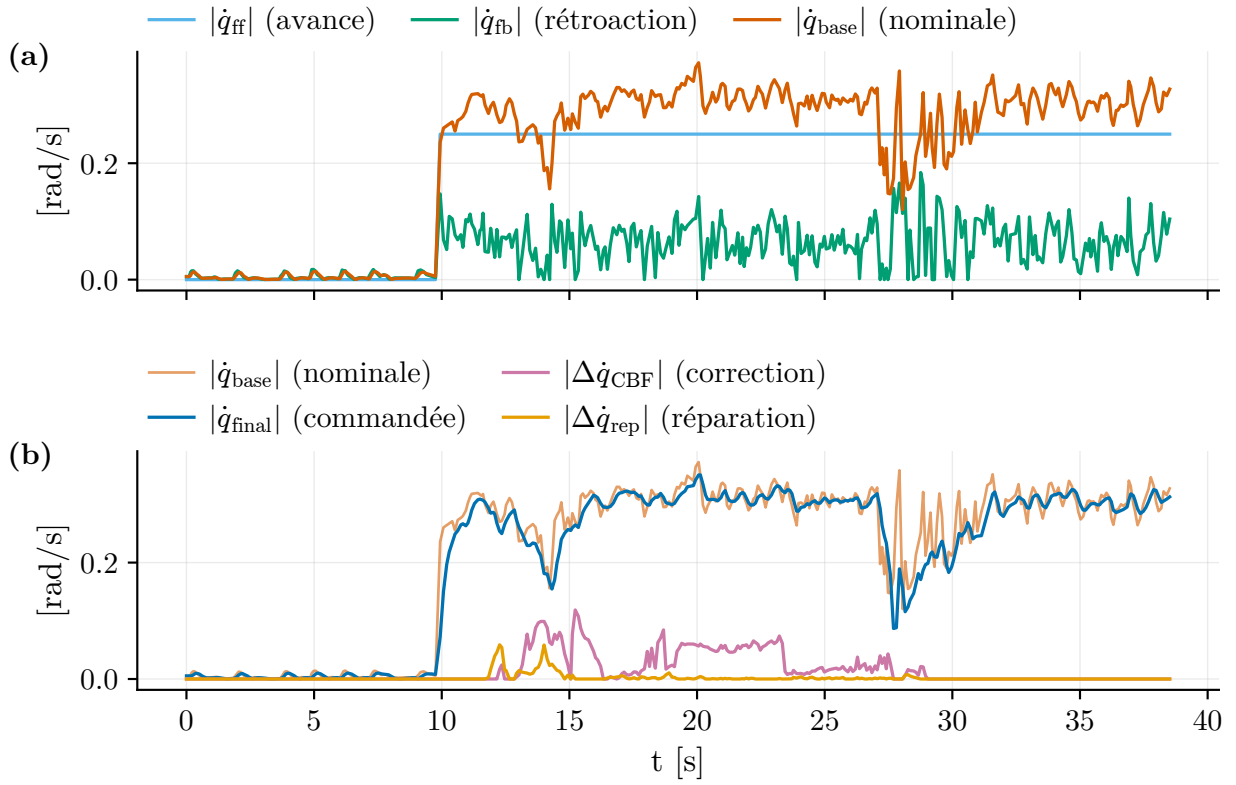


Figure 5.14 Décomposition des vitesses articulaires. (a) Construction de la vitesse nominale : feedforward retimé à norme constante $\dot{q}_{ff}$, correction de suivi $\dot{q}_{fb}$ et vitesse nominale filtrée $\dot{q}_{base}$. (b) Action du filtre de sécurité : corrections de projection $\Delta\dot{q}_{CBF}$ et de réparation $\Delta\dot{q}_{rep}$ (reconstituée par rejeu du filtre de sortie), et vitesse commandée $\dot{q}_{final}$. Courbes lissées sur 0,25 s.

la marge). L'écart entre les deux courbes se lit donc directement comme l'amplitude de la modification. Hors des zones d'activation, $\|\dot{\mathbf{q}}_{\text{final}}\|$ demeure légèrement sous $\|\dot{\mathbf{q}}_{\text{base}}\|$ ($\approx 5\%$ en moyenne) : ce n'est pas une intervention de sécurité mais l'effet du filtre passe-bas de sortie ($\tau_{\text{safe}} = 0,2$ s), qui, en moyennant des vecteurs dont la direction fluctue d'un cycle à l'autre, produit une norme inférieure à la moyenne des normes — le même mécanisme d'annulation vectorielle que celui décrit à la Section 5.5.3.

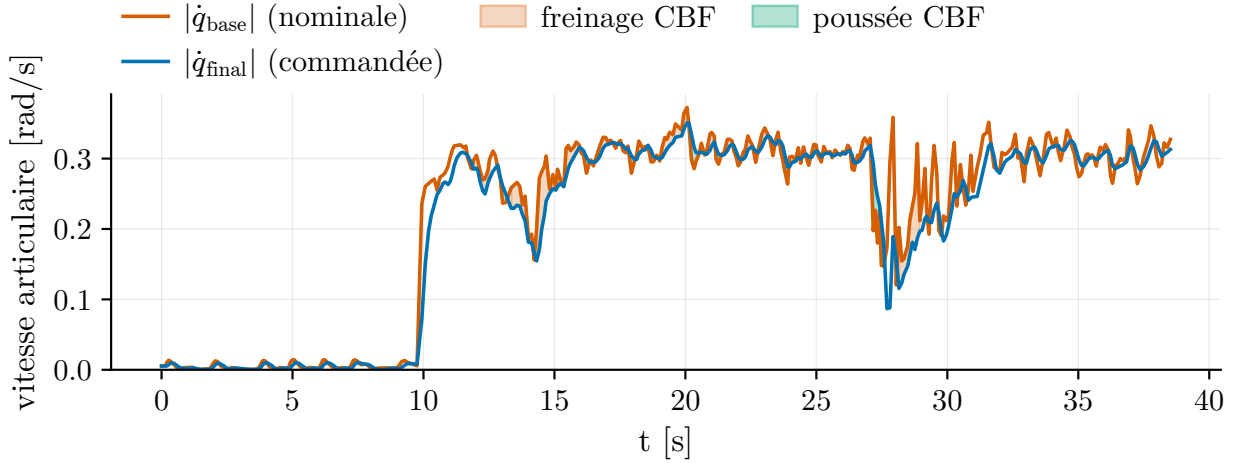


Figure 5.15 Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.

5.5.3 Effet du rééchantillonnage du planificateur sur la vitesse exécutée

La décomposition révèle un effet systémique du fonctionnement en horizon glissant. Bien que $\|\dot{\mathbf{q}}_{\text{ff}}\| = 0,25$ rad/s et que la correction de suivi soit en moyenne alignée avec le feed-forward (composante moyenne de $+0,03$ rad/s le long de $\dot{\mathbf{q}}_{\text{ff}}$), la vitesse nominale filtrée ne moyenne que $\approx 0,15$ rad/s. La cause n'est pas un freinage mais un phénomène vectoriel : à chaque régénération, la *direction* instantanée de $\dot{\mathbf{q}}_{\text{ff}}$ saute (rotations atteignant 80° entre cycles consécutifs au 95° centile), et le filtre passe-bas, en moyennant des vecteurs dont la direction alterne à 12 Hz, produit un vecteur de norme inférieure à la moyenne des normes. Près de 40 % de la vitesse demandée est ainsi perdue par annulation vectorielle, indépendamment de tout obstacle. Ce constat motive l'agrégation temporelle des trajectoires générées (moyennage pondéré des dernières générations) comme amélioration future du planificateur.

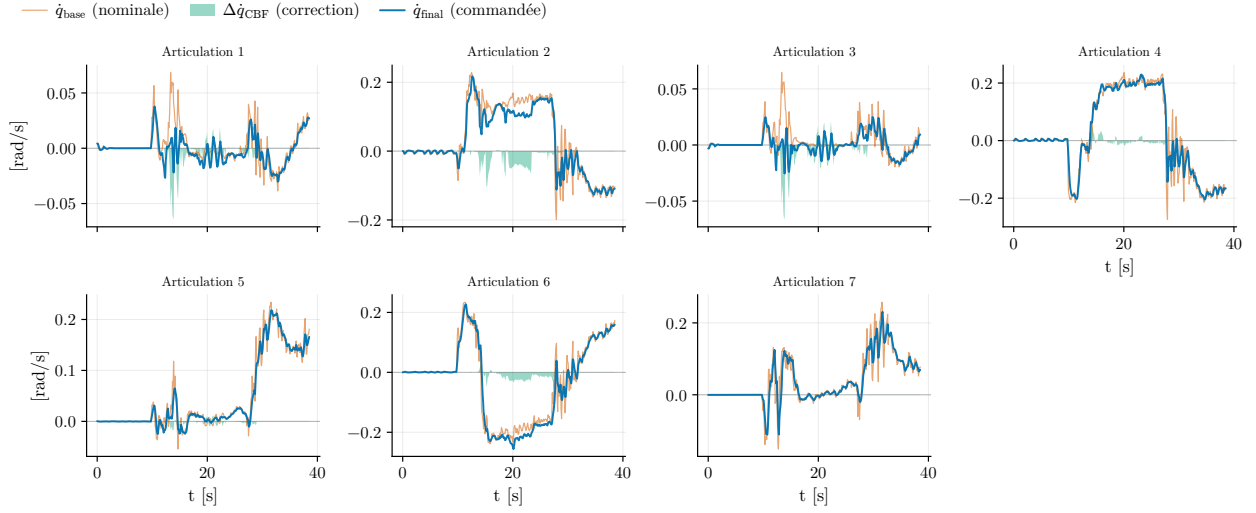


Figure 5.16 Action du filtre de sécurité : vitesse nominale et vitesse commandée superposées. Les surfaces teintées, restreintes aux cycles où le filtre intervient (projection ou réparation), distinguent le freinage de la poussée de récupération ; l'écart résiduel non teinté est l'effet du filtre passe-bas de sortie.

5.5.4 Effet du filtrage sur la commande sécurisée

La Figure 5.17 compare la norme de la vitesse sécurisée avant et après le filtre passe-bas de sortie. Le filtre supprime la gigue de direction du gradient induite par le rafraîchissement de la sélection des points critiques à 30 Hz, au prix d'un retard de l'ordre de τ_{safe} ; la contrainte de sécurité est réévaluée après filtrage par l'étape de réparation (Section 4.7.5).

5.5.5 Analyse de l'interface de commande

L'instrumentation du rapport entre le taux de variation de la barrière commandé ($\nabla h^T \dot{\mathbf{q}}_{\text{final}}$) et celui réellement exécuté ($\nabla h^T \dot{\mathbf{q}}_{\text{mes}}$) a mis en évidence deux modes de défaillance de l'interface entre le filtre de sécurité et le contrôleur de position bas niveau, justifiant a posteriori les choix de conception de la Section 4.7.5. Avec une consigne ré-ancrée à chaque cycle sur la position mesurée, le rapport exécuté/commandé mesuré était proche de zéro : les corrections de sécurité étaient réabsorbées avant d'être réalisées, et le robot demeurerait immobilisé contre la frontière malgré une commande de dégagement soutenue. À l'inverse, avec un pas d'intégration supérieur à la période réelle de la boucle, la référence avançait plus vite que la vitesse certifiée et le rapport devenait négatif et supérieur à l'unité en valeur absolue : le robot traversait la frontière en oscillant, produisant des violations répétées de la marge. L'interface retenue — intégration de la référence persistante au pas réel Δt avec borne d'avance

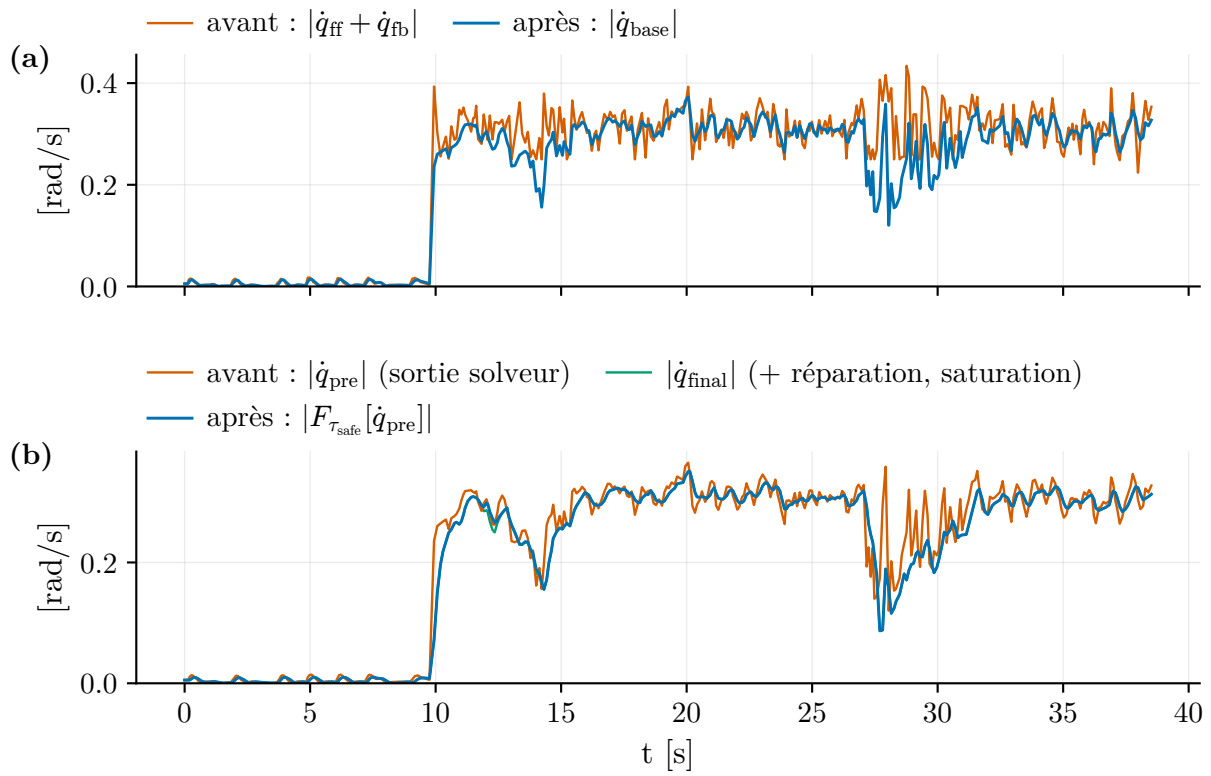


Figure 5.17 Norme de la vitesse sécurisée avant et après le filtre passe-bas de sortie ($\tau_{safe} = 0,2$ s).

— élimine ces deux régimes.

5.6 Performance et temps de calcul

Tableau 5.1 Statistiques descriptives par étape (stage)

Stage	n	Mean	std/min	Med	p95	Max
cbf_command	1660	1.88	2.12	1.38	4.98	31.10
cbf_correction	1660	7.58	5.41	6.50	17.65	41.44
cbf_total	1660	9.91	6.43	8.60	22.06	57.81
critical_point_selection	832	42.27	18.35	39.62	75.49	148.81
fm_encode	263	1.51	2.21	1.02	2.85	20.87
fm_generation	263	62.07	6.40	61.62	73.89	83.89
ik	255	0.27	0.09	0.25	0.43	0.93
persistent_evict	709	3.17	1.74	2.76	5.90	22.08
persistent_integrate	1753	1.02	1.06	0.84	1.80	23.85
sam2_track	132	53.85	15.10	52.77	78.66	91.14
sam3_detect	2	182.45	152.72	182.45	319.89	335.16

À compléter.

5.7 Performances globales et limites

5.7.1 Taux de succès des scénarios

Pick and place

Robot assisted feeding

5.7.2 Cas d'échec

À compléter.

5.8 Discussion

CHAPITRE 6 CONCLUSION

Texte / Text.

6.1 Synthèse des travaux / Summary of Works

Texte / Text.

6.2 Limitations de la solution proposée / Limitations

6.3 Améliorations futures / Future Research

Texte / Text.

RÉFÉRENCES

- [1] S. Karaman et E. Frazzoli, “Sampling-Based Algorithms for Optimal Motion Planning,” *The International Journal of Robotics Research (IJRR)*, vol. 30, n°. 7, p. 846–894, 2011.
- [2] I. A. Şucan, M. Moll et L. E. Kavraki, “The Open Motion Planning Library,” *IEEE Robotics & Automation Magazine*, vol. 19, n°. 4, p. 72–82, 2012.
- [3] S. Liu et P. Liu, “Robot Motion Planning Benchmarking and Optimization using Motion Planning Pipeline,” 2021, arXiv preprint.
- [4] M. Zucker *et al.*, “CHOMP : Covariant Hamiltonian Optimization for Motion Planning,” *The International Journal of Robotics Research (IJRR)*, vol. 32, n°. 9-10, p. 1164–1193, 2013.
- [5] J. Schulman *et al.*, “Motion Planning with Sequential Convex Optimization and Convex Collision Checking,” *The International Journal of Robotics Research (IJRR)*, vol. 33, n°. 9, p. 1251–1270, 2014.
- [6] J. J. Kuffner et S. M. LaValle, “RRT-Connect : An Efficient Randomized Spatio-temporal Planner,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2000, p. 995–1001.
- [7] L. E. Kavraki *et al.*, “Probabilistic Roadmaps for Path Planning in High-Dimensional Configuration Spaces,” *IEEE Transactions on Robotics and Automation*, vol. 12, n°. 4, p. 566–580, 1996.
- [8] C. G. Atkeson et S. Schaal, “Robot Learning from Demonstration,” dans *International Conference on Machine Learning (ICML)*, 1997, p. 12–20.
- [9] S. Schaal, “Is Imitation Learning the Way to Eager Robotics?” *Trends in Cognitive Sciences*, vol. 3, n°. 6, p. 233–242, 1999.
- [10] B. D. Argall *et al.*, “A survey of robot learning from demonstration,” 2009, arXiv preprint.
- [11] D. A. Pomerleau, “ALVINN : An Autonomous Land Vehicle in a Neural Network,” dans *Advances in Neural Information Processing Systems (NeurIPS)*, 1989, p. 305–313.
- [12] M. Bain et C. Sammut, “A Framework for Behavioural Cloning,” dans *Machine Intelligence 15*, 1995, p. 103–129.
- [13] T. Z. Zhao *et al.*, “Learning Fine-Grained Bimanual Manipulation with Low-Cost Hardware,” 2023, arXiv preprint.

- [14] P. Sundaresan, S. Belkhale et D. Sadigh, “Learning Visuo-Haptic Skewering Strategies for Robot-Assisted Feeding,” 2022, arXiv preprint.
- [15] R. Liu, A. Bhaskar et P. Tokekar, “Adaptive Visual Imitation Learning for Robotic Assisted Feeding Across Varied Bowl Configurations and Food Types,” 2024, arXiv preprint.
- [16] S. Levine, “Supervised learning of behaviors,” Lecture notes, CS 285 : Deep Reinforcement Learning, UC Berkeley, 2026, accessed : 2026-07-05. [En ligne]. Disponible : <https://rail.eecs.berkeley.edu/deeprlcourse/static/slides/lec-2.pdf>
- [17] P. Florence *et al.*, “Implicit Behavioral Cloning,” 2021, arXiv preprint.
- [18] D. Jarrett, I. Bica et M. v. d. Schaar, “Strictly Batch Imitation Learning by Energy-based Distribution Matching,” 2021, arXiv preprint.
- [19] P. Florence, L. Manuelli et R. Tedrake, “Self-Supervised Correspondence in Visuomotor Policy Learning,” 2019, arXiv preprint.
- [20] J. Luo *et al.*, “Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning,” 2025, arXiv preprint.
- [21] A. Gupta *et al.*, “Relay Policy Learning : Solving Long-Horizon Tasks via Imitation and Reinforcement Learning,” 2019, arXiv preprint.
- [22] S. M. Khansari-Zadeh et A. Billard, “Learning Stable Nonlinear Dynamical Systems With Gaussian Mixture Models,” 2011, arXiv preprint.
- [23] R. Wolf *et al.*, “Diffusion Models for Robotic Manipulation : A Survey,” *arXiv preprint arXiv :2504.08438*, 2025.
- [24] J. Cho, M. Kim et S. Jung, “A Survey on Flow Matching for Robotic Trajectory Generation and Control,” 2025, arXiv preprint.
- [25] C. Chi *et al.*, “Diffusion Policy : Visuomotor Policy Learning via Action Diffusion,” dans *Proceedings of Robotics : Science and Systems (RSS)*, 2023.
- [26] J. Ho, A. Jain et P. Abbeel, “Denoising Diffusion Probabilistic Models,” 2020, arXiv preprint.
- [27] M. Janner *et al.*, “Planning with Diffusion for Flexible Behavior Synthesis,” 2022, arXiv preprint.
- [28] S. Huang *et al.*, “Diffusion-based Generation, Optimization, and Planning in 3D Scenes,” 2023, arXiv preprint.
- [29] Y. Ze *et al.*, “3D Diffusion Policy : Generalizable Visuomotor Policy Learning via Simple 3D Representations,” 2024, arXiv preprint.

- [30] G. Lu *et al.*, “ManiCM : Real-time 3D Diffusion Policy via Consistency Model for Robotic Manipulation,” 2025, arXiv preprint.
- [31] A. Prasad *et al.*, “Consistency Policy : Accelerated Visuomotor Policies via Consistency Distillation,” 2024, arXiv preprint.
- [32] Z. Wang *et al.*, “One-Step Diffusion Policy : Fast Visuomotor Policies via Diffusion Distillation,” 2024, arXiv preprint.
- [33] Y. Lipman *et al.*, “Flow Matching for Generative Modeling,” dans *International Conference on Learning Representations (ICLR)*, 2023.
- [34] A. Tong *et al.*, “Improving and Generalizing Flow-Based Generative Models with Minibatch Optimal Transport,” *Transactions on Machine Learning Research (TMLR)*, 2024.
- [35] X. Liu, C. Gong et Q. Liu, “Flow Straight and Fast : Learning to Generate and Transfer Data with Rectified Flow,” dans *International Conference on Learning Representations (ICLR)*, 2023.
- [36] E. Chisari *et al.*, “Learning Robotic Manipulation Policies from Point Clouds with Conditional Flow Matching,” 2024, arXiv preprint.
- [37] G. Yan *et al.*, “ManiFlow : A General Robot Manipulation Policy via Consistency Flow Training,” 2025, arXiv preprint.
- [38] Z. Geng *et al.*, “Mean Flows for One-step Generative Modeling,” 2025, arXiv preprint.
- [39] H. Fang *et al.*, “OMP : One-step Meanflow Policy with Directional Alignment,” 2026, arXiv preprint.
- [40] S. Jiang *et al.*, “Streaming Flow Policy : Simplifying diffusion/flow-matching policies by treating action trajectories as flow trajectories,” 2025, arXiv preprint.
- [41] K. Nguyen *et al.*, “FlowMP : Learning Motion Fields for Robot Planning with Conditional Flow Matching,” 2025, arXiv preprint.
- [42] D. Soleymanzadeh, X. Liang et M. Zheng, “Flow Motion Policy : Manipulator Motion Planning with Flow Matching Models,” 2026, arXiv preprint.
- [43] M. Braun *et al.*, “Riemannian Flow Matching Policy for Robot Motion Learning,” 2024, arXiv preprint.
- [44] Y. Lipman *et al.*, “Flow matching guide and code,” 2024. [En ligne]. Disponible : <https://arxiv.org/abs/2412.06264>
- [45] A. Brohan *et al.*, “RT-2 : Vision-Language-Action Models Transfer Web Knowledge to Robotic Control,” 2023, arXiv preprint.

- [46] M. J. Kim *et al.*, “OpenVLA : An Open-Source Vision-Language-Action Model,” 2024, arXiv preprint.
- [47] R. Sapkota *et al.*, “Vision-Language-Action (VLA) Models : Concepts, Progress, Applications and Challenges,” 2026, arXiv preprint.
- [48] D. Li *et al.*, “What Foundation Models can Bring for Robot Learning in Manipulation : A Survey,” *The International Journal of Robotics Research*, 2025.
- [49] J. Wen *et al.*, “TinyVLA : Towards Fast, Data-Efficient Vision-Language-Action Models for Robotic Manipulation,” 2025, arXiv preprint.
- [50] S. Liu *et al.*, “Vision-language model-driven scene understanding and robotic object manipulation,” 2024, arXiv preprint.
- [51] O. Mees, L. Hermann et W. Burgard, “What Matters in Language Conditioned Robotic Imitation Learning over Unstructured Data,” 2022, arXiv preprint.
- [52] N. Ingelhart *et al.*, “A Robotic Skill Learning System Built Upon Diffusion Policies and Foundation Models,” 2024, arXiv preprint.
- [53] G. Yin *et al.*, “CodeDiffuser : Attention-Enhanced Diffusion Policy via VLM-Generated Code for Instruction Ambiguity,” 2025, arXiv preprint.
- [54] A. Kirillov *et al.*, “Segment Anything,” dans *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023, p. 4015–4026.
- [55] Meta AI Research, “SAM 3 : Segment Anything with Concepts,” *arXiv preprint*, 2025.
- [56] N. Ravi *et al.*, “SAM 2 : Segment Anything in Images and Videos,” *arXiv preprint arXiv :2408.00714*, 2024.
- [57] D. Seita *et al.*, “ToolFlowNet : Robotic Manipulation with Tools via Predicting Tool Flow from Point Clouds,” 2022, arXiv preprint.
- [58] H. Li *et al.*, “Language-Guided Object-Centric Diffusion Policy for Generalizable and Collision-Aware Manipulation,” 2025, arXiv preprint.
- [59] R. Yang *et al.*, “FP3 : A 3D Foundation Policy for Robotic Manipulation,” 2025, arXiv preprint.
- [60] Y. Fu *et al.*, “CordViP : Correspondence-based Visuomotor Policy for Dexterous Manipulation in Real-World,” 2025, arXiv preprint.
- [61] R. Yu *et al.*, “No Need for Real 3D : Fusing 2D Vision with Pseudo 3D Representations for Robotic Manipulation Learning,” 2025, arXiv preprint.
- [62] Z. Ni *et al.*, “VO-DP : Semantic-Geometric Adaptive Diffusion Policy for Vision-Only Robotic Manipulation,” 2025, arXiv preprint.

- [63] J. Jones *et al.*, “Beyond Sight : Finetuning Generalist Robot Policies with Heterogeneous Sensors via Language Grounding,” 2025, arXiv preprint.
- [64] J. Morris *et al.*, “DynoSAM : Open-Source Smoothing and Mapping Framework for Dynamic SLAM,” 2025, arXiv preprint.
- [65] Y. Jin *et al.*, “SE(3)-PoseFlow : Estimating 6D Pose Distributions for Uncertainty-Aware Robotic Manipulation,” 2025, arXiv preprint.
- [66] A. Elfes, “Using Occupancy Grids for Mobile Robot Perception and Navigation,” *Computer*, vol. 22, n^o. 6, p. 46–57, 1989.
- [67] A. Hornung *et al.*, “OctoMap : An Efficient Probabilistic 3D Mapping Framework Based on Octrees,” *Autonomous Robots*, vol. 34, n^o. 3, p. 189–206, 2013.
- [68] B. Curless *et al.*, “A Volumetric Method for Building Complex Models from Range Images,” dans *Proceedings of the 23rd Annual Conference on Computer Graphics and Interactive Techniques*, ser. SIGGRAPH ’96. ACM, 1996, p. 303–312.
- [69] R. A. Newcombe *et al.*, “KinectFusion : Real-time Dense Surface Mapping and Tracking,” dans *Proceedings of the IEEE International Symposium on Mixed and Augmented Reality (ISMAR)*, 2011, p. 127–136.
- [70] H. Oleynikova *et al.*, “Voxblox : Incremental 3D Euclidean Signed Distance Fields for On-Board MAV Planning,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, p. 1366–1373.
- [71] A. Millane *et al.*, “nvblox : GPU-Accelerated Incremental Signed Distance Field Mapping,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2024.
- [72] L. Han *et al.*, “FIESTA : Fast Incremental Euclidean Distance Fields for Online Motion Planning of Aerial Robots,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2019, p. 4423–4430.
- [73] Y. Pan *et al.*, “Voxfield : Non-Projective Signed Distance Fields for Online Planning and 3D Reconstruction,” dans *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2022, p. 10 173–10 180.
- [74] W. B. Bedada *et al.*, “Real Time Collision Avoidance with GPU-Computed Distance Maps,” 2024, arXiv preprint.
- [75] J. Ortiz *et al.*, “iSDF : Real-Time Neural Signed Distance Fields for Robot Perception,” dans *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, p. 4654–4663.
- [76] E. Sucar *et al.*, “iMAP : Implicit Mapping and Positioning in Real-Time,” dans *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, 2021, p. 6229–6238.

- [77] Z. Zhu *et al.*, “NICE-SLAM : Neural Implicit Scalable Encoding for SLAM,” dans *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2022, p. 12 776–12 786.
- [78] B. Sundaralingam *et al.*, “CuRobo : Parallelized Collision-Free Robot Motion Generation,” dans *IEEE International Conference on Robotics and Automation (ICRA)*, 2023, p. 8112–8119.
- [79] J. Lee *et al.*, “Learning Fast, Tool aware Collision Avoidance for Collaborative Robots,” 2025, arXiv preprint.
- [80] Y. Li *et al.*, “Representing robot geometry as distance fields : Applications to whole-body manipulation,” dans *Proceedings of the IEEE International Conference on Robotics and Automation (ICRA)*, 2024, p. 15 351–15 357.
- [81] B. Liu *et al.*, “Collision-free Motion Generation Based on Stochastic Optimization and Composite Signed Distance Field Networks of Articulated Robot,” 2023, arXiv preprint.
- [82] A. Govoni *et al.*, “Real-time collision avoidance with robot distance fields in a task-priority framework,” *Robotics and Autonomous Systems*, vol. 200, p. 105394, 2026.
- [83] O. Khatib, “Real-Time Obstacle Avoidance for Manipulators and Mobile Robots,” *The International Journal of Robotics Research (IJRR)*, vol. 5, n^o. 1, p. 90–98, 1986.
- [84] H. Ding *et al.*, “Towards Safe Imitation Learning via Potential Field-Guided Flow Matching,” 2025, arXiv preprint.
- [85] A. D. Ames *et al.*, “Control Barrier Functions : Theory and Applications,” 2019, arXiv preprint.
- [86] P. Wieland et F. Allgöwer, “CONSTRUCTIVE SAFETY USING CONTROL BARRIER FUNCTIONS,” 2007, arXiv preprint.
- [87] A. D. Ames *et al.*, “Control Barrier Function Based Quadratic Programs for Safety Critical Systems,” 2017, arXiv preprint.
- [88] W. S. Cortez *et al.*, “Control Barrier Functions for Mechanical Systems : Theory and Application to Robotic Grasping,” 2019, arXiv preprint.
- [89] K.-C. Hsu, H. Hu et J. F. Fisac, “The Safety Filter : A Unified View of Safety-Critical Control in Autonomous Systems,” *Annual Review of Control, Robotics, and Autonomous Systems*, vol. 7, p. 47–72, 2024.
- [90] K. P. Wabersich et M. N. Zeilinger, “A Predictive Safety Filter for Learning-Based Control of Constrained Nonlinear Dynamical Systems,” *Automatica*, vol. 129, p. 109597, 2021.

- [91] K. P. Wabersich *et al.*, “Data-Driven Safety Filters : Hamilton-Jacobi Reachability, Control Barrier Functions, and Predictive Methods for Uncertain Systems,” *IEEE Control Systems Magazine*, vol. 43, n°. 5, p. 137–177, 2023.
- [92] A. Agrawal et K. Sreenath, “Discrete Control Barrier Functions for Safety-Critical Control of Discrete Systems with Application to Bipedal Robot Navigation,” dans *Robotics : Science and Systems (RSS)*, 2017.
- [93] L. E. Beaver, “Optimal Control Barrier Functions : Maximizing the Action Space Subject to Control Bounds,” 2024, arXiv preprint.
- [94] X. Dai *et al.*, “SafeFlow : Safe Robot Motion Planning with Flow Matching via Control Barrier Functions,” 2025, arXiv preprint.
- [95] J. Yang, S. Jang et S. Han, “SafeFlowMatcher : Safe and Fast Planning using Flow Matching with Control Barrier Functions,” 2026, arXiv preprint.
- [96] J. Long *et al.*, “Constraining Streaming Flow Models for Adapting Learned Robot Trajectory Distributions,” 2026, arXiv preprint.
- [97] K. Mizuta et K. Leung, “CoBL-Diffusion : Diffusion-Based Conditional Robot Planning in Dynamic Environments Using Control Barrier and Lyapunov Functions,” 2024, arXiv preprint.
- [98] Y. Song *et al.*, “OmniGuide : Universal Guidance Fields for Enhancing Generalist Robot Policies,” 2026, arXiv preprint.
- [99] Z. Yang *et al.*, “UniConFlow : A Unified Constrained Flow-Matching Framework for Certified Motion Planning,” 2026, arXiv preprint.
- [100] T.-Y. Huang *et al.*, “SAD-Flow : Flow Matching for Safe, Admissible, and Dynamically Consistent Planning,” 2026, arXiv preprint.
- [101] K. Mizuta et K. Leung, “Unified Generation-Refinement Planning : Bridging Guided Flow Matching and Sampling-Based MPC for Social Navigation,” 2026, arXiv preprint.
- [102] R. Römer *et al.*, “From Demonstrations to Safe Deployment : Path-Consistent Safety Filtering for Diffusion Policies,” 2026, arXiv preprint.
- [103] E. Welte *et al.*, “FlowCorrect : Efficient Interactive Correction of Generative Flow Policies for Robotic Manipulation,” 2026, arXiv preprint.
- [104] A. Liao *et al.*, “Delay-Aware Diffusion Policy : Bridging the Observation-Execution Gap in Dynamic Tasks,” 2025, arXiv preprint.
- [105] X. Ye *et al.*, “RA-DP : Rapid Adaptive Diffusion Policy for Training-Free High-frequency Robotics Replanning,” 2025, arXiv preprint.

- [106] F. Xiang *et al.*, “SAPIEN : A simulated part-based interactive environment,” dans *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2020.
- [107] Y. Zhou *et al.*, “On the Continuity of Rotation Representations in Neural Networks,” 2020, arXiv preprint.
- [108] A. Vaswani *et al.*, “Attention Is All You Need,” dans *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [109] Y. Nakamura et H. Hanafusa, “Inverse Kinematic Solutions With Singularity Robustness for Robot Manipulator Control,” *Journal of Dynamic Systems, Measurement, and Control*, vol. 108, n^o. 3, p. 163–171, 1986.
- [110] C. W. Wampler, “Manipulator Inverse Kinematic Solutions Based on Vector Formulations and Damped Least-Squares Methods,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 16, n^o. 1, p. 93–101, 1986.
- [111] A. Li’geois, “Automatic Supervisory Control of the Configuration and Behavior of Multibody Mechanisms,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 7, n^o. 12, p. 868–871, 1977.

ANNEXE A PROTOCOLE DÉTAILLÉ D’ACQUISITION DES DÉMONSTRATIONS

Cette annexe détaille la mise en œuvre du protocole de collecte de la Section 4.4.1 : la procédure opératoire, l’architecture logicielle d’acquisition et le format de stockage des enregistrements. Ce format, défini en début de projet, est autoporteur : les données sont lisibles sans dépendance à ROS (JSON, PNG, NumPy), ce qui permet au pipeline d’entraînement de les consommer directement, là où un conteneur standard de type `rosvbag` aurait exigé une étape d’extraction préalable.

Procédure opératoire

Chaque démonstration réelle suit la séquence suivante :

1. montage de la caméra RGB-D sur le support officiel Franka à l’effecteur ;
2. positionnement du robot dans la configuration articulaire initiale $\mathbf{q}_0$ donnée à la Section 4.4.1 ;
3. passage du robot en mode de guidage manuel, permettant à l’opérateur de le déplacer librement ;
4. exécution par guidage kinesthésique de la trajectoire à apprendre, depuis la posture initiale jusqu’à la cible, selon le schéma d’approche de la Section 4.4.1 ;
5. arrêt de l’enregistrement et écriture des données de la trajectoire dans un fichier JSON.

Architecture d’acquisition

L’enregistrement repose sur deux nœuds ROS distincts. Le premier, écrit en C++ pour minimiser la latence de lecture, lit l’état du robot et publie :

- la pose du TCP dans le repère de base (1×7) : position cartésienne (trois premières composantes) et orientation en quaternion (quatre dernières) ;
- la vitesse du TCP dans le repère de base (1×6) : vitesses linéaires (trois premières composantes) puis angulaires (trois dernières).

Le second nœud, écrit en Python, s’abonne à ces messages ainsi qu’au flux RGB-D de la caméra à l’effecteur, et écrit l’ensemble sur disque à la cadence de 10 Hz. La Figure A.1 illustre cette architecture.

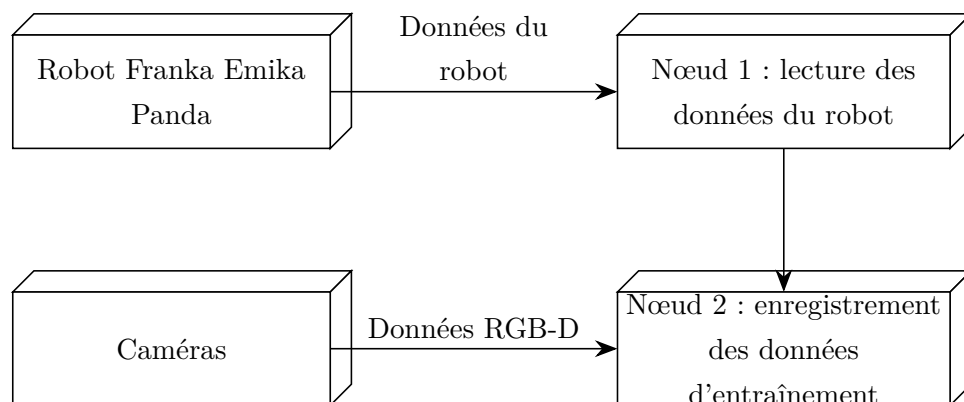


Figure A.1 Architecture du système d'acquisition des données d'entraînement.

Format d'enregistrement

À chaque pas d'échantillonnage, le second nœud consigne dans le fichier JSON, en plus de la pose et de la vitesse du TCP, les champs structurels suivants :

- `step_number` : l'indice de l'échantillon dans la trajectoire ;
- `time_step` : l'instant de la prise de données depuis le début de la trajectoire ;
- `rgb` : le nom du fichier de l'image couleur, nommé par convention `ee_rgb_step_{step_number}.png` ;
- `depth_npy` : le nom du fichier de la carte de profondeur, nommé par convention `ee_depth_step_{step_number}.npy`.

Les images et cartes de profondeur sont écrites dans un sous-dossier dédié ; le fichier JSON n'en contient que les noms.

Organisation des fichiers

Chaque trajectoire enregistrée occupe un dossier `Trajectory_X`, où `X` est le numéro de la trajectoire, regroupant le fichier JSON des données numériques et le sous-dossier des images et cartes de profondeur (Figure A.2).

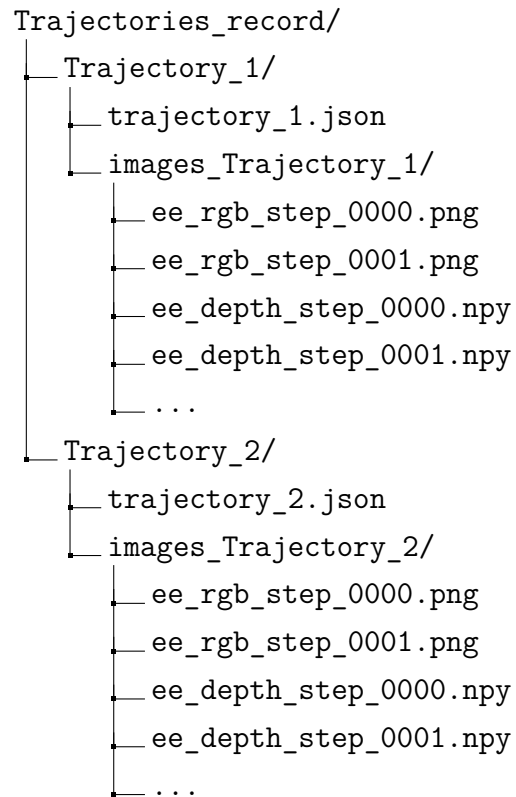


Figure A.2 Structure des dossiers du jeu de données brut.

Le prétraitement décrit à la Section 4.4.2 produit un second dossier racine, miroir du premier : les nuages de points conditionnels fusionnés y sont enregistrés sous le nom `Merged_{step_number}.npy` dans un sous-dossier `Merged_Trajectory_X`, et le fichier JSON de la trajectoire y est copié, augmenté de la pose de l'outil dans le repère de base (Figure A.3).

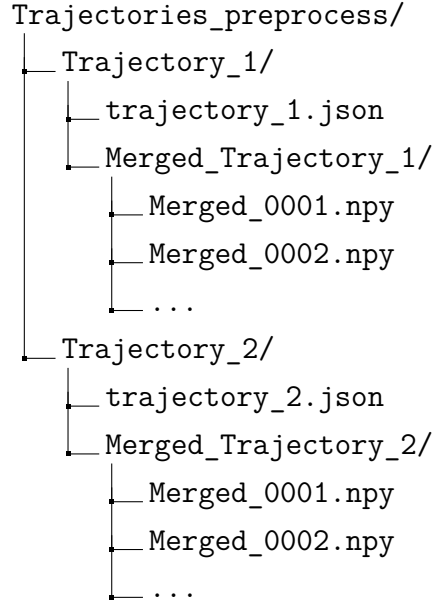


Figure A.3 Structure des dossiers du jeu de données prétraité.

Collecte en simulation

Les collectes simulées reprennent l’infrastructure d’enregistrement ci-dessus à l’identique (cadence de 10 Hz en temps simulé, nomenclature et organisation des fichiers identiques), en l’enrichissant : les environnements *ManiSkill 3* sont vectorisés (plusieurs environnements simulés en parallèle sur GPU, commande en mode `pd_ee_delta_pose`), 25 pas de stabilisation physique précèdent l’enregistrement, et le fichier JSON contient en plus les positions et vitesses articulaires, le flux d’une seconde caméra statique, les matrices extrinsèques des deux caméras (converties de la convention SAPIEN vers la convention OpenCV) ainsi que des métadonnées de scène (tâche, type de cible, type de support).

Pour la tâche d’alimentation, le robot est le Panda muni de la fourchette, obtenu par composition d’URDF, la caméra de poignet (640×480 , champ de vue de 42°) répliquant le montage réel. La cible est un maillage rigide tiré parmi sept aliments (pomme de terre, poitrine de poulet, brocoli, sushi, rouleau de sushi, cube de saumon, saucisse), placé aléatoirement dans un rectangle de 12×16 cm du plan de travail avec un lacet uniforme sur $[-\pi, \pi]$; une fois sur deux, il repose sur un piédestal de hauteur uniforme entre 4 et 10 cm. La politique experte procède en trois phases : approche avec alignement du lacet sur l’axe principal de la cible (abordée du côté opposé au robot, tangage de -10°), survol à 3 cm puis piquage vertical d’une profondeur $\min(2h_z - 2 \text{ mm}, 15 \text{ mm})$, où h_z est la demi-hauteur de l’aliment, bornée par la surface d’appui ; redressement du poignet le long d’un arc de rayon 3,5 cm jusqu’à un

tangage de 80° ; transport vers une pose de livraison fixe. Dès le piquage, l'aliment est cinématiquement solidarisé à la fourchette (pose relative verrouillée réappliquée à chaque pas), idéalisant un piquage ferme. Trois environnements parallèles sur cinq épisodes produisent les 15 trajectoires (Figure 4.4).

Pour la tâche de saisie et dépôt, le robot est le Panda standard et la saisie est physique : la politique ne commande que la position et la pince, l'orientation verticale de la pince étant fixée par la configuration initiale et maintenue par le contrôleur. Le cube (dynamique, 4 cm, lacet aléatoire) apparaît dans un rectangle de 10×10 cm sous l'empreinte initiale de la caméra de poignet, ce qui garantit sa visibilité dans la première image ; la boîte de dépôt ($16 \times 16 \times 1$ cm) apparaît dans une zone aléatoire à la droite du robot couvrant les positions utilisées au déploiement. La machine à états à huit phases (survol, descente, saisie, dégagement, transport, dépôt, relâchement, retrait) suit ses segments de chemin par jalon : le paramètre d'avancement ne progresse que lorsque le TCP a rejoint le point courant du chemin, de sorte que les portions raides de l'arc de transport sont effectivement parcourues plutôt que coupées. Comme pour la tâche d'alimentation, le cube et la boîte sont segmentés par SAM 3 dans la première image et leurs nuages de points sont reconstruits par la chaîne de rétroprojection de la Section 4.4.2. Quatre environnements parallèles sur dix épisodes produisent les 40 trajectoires (Figure 4.5).

ANNEXE B ENTRAÎNEMENT DU CHAMP DE DISTANCE SIGNÉE DU ROBOT

Cette annexe rassemble le détail de l'apprentissage du modèle de distance de la Section 4.7.1. La génération des étiquettes et la régression récursive reproduisent le protocole *Robot Distance Fields* de Li et al. [80] et son implémentation publique ; elles ne constituent pas une contribution de ce mémoire et sont reproduites ici pour rendre le document autonome. Seuls les hyperparamètres du raffinement géométrique, introduit à la Section 4.7.1, sont propres à ce travail.

Génération des étiquettes de distance

Pour chaque lien protégé, le maillage est d'abord normalisé selon l'équation (4.16), puis un jeu d'étiquettes de distance signée exacte est construit suivant le protocole de [80], lui-même hérité de DeepSDF : 8×10^5 points concentrés près de la surface, obtenus par balayages virtuels du maillage, le signe étant déterminé par les normales, et autant de points tirés uniformément dans le cube $[-1, 1]^3$, chacun étiqueté de sa distance signée exacte au maillage. Les étiquettes négatives situées hors de la boîte englobante du lien, artefacts du calcul de signe par normales, sont écartées.

Régression récursive des poids

La sortie du modèle étant linéaire en ses paramètres $\mathbf{w}_l$, l'ajustement est un problème de moindres carrés, résolu de manière récursive pour éviter l'inversion d'une matrice de n^3 lignes assemblée sur l'ensemble du jeu de données. À chaque itération, un lot de points encodés Φ_k , d'étiquettes $\mathbf{d}_k$, met à jour les poids en forme close :

$$\mathbf{K}_k = \mathbf{B}_{k-1} \Phi_k^\top \left(\mathbf{I} + \Phi_k \mathbf{B}_{k-1} \Phi_k^\top \right)^{-1} \quad (\text{B.1})$$

$$\mathbf{B}_k = \mathbf{B}_{k-1} - \mathbf{K}_k \Phi_k \mathbf{B}_{k-1} \quad (\text{B.2})$$

$$\mathbf{w}_k = \mathbf{w}_{k-1} + \mathbf{K}_k (\mathbf{d}_k - \Phi_k \mathbf{w}_{k-1}) \quad (\text{B.3})$$

avec l'initialisation $\mathbf{B}_0 = 10^2 \mathbf{I}$, équivalente à une régularisation de type *ridge* de la solution. Ces mises à jour correspondent à l'algorithme d'apprentissage récursif de [80] ; 80 itérations sont conduites par lien.

Hyperparamètres du raffinement géométrique

Le raffinement géométrique de la Section 4.7.1 part des poids issus de la régression récursive et les affine par descente de gradient sur la fonction de coût (4.20). Le jeu de supervision compte 20 000 rayons normaux vérifiés par lien : les décalages δ sont concentrés à 75 % dans la coquille critique $[1, 10]$ mm, cohérente avec la marge $d_{\text{safe}} = 15$ mm, et étendus jusqu'à 50 mm ; un candidat n'est retenu que si sa distance exacte recalculée coïncide avec le décalage à 0,25 mm près et que la direction exacte du gradient s'écarte de moins de 5° de la normale de départ. Le tableau B.1 rassemble les valeurs numériques.

Tableau B.1 Hyperparamètres d'entraînement du modèle de distance du robot.

Symbole	Description	Valeur
<i>Régression de valeur (protocole de [80])</i>		
—	points près de la surface / uniformes	8×10^5 / 8×10^5
$\mathbf{B}_0$	initialisation des moindres carrés récursifs	$10^2 \mathbf{I}$
—	itérations récursives par lien	80
<i>Raffinement géométrique (contribution)</i>		
—	rayons normaux vérifiés par lien	20 000
δ	décalages : coquille critique / maximum	75 % dans $[1, 10]$ mm / 50 mm
—	tolérances de vérification (distance, direction)	0,25 mm, 5°
$\lambda_{\text{ray}}, \lambda_{\text{eik}}$	poids des rayons et du terme eikonal	2, 0,05
$\lambda_{\text{dir}}, \lambda_{\text{vec}}$	poids d'alignement du gradient	0,2, 0,25
ω	pondération de proximité	$1 + 4e^{-\bar{d}/\bar{\delta}}$
—	optimiseur, ordonnancement	Adam, recuit en cosinus
—	époques de raffinement	1500